

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.TTNT

SINH VIÊN THỰC HIỆN

Mã sinh viên: 25520974

Họ và tên: Hoàng Lê Kim Lâm

TÊN ĐỀ TÀI:

Cypher Defense — Game Tower Defense với Thuật toán Pathfinding



CÁC NỘI DUNG CẦN BÁO CÁO:

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

Bối cảnh (Lore): Người chơi vào vai Admin Hệ thống Data Center của trường đại học (UIT). Kẻ địch không phải là con người, mà là một Siêu AI nổi loạn (Rogue AI) đang liên tục gửi các mã độc (Malware) để phá hủy Máy chủ cốt lõi (Main Server)

Luật chơi cơ bản của các màn: Bản đồ mê cung . Malware sinh tại ô SPAWN, di chuyển qua mê cung tới SERVER ở trung tâm. Người chơi đặt Tower lên ô WALL bằng tiền thưởng để ngăn chặn. Có 2 loại quái chính: quái chuyên phá server và quái chuyên tháp bảo vệ. Server mất HP mỗi khi Malware đến nơi — HP về 0 là thua. Tiêu diệt hết tất cả wave là thắng (wave cuối là đánh boss). Có tổng cộng 5 màn chơi. Mỗi màn sẽ có độ khó khác nhau, thêm các thử thách, độ khó mỗi boss khác nhau (mỗi boss đều có skill

đặc biệt), điều kiện khắt khe hơn, quái vật càng về càng được nâng cấp có thêm nhiều skill, nhiều sự kiện (như bom rơi gây nổ map, choáng server) xảy ra, buộc người chơi xử lý kịp thời.

b. Các CTDL và thuật toán tự cài đặt sử dụng đến hiện tại

CustomLinkedList (core/data_structures.py) [1]

What: Danh sách liên kết đơn với Node(value, next). Hỗ trợ append_head, append_tail, pop_head ($O(1)$). Why: Lưu Malware.path — chuỗi ô cần đi. Mỗi frame gọi pop_head() lấy ô tiếp theo. LinkedList phù hợp hơn list vì chỉ truy cập đầu danh sách, không cần random access.

CustomStack (core/data_structures.py) [1]

What: Stack trên mảng động, push/pop ở cuối mảng. Why: Làm Game.undo_stack. Ctrl+Z gọi pop() lấy thao tác xây Tower gần nhất để hoàn tác — đúng nguyên lý LIFO.

CustomQueue (core/data_structures.py) [1]

What: Queue với mảng và con trỏ _front, tự thu gọn khi _front vượt nửa mảng. Why: Dùng trong BFS làm frontier (FIFO đảm bảo duyệt theo chiều rộng). Cũng dùng trong WaveSpawner để quản lý hàng đợi spawn theo thứ tự. Cũng như trong quản lý đạn bắn từ Malware.

CustomMinHeap (core/data_structures.py) [1]

What: Binary heap tối thiểu trên mảng, push(tuple)/pop() theo priority = tuple[0], dùng _bubble_up/_bubble_down. Why: (1) A* dùng làm open set — luôn mở rộng node có f-score nhỏ nhất, đảm bảo đường tối ưu. (2) Tower "min_dist" targeting — push(khoảng_cách_đến_server, malware), pop() ra kẻ gần Server nhất.

CustomMaxHeap (core/data_structures.py) [1]

What: Bọc CustomMinHeap bằng đảo dấu priority (push -value[0]) — không cài lại heap. Why: IceWall targeting "max_hp" — pop() cho ra Malware nhiều máu nhất để làm chậm kẻ cứng nhất trước khi BasicNode tiêu diệt.

GridGraph (core/graph.py)

What: Đồ thị lưới 2D với 5 loại ô WALL/PATH/TOWER/SERVER/SPAWN.
get_neighbors() 4 chiều chỉ trả ô walkable. get_spawn_weight() trả trọng số spawn theo Manhattan distance đến Server. Why: Cấu trúc map trung tâm — tất cả pathfinding và placement logic đều đọc/ghi qua đây.

Thuật toán A* (core/pathfinding.py) [2]

What: Tìm đường ngắn nhất; open set = CustomMinHeap, heuristic = Manhattan+g-score lưu trong dict. Trả về CustomLinkedList. Why: Trojan, Worm cần đường ngắn nhất đến Server. A* với heuristic Manhattan đảm bảo tối ưu trong lưới 4 chiều. Chạy lại cho mọi Malware khi kích hoạt Partition skill.

Thuật toán BFS (core/pathfinding.py) [3]

What: Duyệt chiều rộng dùng CustomQueue, không heuristic. find_nearest_tower() cũng dùng BFS. Trả về CustomLinkedList. Why: Spyware/Ransomware tìm Tower gần nhất — tất cả cạnh bằng nhau nên BFS tối ưu và đơn giản hơn A*. Quét đường đi có thể bắt được cho Tower

SpatialHash (systems/spatial_hash.py)

What: Chia lưới thành bucket 2×2 ô, dùng dict{(brow,bcol): list[malware]}.
insert/remove/update_position/query_range. Why: Tower cần tìm Malware trong tầm bắn mỗi frame (60fps). Duyệt toàn bộ Malware là $O(\text{towers} \times \text{malwares})$. SpatialHash chỉ kiểm tra bucket lân cận $\rightarrow$ gần $O(1)$ trung bình.

N – ary Tree (systems/upgrade_tree.py)

What: nó là cây nhiều nhánh. Why: sử dụng cấu trúc dữ liệu để xây dựng hệ thống cây nâng cấp.

Upper Bound (thay vì MergeSort , thay đổi so với bản đề xuất đồ án sau khi xem xét)

What: Tìm phần tử đầu tiên > giá trị đang xét. Why : Dùng trong tổng kết leaderboard. Vì mỗi lần chèn vào LeaderBoard đều được sắp xếp hết nên Sort lại là không cần thiết mà chỉ là tìm vị trí thôi nên dùng UpperBound.

c. Quá trình thực hiện

Tuần 1: Xây dựng hệ thống thuật toán, cấu trúc dữ liệu, các class entity và cơ chế game cơ bản.

Thuật toán, cấu trúc dữ liệu: Xây dựng toàn bộ thư mục core/ — chưa có Pygame, chỉ Python thuần. Có thêm thư mục system.

core/data_structures.py: Cài đặt 6 CTDL từ đầu: Node, CustomLinkedList (append_head/tail, pop_head), CustomStack (push/pop), CustomQueue (enqueue/dequeue với _front pointer), CustomMinHeap (_bubble_up/_bubble_down), CustomMaxHeap (bọc MinHeap với đảo dấu). Không dùng heapq hay collections. (tự tạo cấu trúc dữ liệu chứ không dùng thư viện)

core/graph.py: Class Celltype (5 hằng số) và GridGraph — load_from_list() đọc JSON, get_neighbors() 4 chiều, is_tower_placeable() kiểm tra WALL + ngoài server_radius, get_spawn_weight() tính trọng số theo Manhattan distance.

core/pathfinding.py: heuristic() Manhattan, reconstruct_path() truy vết ngược trả LinkedList xuôi, astar() dùng MinHeap, bfs() dùng Queue, find_nearest_tower() BFS phát hiện ô PATH kẻ Tower gần nhất. Kiểm tra thủ công qua Python shell — A*, BFS, LinkedList hoạt động đúng.

systems/spatial_hash.py: _hash() ánh xạ (row,col)→(brow,bcol), insert/remove/update_position/query_range đều viết tay, không dùng dict có sẵn ngoài làm lỗi lưu trữ.

Xây dựng entity và cơ chế game cơ bản

entities/malware.py

Malware (lớp cha) — chứa logic chung cho mọi kẻ thù: pos, hp/max_hp, speed, move_timer, reward, path (CustomLinkedList).

Cơ chế di chuyển **delta-time**: mỗi frame cộng dt vào move_timer, khi đủ 1/speed giây thì gọi path.pop_head() lấy ô tiếp theo. Dùng while thay if để tốc độ không bị ảnh hưởng khi FPS thay đổi. draw() vẽ hình tròn + thanh máu theo tỉ lệ hp/max_hp. apply_slow(factor, duration) nhân tốc độ với factor, đếm ngược tự phục hồi.

Trojan — `_calculate_path()` gọi `astar()` đến Server, luôn đi đường ngắn nhất, bỏ qua Tower.

Worm — Cũng dùng `astar()`, nguy hiểm vì tốc độ cao.

Spyware — `_calculate_path()` gọi `find_nearest_tower()` + `bfs()` đến ô PATH kề Tower gần nhất để phá. Không có Tower thì fallback `astar()` đến Server. Tự tính lại đường mỗi khi Tower thay đổi.

Ransomware(Spyware) Kế thừa hoàn toàn hành vi Spyware, chỉ ghi đè stats. Khó nhất: vừa tanky vừa phá Tower.

entities/tower.py

Tower (lớp cha) — `pos` (ô WALL), `range`, `damage`, `fire_rate`, `fire_timer`, `targeting`.

`update(dt, candidates)`: cộng `dt` vào `fire_timer`, đủ `cooldown` thì gọi `_find_target()` + `shoot()`. `_find_target()`: nếu `"min_dist"` dùng **CustomMinHeap** chọn Malware gần Server nhất; nếu `"max_hp"` dùng **CustomMaxHeap** chọn Malware nhiều máu nhất. `shoot()` tạo và trả về `Projectile` (import bên trong hàm để tránh circular import).

BasicNode : Tháp cơ bản, ưu tiên kẻ sắp phá Server.

IceWall : Override `shoot()`: gọi `apply_slow(0.5, 2.0)` trước khi bắn — làm chậm kẻ nhiều máu nhất 50% trong 2 giây.

RadarNode : Override `shoot()` luôn trả `None` → không tạo đạn, chỉ phát hiện Malware trong vùng rộng. (Tiền đề để phát hiện quái tàng hình.

entities/projectile.py

Projectile — lưu tham chiếu `target`, vị trí pixel (`x`, `y`), `damage`, `speed`, `_hit`.

`update(dt)`: tính vector đến tâm pixel của `target`, chuẩn hóa, di chuyển `speed * dt` mỗi frame. Khi khoảng cách $< speed * dt$ → gọi `target.take_damage()`, đặt `_hit = True`. `game.py` dùng `has_hit()` để xóa `Projectile` đã kết thúc khỏi danh sách.

game.py: Vòng lặp 60fps — `load_level()` đọc JSON tạo GridGraph+SpatialHash, `_update_spawner()` spawn Malware theo trọng số, `_update_malwares()` cập nhật SpatialHash + xử lý chết/đến server, `_update_towers()` query_range + Heap targeting + bắn, `place_tower()` kiểm tra `is_tower_placeable` + push `undo_stack` + recalc Spyware path, `undo()` pop stack + restore WALL + recalc, `draw()` 7 layer.

settings.py: Toàn bộ hằng số — `CELL_SIZE=48`, `FPS=60`, `GRID 15×15`, màu sắc, tham số HP/speed/reward cho 4 Malware (cơ bản khởi đầu cho sự nâng cấp các quái khác trong 4 nhánh quái cơ bản này), `cost/range/damage/fire_rate` cho 3 Tower.(cơ bản, tiền đề cho hệ thống nâng cấp tháp tăng sức mạnh sau này sau này.)

Tuần 2: Xây dựng hệ thống đồ họa Sprite, nâng cấp toàn bộ entities và render pipeline

ui/sprites.py — File hoàn toàn mới (liên quan đến UI) [5]

Tạo hệ thống cache sprite: `_cache` dict lưu toàn bộ Surface(ảnh frame các chuyển động của các entity) đã load. `init()` gọi sau `pygame.display.set_mode()` để `convert_alpha()` hoạt động đúng. `get(name)` trả về Surface hoặc list[Surface] từ cache. `action_frame_count(key)` trả về số frame của animation.

Map tiles — `_load_map_tiles()`: Load 2 ảnh PATH và 3 ảnh WALL từ `data/sprites/Map/`. WALL được dựng pseudo-3D: cắt 20px đáy ảnh → kéo thành thân tường cao `TALL_FACTOR×CELL_SIZE`, phủ overlay màu tối → ghép vào Surface `tall_wall`. PATH tải ảnh thực, phủ dark overlay nhẹ để trông tối hơn. Cả hai lưu dạng list để `game.py` chọn ngẫu nhiên tạo họa tiết không lặp.

Tower sprites: BasicNode, IceWall, RadarNode — scale đến `TOWER_SCALE×CELL_SIZE`. Projectile sprites: BasicNode và IceWall từ PNG. Server: 4-frame pulse animation dùng `_pulse_tint()` (sin-based tint mỗi frame). Portal spawn: 14-frame sprite sheet 100×100px.

Trojan sprites — Mushroom(tượng trưng cho quái trojan) sprite sheets (80×64 px/frame): Mushroom-Run.png (8 frames), Mushroom-Idle.png (7), Mushroom-Attack.png (10), Mushroom-Hit.png (5), Mushroom-Die.png (15), Mushroom-Stun.png

(18), Mushroom-AttackWithStun.png (24). Scale $1.8 \times \text{CELL_SIZE}$. Dùng hàm `_sheet(path, fw=80, fh=64, max_frames, target)` để cắt sheet.

Worm sprites — Plant3 (tượng trưng cho quái worm) sheets (64×64 px/frame, 8 cột $\times$ 4 dòng = 4 hướng): Walk(6), Run(8), Attack(7), Idle(4), Death(10), Hurt(5). Sau khi load toàn bộ frame, chia thành 4 hướng up/down/left/right. Cache riêng từng cặp: `worm_Run_right`, `worm_Attack_down`... Scale $1.8 \times \text{CELL_SIZE}$.

Cũng như tương tự các con Malware khác.

entities/malware.py — Chỉnh sửa thêm

Lớp cha Malware: thêm hệ thống animation (`_sprite_key`, `_anim_timer`, `_anim_frame`, `_anim_frames`, `_death_timer`). `update()` bổ sung animation tick: cứ $1/\text{ANIM_FPS}$ giây tăng `_anim_frame`, đồng thời tích lũy `_death_timer` khi `is_dead()`.

Hệ thống tấn công trực tiếp: thêm state ("`moving`" / "`attacking_server`" / "`attacking_tower`") (tuần 1 thì tắt cả mặc định đánh tháp), `attack_timer`, `_pending_hit` (dict), `goal` (ô đích), `attack_type` ("`melee`" / "`ranged`"), `attack_range`. `update()` kiểm tra state $\rightarrow$ tăng `attack_timer` $\rightarrow$ khi đủ $1/\text{attack_speed}$ $\rightarrow$ đặt `_pending_hit`. `game.py` đọc `_pending_hit` để gây damage server hoặc tower. Bổ sung `_should_attack_tower()`, `get_pending_hit()`, `clear_pending_hit()`, `is_death_animation_done()`.

`get_render_data(cell_size)`: Thay thế `draw()` cho hệ Y-sort. Trả về dict {`surf`, `pos`, `sort_y`, `type`} để `game.py` gom vào `render_list` rồi sort theo `sort_y` trước khi blit. (không vẽ các hình khối nữa và hình có chuyển động, layer sắp xếp)

`Spyware._calculate_path()`: sửa lỗi thiếu `self.goal` — trước đây (chỉ luôn tấn công server) chỉ set `self.path`, không set `self.goal` $\rightarrow$ ranged check tại `update()` luôn fail. Nay: `self.goal = attack_pos` (nếu có tower) hoặc `server_pos` (fallback). Thêm logic tìm `self._attack_target_cell` (ô TOWER kề `attack_pos`) để truyền vào `_pending_hit`.

`TrojanRanged` (class mới, kế thừa `Trojan`): `attack_type="ranged"`, `attack_range=3`. Dừng di chuyển khi `heuristic(pos, server) <= 3`. Thay vì gây damage trực tiếp, override `update()` để chuyển `_pending_hit` $\rightarrow$ `MalwareProjectile`, enqueue vào `_projectile_queue`

(CustomQueue). game.py gọi get_projectiles() mỗi frame để thu thập và thêm vào self.projectiles.

Các con malware được cập nhật đồ họa lấy từ file sprites.py

entities/projectile.py — Sửa thành hierarchy 3 lớp (do có thêm đạn của quái)

BaseProjectile (lớp cha mới): chứa logic chung — vị trí pixel (x, y), speed, damage, _hit, tower_type. draw() render sprite hoặc vẽ procedural nếu không có sprite. Riêng trojan_ranged dùng _draw_trojan_ranged_projectile() thiết kế đạn riêng .

Projectile (Tower → Malware, giữ nguyên homing behavior): kế thừa BaseProjectile. update(dt) tính vector đến target mỗi frame (homing), gọi target.take_damage() khi dist < speed×dt, đặt _hit=True khi target chết.(để xóa đạn luôn)

MalwareProjectile (mới — Ranged Malware → Server/Tower): bay thẳng đến goal_pos cố định (không homing). apply_hit(game): nếu goal_type=="server" → game.server_hp -= damage, nếu goal_type=="tower" → game._get_tower_at(goal_pos).take_damage() → gọi game._on_tower_destroyed() nếu tower phá vỡ.

entities/tower.py — Thêm HP system, _path_can_shoot(), sprite

Thêm self.hp / self.max_hp

_path_can_shoot(): BFS từ vị trí tower (pos) duyệt toàn bộ ô PATH có thể đến được (dùng CustomQueue), lưu vào dict self.path. _find_target() lọc candidates theo candidate.pos in self.path trước khi dùng Heap — đảm bảo chỉ bắn malware đang đi trên PATH mà Tower có thể nhắm được (có nghĩa từ vị trí quái đó có thể đến vị trí tháp , tháp không bắn những quái không có khả năng đi qua tháp được, thêm độ khó phải xác định chính xác đường giải mê cung, không bắn xuyên tường.)

Thêm self.tower_type (overridden: "basic"/"ice"/"radar") để Projectile biết dùng sprite nào. Thêm get_render_data(cell_size) trả về dict Y-sort tương tự Malware. Import ui.sprites để lấy tower sprite. Import CustomQueue cho _path_can_shoot().

game.py

`__init__`: thêm `sprites.init()` (sau `set_mode()`). Thêm `_portal_timer`, `_portal_frame`, `_server_timer`, `_server_frame` cho animation portal và server. `MALWARE_FACTORY`: bổ sung `"trojan_ranged"`: `TrojanRanged`.

`load_level()`: thêm `_pre_wave=True`, `_pre_wave_timer=PRE_WAVE_DURATION` (đếm ngược 10s). `self.portal=[]` lưu danh sách ô spawn hiện tại. Gọi `graph.get_spawn_weight()` lưu `weights_and_cells` cho random portal placement.

`_start_wave()`: đọc `num_portals` từ wave JSON. Chuyển portal cũ về `PATH`, `random.choices()` chọn `k=num_portals` ô mới theo spawn weight, `set_cell(SPAWN)`. Malware spawn ngẫu nhiên vào một trong các ô `SPAWN` đang active.

Hệ thống `MalwareProjectile`: `_update_malwares()` gọi `malware.get_projectiles()` cho mỗi `TrojanRanged`, append vào `self.projectiles`. `_update_projectiles()` gọi `proj.apply_hit(self)` khi `has_hit()` — polymorphic, hoạt động đúng cho cả `Projectile` (tower) và `MalwareProjectile` (malware ranged).

Tower destroy: thêm `_on_tower_destroyed(tower)` — xóa tower khỏi `self.towers`, `set_cell(WALL)`, recalc path cho tất cả malware đang sống. `_get_tower_at(pos)` tra cứu tower theo vị trí.

`_draw_grid()` — Y-Sort render pipeline: (1) Vẽ sàn `PATH` toàn bộ map với `random.seed(42)` để đa dạng tile nhưng ổn định. (2) Gom `Wall`, `Malware`, `Tower`, `Server`, `Portal` vào `render_list` dạng dict `{surf, pos, sort_y, type}`. (3) `sort(key=lambda o: o["sort_y"])` — cái nào `sort_y` nhỏ hơn (gần trên) vẽ trước. (4) Vẽ bóng ellipse (alpha 80) dưới chân mỗi đối tượng trước khi blit sprite. Tạo cảm giác giả 3D

data/levels/level1.json — Mở rộng bản đồ và wave config: Grid $15 \times 15 \rightarrow 25 \times 25$: đường đi dài hơn, nhiều nhánh hơn, tạo chiến thuật đặt tháp phong phú hơn. Thêm trường `"portals"` mỗi wave (số cổng spawn ngẫu nhiên). Thêm `"trojan_ranged"` vào danh sách enemies từ wave 2. Tăng `server_radius` để vùng cấm xây tháp quanh server phù hợp map lớn hơn.

Tuần 3: chủ yếu dựa vào nền logic game xây ở week 1 và 2 thì week 3 chủ yếu kế thừa để tạo nên đa dạng quái, tháp, các cơ chế đặc biệt.

TUẦN 3 - FILE MỚI (8 files):

entities/bomb.py

Explosion projectile với animation lifecycle (cơ chế đặc biệt đầu tiên nổ là choáng tất cả trụ, trừ 100Hp server)

- Attributes: pos, damage, radius, frame_index, is_alive, is_detonated
- Methods: update(dt), is_dead(), is_exploded(), draw(screen)

entities/fire_mark.py

Vết lửa tại vị trí target (hiệu ứng và cơ chế ô lửa khi tháp lửa tấn công)

- Attributes: pos, duration, damage_per_sec, radius, age
- Methods: update(dt), is_dead(), draw(screen)
- Cơ chế: gây damage = damage_per_sec * dt mỗi frame

entities/server.py (sửa từ biến cố định ở game.py thành class)

Server là win condition (mục tiêu cuối)

- Attributes: pos, hp, max_hp, posion_timer(thời gian bị tấn công bởi độc)
- Methods: take_damage(amount), is_destroyed(), draw(screen), update(dt)(trừ máu theo thời gian bởi độc)
- Game over nếu hp ≤ 0

entities/wall.py: Tường map (Cơ chế đặt tường tạm thời, nhưng chỉ mới tạo class chưa áp vào game chính thức)

- Attributes: pos, color
- Methods: draw(screen)

systems/upgrade_tree.py (N ary tree: Hệ thống nâng cấp sử dụng CTDL cây)

Quản lý cây nâng cấp với node parent-child

- Lớp SkillNode:

Attributes: node_id, name, cost, is_unlocked, parent, children

Methods: add_child(), can_be_unlocked(money), unlock()

get_available_upgrades(current_id): trả về list child nodes

- Hàm tạo cây: (có 3 cây chính tương ứng 3 tháp chính ban đầu)

1. create_basic_upgrade_tree()

Root: BasicNode (50 tiền)

Tầng 1: Nâng cấp dạng nâng cấp chỉ số (damage hoặc tốc độ bắn)

Tầng 2: Nếu tầng 1 nâng damage: FireNode (100), SniperNode (150), tầng 1 nâng tốc độ đánh: SpeedNode (120)

2. create_ice_upgrade_tree()

Root: IceWall (80 tiền)

Tầng 1: Nâng chỉ số dạng nâng làm chậm hoặc tăng tầm bắn

Tầng 2: FreezeNode (150) (nếu tăng làm chậm) , SpreadNode (130) (nếu tăng tầm bắn)

3. create_radar_upgrade_tree()

Root: RadarNode (120 tiền)

Tầng 1: Nâng chỉ số phạm vi quét

Tầng 2: PoisonNode (200)

ui/upgrade_menu.py (liên quan phần UI hiển thị) Menu nâng cấp Tower

Attributes: tower, upgrade_tree, current_node_id, available_upgrades, is_visible

Methods:

- show() / hide(): hiển thị/ẩn menu

- get_clicked_upgrade(pos): kiểm tra click trúng nút

- try_upgrade(upgrade_node, money): thực hiện nâng cấp

- draw(screen): vẽ menu + nút với unlock status check

TUẦN 3 - FILE CẬP NHẬT (7 files):

entities/tower.py

Có 8 tower subclasses:

Với 3 tháp cơ bản đã code từ trước

BasicNode (Tower cơ bản, min_dist targeting)

IceWall (Tower làm chậm, slow effect)

RadarNode (Tower phát hiện, không bắn)

Các tháp dựa trên cây nâng cấp

FireNode (Tower lửa, đốt - kế thừa BasicNode). game.py khi projectile hit → tạo FireMark tạo ô lửa đốt malware nào đi qua

SniperNode (Tower xạ thủ - kế thừa BasicNode) damage đơn mục tiêu lớn

SpeedNode (Tower tăng tốc - kế thừa BasicNode) tháp bắn liên thanh

FreezeNode (Tower đóng băng - kế thừa IceWall) Ghi đè: slow_factor = 0.0 (quái không di chuyển)

SpreadNode (Tower lây lan slow - kế thừa IceWall) Thêm: spread_range (bán kính lan slow) game.py khi hit → find malware trong radius → apply_slow all

Thêm một số chỉ số cho Tower:

Thêm slow_timer, stunned_timer tracking

original_fire_rate: lưu base fire_rate để khôi phục

apply_slow(factor, duration): fire_rate *= factor

entities/malware.py

- Malware base class

- 5 subclasses: Trojan, Worm, Spyware, Ransomware, TrojanRanged

Sửa lại đồ họa của Trojan, TrojanRanged, Worm, Spyware (thay sprite mới)

Week 3 thêm 5 malware ranged classes MỚI:

PoisonWorm: kế thừa từ worm thêm khả năng tấn công độc lên server (logic vẫn giống chỉ thêm khả năng tấn công có độc)

SlowSpy: kế thừa từ SpyWare thêm khả năng làm chậm tốc độ bắn của tháp .

Spyware_Ranged(Spyware bắn projectile). Kế thừa Spyware nhưng thay vì melee attack. Bắn MalwareProjectile đến Tower hoặc Server. get_projectiles() method: trả về list projectiles tạo trong frame. Override update(): thay vì melee damage, tạo projectile

SlowSpy_Ranged(SlowSpy bắn projectile + slow . Kế thừa SlowSpy nhưng bắn projectile. Projectile vẫn áp dụng slow effect khi trúng Tower. Override update(): tạo projectile + slow effect

VaultWare: kế thừa từ ransomware có khả năng 50% sẽ hút đạn từ các tower về phía mình (như vai trò hỗ trợ), tác động bằng cách tác động trực tiếp method find target của tower.

entities/projectile.py Week 3 cập nhật:

- Hỗ trợ tower_type="fire" → tạo FireMark trên hit
- Hỗ trợ tower_type="spread" → spread_range, slow_factor, slow_duration
- Hỗ trợ malware_type cho MalwareProjectile

game.py (logic game đa số đã hợp lí rồi) Week 3 cập nhật:

- Cập nhật các loại hit(kèm hiệu ứng) đa dạng hơn để tower biết mình đang dính hiệu ứng gì)
- Thêm cơ chế bom rơi random mọi nơi trên map , ở khoảng thời gian bất kì.
- Thêm upgrade_tree integration: 3 nhánh
- Thêm UpgradeMenu integration: click tower → show menu

- Xử lý projectile hit → tạo FireMark (nếu tower_type="fire")
- Xử lý projectile hit → spread slow (nếu tower_type="spread")

settings.py (Bổ sung các setting cho các class mới)

ui/sprites.py Week 3 cập nhật:

Cập nhật đồ họa Trojan, worm với spyware mới. Thêm đồ họa các class mới vào, xử lý frames tương tự.

- Fire mark sprite: khắc phục frame size 64×64 → 48×48
- Tower sprite sheets: basic, ice, fire, sniper, speed, freeze, spread, poison
- Malware sprite sheets: trojan, worm, spyware, ransomware, spyware_ranged, slowspy_ranged,...

Tuần 4: hoàn thiện game hoàn chỉnh với Boss system, malware mới, cơ chế Shock Spread BFS, 5 level đầy đủ, GameManager state machine, bảng xếp hạng binary search và AudioManager. Cân bằng lại các chỉ số trong setting.

TUẦN 4 - FILE CẬP NHẬT (4 files):

main.py (cập nhật đồ họa màn hình chơi game và quản lí màn chơi)

Viết lại hoàn chỉnh thành GameManager với state machine đầy đủ:

- States: START → PLAYING → LEVEL_COMPLETE / GAME_OVER → VICTORY
- Menu chính với background PNG; Tower Codex (bảng giới thiệu tất cả tháp).
- Leaderboard: chèn điểm bằng binary search upper_bound tìm kiếm nhị phân, đảm bảo danh sách luôn tăng dần. (phần này việc dùng thuật toán sắp xếp như bản đề xuất đề án thì nó không hiệu quả vì kết quả sẽ được thêm liên tục nên không cần phải sắp xếp lại mà là tìm vị trí)
- Confirm dialog trước New Game.
- Save/load progress.json: lưu level đã mở khóa, load lại khi mở lại game.

game.py

Cập nhật game loop chính với các tính năng tuần 4:

- MALWARE_FACTORY mở rộng lên 13 entry (thêm LightSpy, LightSpy_Ranged, RiposteWare).
- BOSS_FACTORY: fireworm, flyingdemon, riposteboss, shadow, final (5 boss với class mapping). Cập nhật cách update boss (đưa boss vào game và xử lý các skill của boss tương tác với các entity khác)
- ShockEffect integration: khi tháp bị stun bởi shock_spread() → tạo ShockEffect hiển thị animation.
- Sửa cơ chế undo dùng ctrl Z để trở lại trạng thái trước nếu đặt nhảm tháp là stack chỉ tồn tại trong 3s còn lại là phá tháp.
- RightPanel HUD kết nối upgrade_tree: click tháp → hiện panel nâng cấp bên phải.
- audio_manager.play_sound() tại các sự kiện: bắn, quái chết, boss xuất hiện.
- Thêm cơ chế camera, có nghĩa bây giờ game full màn hình và để tạo sống động chơi chơi thì dùng phím WASD để di chuyển map.
- thêm self.invisible_duration là thời gian tàng hình, xuất hiện ngẫu nhiên, thời gian được tải từ file json của màn đó. Và cứ gán self.invisible_duration cho malware.invisible thì đã có cơ chế tàng hình

entities/malware.py

- Thêm cơ chế tàng hình bằng cách thêm các attr là self.invisible (thời gian tàng hình của quái, mặc định là 0), self.in_radar_range (kiểm tra xem có trong vùng range của radar không). Thì lúc cập nhật đồ họa nếu self.invisible > 0 và not self.in_radar_range thì không cập nhật đồ họa cho **game.py** xử lý từ xóa luôn đồ họa trong khoảng thời gian đó. Cũng như sửa trong cơ chế **tower.py** là chỉ shoot những quái không tàng hình. Các quái mà trong range của radar thì quái đó để True.

Thêm 3 malware mới tuần 4:

- LightSpy (kế thừa Spyware): khi melee attack tháp → kích hoạt shock_spread() lan điện BFS sang tháp liền kề. (có xác suất làm stun tháp)

- LightSpy_Ranged (kế thừa LightSpy): phiên bản tầm xa — bắn MalwareProjectile, khi trúng tháp gây shock.

- RiposteWare (kế thừa Ransomware): 50% xác suất phản đòn — projectile tháp bắn trúng sẽ bay ngược về tháp nguồn thay vì gây sát thương.

ui/sprites.py

Thêm SpriteCache entries cho các entities tuần 4:

- Boss: fireworm_Walk, fireworm_Attack, fireworm_Die; flyingdemon; shadow; riposteboss; final.

- Malware mới: lightspy, lightspy_ranged, ripostaware.

- Effect: shock_effect frames.

TUẦN 4 - FILE MỚI (5 files):

entities/boss.py

Boss system phân cấp — Boss (lớp cơ sở) + 5 subclass với cơ chế riêng:

- Attributes chính : hp, max_hp, pos, graph, path, speed, original_speed, slow_timer, slow_factor, reward

- Methods: take_damage(damage), apply_slow(factor, duration), update(dt), get_render_data(cell_size)

- **RiposteBoss (Level 1):** 50% xác suất phản đòn projectile ngược về tháp nguồn; 50% nhận sát thương bình thường. (dùng random khi mà nó nhận đạn, thì có nên bắn lại đạn vào tháp không)

- **FireWorm (Level 2):** Boss tấn công tất cả tháp trong phạm vi mỗi duration_attack_tower giây + lan cháy (khiến các ô xung quanh tháp đó không thể đặt tháp). get_tower_attacks() trả về list dict để game.py xử lý; get_server_attack() khi chạm server.

- **FlyingDemon (Level 3):** Thả bomb ngẫu nhiên mỗi bomb_duration giây (thêm attr bomb_duration và 1 bộ đếm để khi cứ update là cộng thêm và đến thời gian sẽ gọi class

Bomb)— 75% Bomb thường (-300 HP), 25% Atomic bomb (-10000 HP, thua game ngay khi nổ).

- **Shadow (Level 4):** Roll invincibility mỗi 5 giây (tốc độ $\times 3$, nhận sát thương = 0 trong thời gian roll). (cũng thêm các attr về đếm ngược thời gian roll và đến roll thì tăng chỉ số lên, roll= True). Khi boss này xuất hiện thì mọi malware sẽ rơi vào trạng thái tàng hình. (để invisible_duration=1e9)

- **Final (Level 5):** Di chuyển 1 đường thẳng (không có thuật toán tìm đường), phá hủy ô 3×3 cell khi gặp vật cản, spawn RiposteWare, 25% reflect sát thương lên server, thả bomb mỗi 15 giây. Boss này chủ yếu dùng các kĩ thuật đã làm ở các boss trước đó

entities/shock_effect.py

Hiệu ứng điện giật hiển thị trên tháp khi bị shock lan truyền.

- Attributes: pos (tuple), age (float), duration (float)
- Methods: update(dt), is_alive() -> bool, draw(screen, cell_size)

systems/shock_spread.py

Hàm shock_spread() — lan truyền điện theo BFS qua các tháp liền kề trên lưới.

- shock_spread(graph, start_pos, max_hops, towers_dict): BFS từ start_pos qua các TOWER cell cách nhau không quá 1 cell, gây damage tất cả tháp kết nối.
- Dùng CustomQueue (CTDL tự cài đặt trong core/data_structures.py) — không dùng collections.deque.
- Trả về các cell trong vùng shock

systems/audio.py [6]

AudioManager singleton — quản lý âm thanh nền và sound effect toàn game.

- Attributes: _sounds (dict[str, pygame.Sound]), _music_playing (str | None)
- Methods: play_sound(name), play_music(name, loops=-1), stop_music()
- Khởi tạo: audio_manager = AudioManager() ở module level, import dùng chung.

ui/right_panel.py

Panel HUD bên phải — cây nâng cấp trực quan và thông số tháp đang chọn.

- Attributes: rect, tower, upgrade_tree, current_node_id, font, small_font
- Methods: set_tower(tower, tree, node_id), draw(screen, money), handle_click(pos, money) -> bool
- Vẽ cây nâng cấp dạng node-edge đồ thị; highlight node hiện tại; hiển thị thông số tháp real-time.

d. Kết quả đạt được

Kết quả tuần 1: Game chạy hoàn chỉnh — Malware di chuyển đúng A*/BFS, Tower phát hiện và bắn qua SpatialHash+Heap, IceWall làm chậm đúng, Spyware tự recalc path khi Tower thay đổi, Ctrl+Z undo hoạt động, GAME OVER/WIN hiển thị đúng. Đồ họa tạm thời dùng hình tròn và hình chữ nhật.

Kết quả tuần 2:

Đồ Họa Sprite Hoàn Chỉnh

Toàn bộ 5 loại Malware, 3 loại Tower, Server và Portal đều được hiển thị bằng hình ảnh **Sprite PNG thực tế**. Loại bỏ hoàn toàn các hình khối (tròn/chữ nhật) tạm thời, mang lại giao diện trực quan sinh động.

Hệ Thống State Machine & Animation

- **Trojan:** Sở hữu 7 trạng thái hành động (Chạy, Nghỉ, Tấn công, Bị trúng đòn, Chết, Choáng, Tấn công kèm choáng).
- **Worm:** 6 hành động kết hợp 4 hướng; Sprite tự động xoay theo hướng di chuyển.
- **TrojanRanged:** 5 hành động riêng biệt.
- **Hiệu ứng:** Malware chỉ bị xóa khỏi bộ nhớ sau khi hoàn tất toàn bộ hiệu ứng diễn hoạt khi chết (Die animation).

Kẻ Địch Mới: TrojanRanged

- Khả năng **tấn công từ xa** (cắm Server/Tower 3 ô).
- Bắn ra các **Projectile (đạn)** có hiệu ứng phát sáng (Glow) và đuôi (Trail) bay thẳng đến mục tiêu.

- Đã sửa lỗi self.goal=None để đảm bảo cơ chế tấn công tầm xa hoạt động ổn định.
- **Bản Đồ 25x25 Thực Tế (mê cung thiết, không còn demo nữa)**
- Sử dụng Tilemap cho đường đi (PATH) và tường (WALL) từ file PNG.
- **Hiệu ứng Pseudo-3D:** Tường có độ cao thực tế với TALL_FACTOR = 1.5 (kết hợp Texture thân tường và lớp phủ Overlay).
- Bản đồ được tạo ngẫu nhiên nhưng đảm bảo tính nhất quán bằng **Seed cố định**.

Pipeline Render Y-Sort 2.5D

- Hệ thống tự động sắp xếp tất cả đối tượng theo tọa độ Y (sort_y).
- **Quy tắc hiển thị:** Sprite ở hàng dưới sẽ đè lên hàng trên, tạo cảm giác chiều sâu không gian.
- Mỗi đối tượng đều có **bóng hình Elip (Alpha)** dưới chân để tăng độ chân thực.

Cổng Portal Động

- Vị trí xuất hiện cổng được chọn ngẫu nhiên theo trọng số (spawn weight) trước mỗi đợt tấn công (Wave).
- Hệ thống **đếm ngược 10 giây** để người chơi chuẩn bị trước khi đợt Malware mới bắt đầu.

Độ Bền Của Trụ (Tower HP)

Các trụ phòng thủ hiện đã có lượng máu riêng và có thể bị phá hủy:

- **BasicNode:** 100 HP
- **IceWall:** 80 HP
- **RadarNode:** 60 HP
- **Cơ chế:** Khi một trụ bị phá hủy, hệ thống sẽ tự động **tính toán lại đường đi (recalc path)** cho toàn bộ Malware.

Cấu Trúc Projectile (Đạn) Phân Cấp

Hệ thống đạn được tổ chức logic:

- BaseProjectile: Lớp cơ sở.
- Projectile: Đạn đuổi mục tiêu (Homing).
- MalwareProjectile: Đạn bay thẳng, áp dụng cơ chế đa hình apply_hit để xử lý va chạm.

Thuật Toán Bắt Của Trụ (BFS Path)

Sử dụng hàm _path_can_shoot() dựa trên thuật toán BFS:

Chỉ nhắm mục tiêu Malware đang di chuyển trên đường đi có mà trụ quét được.

Không thể bắn xuyên qua các path mà không thông được.

Kết quả tuần 3: Hoàn hiện các kĩ năng quái, tháp, cơ chế đặc biệt

Cơ chế đặc biệt : Bomb và tường tạm thời

Hệ thống Tower (8 loại tower)

- BasicNode: Tower cơ bản
- IceWall: Tower làm chậm
- RadarNode: Tower phát hiện
- FireNode: Tower lửa
- SniperNode: Tower xạ thủ
- SpeedNode: Tower tăng tốc
- FreezeNode: Tower đóng băng
- SpreadNode: Tower lây lan slow
- PoisonNode: Tower độc

Hệ thống Upgrade Tree (3 nhánh) (sử dụng CTDL cây nhiều nhánh)

- BasicNode tree → FireNode/SniperNode/SpeedNode
- IceWall tree → FreezeNode/SpreadNode
- RadarNode tree → PoisonNode
- Mechanics: parent-child constraint, cost progression, unlock check

Tower Mechanics

- Slow effect: $\text{fire_rate} \times \text{slow_factor}$, set slow_timer
- Stun effect: $\text{stunned_timer} > 0 \rightarrow \text{skip shooting}$
- Fire rate restore: $\text{fire_rate} = \text{original_fire_rate}$ khi slow expire

Malware System - Ranged Attacks (5 loại MỚI)

- Spyware_Ranged: Bắn projectile đến Tower hoặc Server
- SlowSpy_Ranged: Bắn projectile + áp dụng slow effect
- VaultWare: Thu hút đạn
- PoisonWorm: gây gộc lên server
- SpyWare: tạo slow effect

Kết quả tuần 4: Game hoàn chỉnh — 5 level đầy đủ, boss system, 13 loại malware, cơ chế Shock Spread BFS, GameManager, leaderboard và audio.

Boss System (5 loại Boss):

- FireWorm: AoE burn attack tất cả tháp trong phạm vi, kích hoạt mỗi `duration_attack_tower` giây.
- FlyingDemon: Thả bomb ngẫu nhiên (75% thường / 25% atomic — thua ngay nếu nổ atomic).
- Shadow: Roll invincibility + tốc độ $\times 3$ mỗi 5 giây, miễn sát thương khi roll.
- RiposteBoss: 50% phản đòn projectile ngược về tháp bắn, 50% nhận sát thương.
- Final Boss: Phá ô 3×3 cell $\rightarrow$ recalc path mọi malware; spawn RiposteWare; reflect lên server; thả bomb liên tục.

Malware System (13 loại đa dạng chiến thuật):

- Tuần 4 thêm: LightSpy (shock spread BFS), LightSpy_Ranged (shock tầm xa), RiposteWare (phản đòn 50%).
- Tổng hệ thống: Trojan, Worm, Spyware, Ransomware, TrojanRanged, PoisonWorm, SlowSpy, Spyware_Ranged, SlowSpy_Ranged, VaultWare + 3 mới tuần 4.

Cơ Chế Shock Spread (CTDL — BFS trên lưới tháp):

- LightSpy tấn công tháp $\rightarrow$ `shock_spread()` BFS qua TOWER cells liền kề dùng CustomQueue.

5 Level Hoàn Chính (level1.json → level5.json): Độ khó tăng dần; mỗi level 5 wave; wave cuối mỗi level có boss đặc trưng. Level 1: riposteboss; Level 2: fireworm ; Level 3: flyingdemon; Level 4: shadow; Level 5: final.

GameManager State Machine (main.py):

- START → PLAYING → LEVEL_COMPLETE / GAME_OVER → VICTORY.
- Tower Codex (xem thông số tháp); Leaderboard binary search insert (CTDL); save/load progress.json.

Audio & HUD:

- AudioManager singleton: nhạc nền menu/gameplay + sound effect sự kiện.
- RightPanel HUD: cây nâng cấp trực quan, thông số tháp real-time.

e. Tài liệu tham khảo

[1] VisuAlgo. Linked List, Stack, Queue, Heap — Interactive Visualization.

<https://visualgo.net/en>

→ Tham khảo: cấu trúc hoạt động của LinkedList (append/pop), Stack (LIFO), Queue (FIFO), MinHeap (bubble-up/bubble-down) để định hướng tự cài đặt trong core/data_structures.py.

[2] Red Blob Games — Amit Patel. Introduction to A* Pathfinding (2014, updated 2020). <https://www.redblobgames.com/pathfinding/a-star/introduction.html>

→ Tham khảo: nguyên lý hoạt động A* trên lưới 2D, rồi tự cài đặt và áp dụng.

[3] Algorithm Visualizer. BFS / Graph Traversal. <https://algorithm-visualizer.org/graph/breadth-first-search>

→ Tham khảo: các bước duyệt BFS từ đó cài đặt bfs với mỗi tác dụng khác nhau.

[4] Pygame Community. Pygame Documentation — pygame.draw, pygame.event, pygame.time. <https://www.pygame.org/docs/>

→ Tham khảo: API vẽ (draw.rect, draw.circle), xử lý sự kiện (KEYDOWN, MOUSEBUTTONDOWN), đồng hồ (clock.tick) cho vòng lặp game trong game.py.

[5] itch.io. Game Assets — Free & Commercial 2D/Pixel Art Sprites.

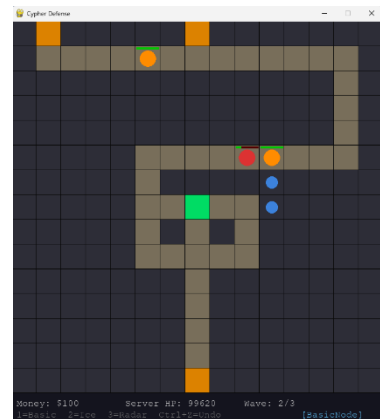
<https://itch.io/game-assets> → Tham khảo: tải sprite sheet nhân vật và môi trường

(Mushroom, Plant3, Enemy3, tower sprites, tilemap PNG) dùng cho hệ thống animation tuần 2.

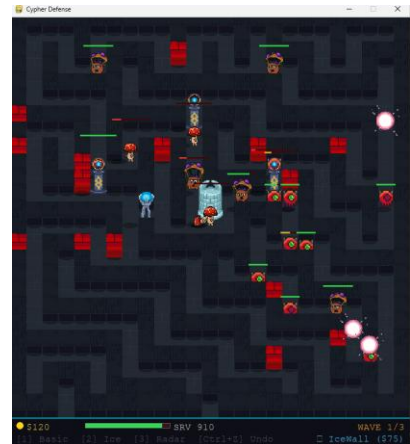
[6] Freesound Project, "Freesound — Collaborative database of Creative Commons Licensed sounds," [Online]. Available: <https://freesound.org/>.

f. Phụ lục 1: Giới thiệu (demo) kết quả

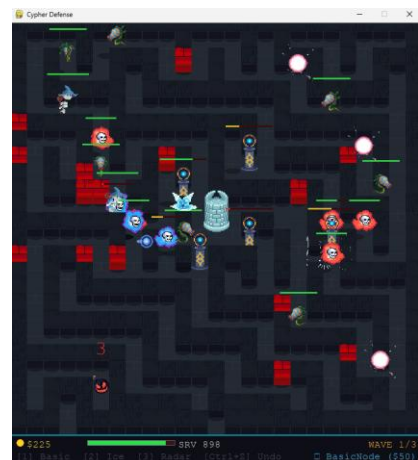
Tuần 1: kết quả chỉ trạng thái game đơn sơ chỉ mô phỏng được cơ chế bắn, di chuyển, các class tương tác với nhau (chưa có gameplay, cốt truyện hoàn chỉnh, UI còn chỉ là các hình khối đơn giản để mô phỏng xem chương trình có chạy và có bug gì không, chưa có tính độ khó, thử thách, chỉ dừng mức đặt tháp , bắn quái, mất máu)



Tuần 2: Nâng cấp đồ họa chính, chỉnh sửa game logic hợp lí, thêm các tính năng liên quan đến độ khó.



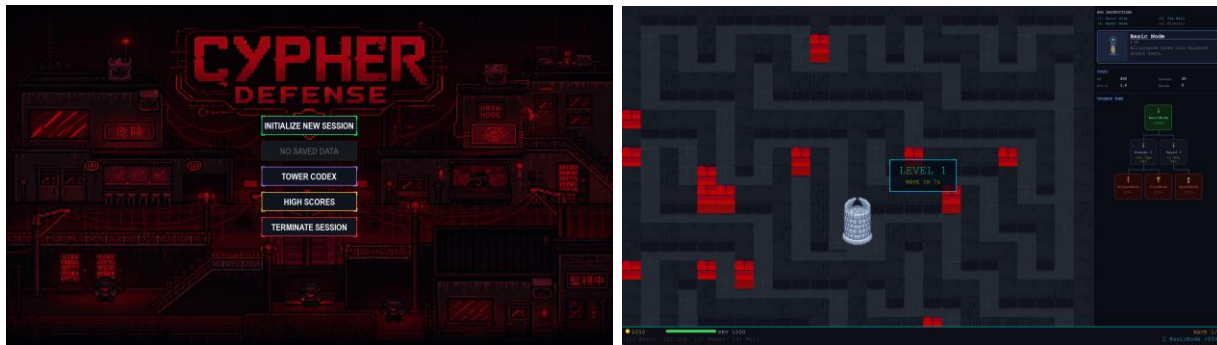
Tuần 3: thêm quái mới , tháp mới



Tuần 4: game hoàn chỉnh — boss system
(FireWorm/FlyingDemon/Shadow/RiposteBoss/Final), Shock Spread BFS qua CustomQueue, 5 level hoàn chỉnh, GameManager state machine, leaderboard binary search, AudioManager, RightPanel HUD.

Link demo video chơi:

https://drive.google.com/drive/folders/19aSSi_xTE8GAjyq8LKPl4IU1y4W5hfw?usp=sharing



j. PHỤ LỤC 2: DOCSTRING CÁC LỚP CHÍNH

Link github chứa các file liên quan:

https://github.com/HoangLeKimLam/cypher_defense

Dưới đây là toàn bộ docstring của các lớp và phương thức trong dự án Cypher Defense, mô tả trách nhiệm và giao diện công khai của từng thành phần. Docstring tuân theo định dạng chuẩn: mô tả chức năng, Attributes, Args, Returns, Side effects, Note, Usage.

Cấu trúc thư mục dự án

```
cypher_defense/
├── core/
│   ├── __init__.py
│   ├── data_structures.py    # LinkedList, Stack, Queue, MinHeap, MaxHeap
│   ├── graph.py              # Celltype, GridGraph
│   └── pathfinding.py         # astar, bfs, find_nearest_tower
├── entities/
│   ├── __init__.py
│   └── malware.py            # Malware + 13 subclasses
```



```

|   └─ boss.py                # Boss + 5 subclasses
(FireWorm/FlyingDemon/Shadow/RiposteBoss/Final)
|   └─ tower.py               # Tower + 9 subclasses
|   └─ projectile.py          # BaseProjectile, Projectile, MalwareProjectile
|   └─ fire_mark.py           # FireMark (DoT hieu ung lua)
|   └─ bomb.py                # Bomb (no thuong + atomic)
|   └─ server.py              # Server (win condition)
|   └─ wall.py                # Wall (tuong tam thoi)
|   └─ shock_effect.py        # ShockEffect (hieu ung dien)
└─ systems/
|   └─ __init__.py
|   └─ spatial_hash.py        # SpatialHash (range query)
|   └─ upgrade_tree.py        # SkillNode, UpgradeTree, create*_upgrade_tree()
|   └─ shock_spread.py        # shock_spread() BFS lan dien qua thap lien ke
|   └─ audio.py               # AudioManager singleton (nhac nen + SFX)
└─ ui/
|   └─ __init__.py
|   └─ sprites.py             # SpriteCache singleton
|   └─ upgrade_menu.py        # UpgradeMenu UI
|   └─ right_panel.py         # RightPanel HUD (cay upgrade + stats)
└─ data/
|   └─ sprites/
|   └─ levels/
|       └─ level1.json        # Level 1: 5 waves, boss riposteboss
|       └─ level2.json        # Level 2: 5 waves, boss fireworm + flyingdemon
|       └─ level3.json        # Level 3: 5 waves, boss shadow
|       └─ level4.json        # Level 4: 5 waves, boss riposteboss
|       └─ level5.json        # Level 5: 5 waves, boss final
|   └─ audio/
|       └─ background.ogg     # Nhac nen gameplay (loop)
|       └─ explosion.wav      # Tieng bom no (moi loai)
|       └─ firenode.wav       # Tieng pha huy FireNode
|       └─ freezeNode.wav     # Tieng pha huy FreezeNode
|       └─ spreadNode.wav     # Tieng pha huy SpreadNode
|       └─ warning.wav        # Canh bao bom roi / tang hinh
└─ settings.py               # Hang so toan cuc + cau hinh audio
└─ game.py                   # Game loop chinh + Undo system (stack_tower)
└─ main.py                   # GameManager + Tower Codex + Leaderboard
└─ progress.json             # Tien trinh nguoi choi (tu sinh khi choi)

```

File: core/data_structures.py

Lớp Node (đòng 1)

Đại diện cho một node trong linked list.

Attributes:

value: Dữ liệu lưu trong node (thường là tuple như (row, col)).
next (Node | None): Con trỏ tới node tiếp theo (None nếu là node cuối).

Usage:

Không dùng trực tiếp – CustomLinkedList quản lý Node nội bộ::

```
linked = CustomLinkedList()
linked.append_tail((3, 5))
```

Phương thức `__init__(self, value)` (dòng 14)

Khởi tạo Node với giá trị cho trước.

Args:

value: Dữ liệu cần lưu (thường là tuple (row, col) cho pathfinding).

Lớp CustomLinkedList (dòng 24)

Linked List tự cài đặt (singly linked list) – dùng để lưu path trong game.

Dùng làm hàng đợi path cho Malware: `reconstruct_path()` thêm vào tail,
`Malware.update()` pop từ head từng ô một theo thứ tự đi.

Attributes:

head (Node | None): Node đầu danh sách.
tail (Node | None): Node cuối danh sách.
size (int): Số phần tử hiện tại.

Usage:

`astar()` và `bfs()` trả về CustomLinkedList chứa path từ start đến goal::

```
path = astar(graph, spawn_pos, server_pos)
next_cell = path.pop_head() # lấy ô tiếp theo để đi
```

Phương thức `__init__(self)` (dòng 42)

Khởi tạo LinkedList rỗng.

Phương thức `append_head(self, value)` (dòng 48)

Thêm phần tử vào đầu danh sách.

Args:

value (tuple): Dữ liệu cần thêm (thường là (row, col)).

Note:

Độ phức tạp $O(1)$. `reconstruct_path()` dùng `append_head()` để xây path theo thứ tự xuôi (truy vết ngược từ goal $\rightarrow$ start).

Phương thức `append_tail(self, value)` (dòng 67)

Thêm phần tử vào cuối danh sách.

Args:

value (tuple): Dữ liệu cần thêm (thường là (row, col)).

Note:

Độ phức tạp $O(1)$ nhờ con trỏ tail.

Phương thức `pop_head(self)` (dòng 87)

Lấy và xóa phần tử đầu danh sách.

Returns:

tuple: Giá trị của node đầu (thường là (row, col)) nếu tồn tại.

None: Nếu danh sách rỗng.

Note:

Độ phức tạp $O(1)$. `Malware.update()` gọi mỗi bước di chuyển.

Phương thức `is_empty(self)` (dòng 109)

Kiểm tra danh sách có rỗng không.

Returns:

bool: True nếu `size == 0`, False nếu còn phần tử.

Phương thức `__len__(self)` (dòng 117)

Trả về số phần tử trong danh sách.

Returns:

int: Giá trị `self.size`.

Lớp `CustomStack` (dòng 126)

Stack (LIFO) tự cài đặt – dùng để lưu lịch sử xây tower (Ctrl+Z).

Nguyên lý: push vào cuối list, pop từ cuối list $\rightarrow O(1)$ cả hai chiều.

`game.py` dùng để lưu mỗi action xây tower; Ctrl+Z gọi `pop()` để hoàn tác.

Attributes:

data (list): Danh sách nội bộ lưu các phần tử.

Usage:

`game.py` lưu action xây tower và hoàn tác::

```
undo_stack = CustomStack()
undo_stack.push({"pos": (row, col), "tower": tower, "cost": cost})
action = undo_stack.pop() # Ctrl+Z
```

Phương thức `__init__(self)` (dòng 143)

Khởi tạo Stack rỗng.

Phương thức `push(self, value)` (dòng 147)

Thêm phần tử vào đỉnh stack.

Args:

value: Dữ liệu cần push (bất kỳ kiểu nào).

Note:

Độ phức tạp $O(1)$ – append vào cuối list Python.

Phương thức `pop(self)` (dòng 158)

Lấy và xóa phần tử ở đỉnh stack.

Returns:

Phần tử ở đỉnh (LIFO) nếu stack còn phần tử.

None nếu stack rỗng.

Note:

Độ phức tạp $O(1)$.

Phương thức `peek(self)` (dòng 172)

Xem phần tử đỉnh stack nhưng không xóa.

Returns:

Phần tử ở đỉnh nếu stack còn phần tử.

None nếu stack rỗng.

Phương thức `is_empty(self)` (dòng 183)

Kiểm tra stack có rỗng không.

Returns:

bool: True nếu không có phần tử nào, False nếu còn phần tử.

Phương thức `__len__(self)` (dòng 191)

Trả về số phần tử trong stack.

Returns:

int: Số lượng phần tử hiện tại.

Lớp `CustomQueue` (dòng 200)

Queue (FIFO) tối ưu bằng kỹ thuật lazy slicing – dùng cho BFS và spawn queue.

Không dùng `pop(0)` ($O(n)$). Thay vào đó dùng pointer `_front` để track vị trí đầu.

Khi `_front > len/2` thì slice lại list để thu hồi bộ nhớ (amortized $O(1)$).

Attributes:

`_data` (list): Danh sách nội bộ lưu các phần tử.

```
_front (int): Chỉ số phần tử đầu hàng đợi trong _data.
```

Usage:

BFS trong pathfinding.py và SpatialHash::

```
queue = CustomQueue()
queue.enqueue((3, 5))
cell = queue.dequeue()
```

Phương thức `__init__(self)` (dòng 218)

Khởi tạo Queue rỗng.

Phương thức `enqueue(self, value)` (dòng 223)

Thêm phần tử vào cuối queue.

Args:

value: Dữ liệu cần thêm (bất kỳ kiểu nào).

Note:

Độ phức tạp $O(1)$ – append vào cuối list Python.

Phương thức `dequeue(self)` (dòng 234)

Lấy và xóa phần tử đầu queue.

Returns:

Phần tử đầu hàng đợi (FIFO) nếu còn phần tử.
None nếu queue rỗng.

Note:

Amortized $O(1)$ – dùng `_front` pointer thay vì `pop(0)`.
Khi `_front > len(_data)//2`, slice lại list để thu hồi bộ nhớ.

Phương thức `is_empty(self)` (dòng 258)

Kiểm tra queue có rỗng không.

Returns:

bool: True nếu không có phần tử nào, False nếu còn phần tử.

Phương thức `peek(self)` (dòng 266)

Xem phần tử đầu queue nhưng không xóa.

Returns:

Phần tử đầu hàng đợi nếu còn phần tử.
None nếu queue rỗng.

Phương thức `__len__(self)` (dòng 277)

Trả về số phần tử trong queue.

Returns:

int: Số lượng phần tử hiện tại (chưa dequeue).

Lớp CustomMinHeap (dòng 286)

Min Heap (Priority Queue) tự cài đặt – dùng cho A* và Tower targeting.

Lưu dưới dạng array-based binary heap: $\text{parent}[i] \leq \text{children}[i]$.
Phần tử lưu dạng tuple (priority, data) – so sánh theo priority.

Attributes:

`_data (list)`: Danh sách nội bộ lưu các tuple (priority, data).

Usage:

A* dùng để chọn ô có f-score nhỏ nhất, Tower dùng để chọn target có khoảng cách đến server nhỏ nhất::

```
heap = CustomMinHeap()
heap.push((f_score, (row, col)))
priority, cell = heap.pop()
```

Phương thức `__init__(self)` (dòng 304)

Khởi tạo MinHeap rỗng.

Phương thức `_parent(self, i)` (dòng 308)

Trả về chỉ số node cha của node tại i.

Phương thức `_left(self, i)` (dòng 312)

Trả về chỉ số node con trái của node tại i.

Phương thức `_right(self, i)` (dòng 316)

Trả về chỉ số node con phải của node tại i.

Phương thức `_swap(self, i, j)` (dòng 320)

Hoán đổi hai phần tử tại chỉ số i và j trong `_data`.

Phương thức `_bubble_up(self, i)` (dòng 324)

Đẩy phần tử tại i lên đúng vị trí trong heap (sau push).

Args:

i (int): Chỉ số phần tử vừa thêm vào.

Phương thức `_bubble_down(self, i)` (dòng 338)

Đẩy phần tử tại i xuống đúng vị trí trong heap (sau pop).

Args:

i (int): Chỉ số phần tử cần sắp xếp xuống (thường là 0).

Phương thức `push(self, value)` (dòng 362)

Thêm phần tử vào heap và duy trì tính chất heap.

Args:

value (tuple): (priority, data) – so sánh theo priority[0].

Note:

Độ phức tạp $O(\log n)$.

Phương thức `pop(self)` (dòng 374)

Lấy và xóa phần tử có priority nhỏ nhất.

Returns:

tuple: (priority, data) của phần tử nhỏ nhất.

None: Nếu heap rỗng.

Note:

Độ phức tạp $O(\log n)$.

Phương thức `peek(self)` (dòng 393)

Xem phần tử có priority nhỏ nhất nhưng không xóa.

Returns:

tuple: (priority, data) của phần tử đỉnh heap.

None: Nếu heap rỗng.

Phương thức `is_empty(self)` (dòng 404)

Kiểm tra heap có rỗng không.

Returns:

bool: True nếu không có phần tử nào.

Phương thức `__len__(self)` (dòng 412)

Trả về số phần tử trong heap.

Returns:

int: Số lượng phần tử hiện tại.

Lớp `CustomMaxHeap` (dòng 421)

Max Heap xây dựng dựa trên CustomMinHeap – dùng cho Tower targeting "max_hp".

Ý tưởng: đảo dấu priority ($x \rightarrow -x$) khi push/pop, dùng lại toàn bộ logic sắp xếp của MinHeap.

Attributes:

`_heap` (CustomMinHeap): MinHeap nội bộ lưu các tuple `(-priority, data)`.

Usage:

Tower với targeting="max_hp" dùng để chọn malware có HP nhiều nhất::

```
heap = CustomMaxHeap()
heap.push((malware.hp, malware))
priority, target = heap.pop()
```

Phương thức `__init__(self)` (dòng 438)

Khởi tạo MaxHeap rỗng (bọc MinHeap nội bộ).

Phương thức `push(self, value)` (dòng 442)

Thêm phần tử vào heap với priority đảo dấu.

Args:

value (tuple): (priority, data) – priority sẽ được lưu âm trong MinHeap.

Phương thức `pop(self)` (dòng 450)

Lấy và xóa phần tử có priority lớn nhất.

Returns:

tuple: (priority, data) với priority dương (đã đảo lại dấu).
None: Nếu heap rỗng.

Phương thức `peek(self)` (dòng 460)

Xem phần tử có priority lớn nhất nhưng không xóa.

Returns:

tuple: (priority, data) với priority dương.
None: Nếu heap rỗng.

Phương thức `is_empty(self)` (dòng 470)

Kiểm tra heap có rỗng không.

Returns:

bool: True nếu không có phần tử nào.

Phương thức `__len__(self)` (dòng 478)

Trả về số phần tử trong heap.

Returns:

int: Số lượng phần tử hiện tại.

File: `core/graph.py`

Lớp `Celltype` (dòng 4)

Hằng số kiểu ô trong lưới map – dùng thống nhất giữa graph, pathfinding và game.

Attributes:

WALL (int): 0 – ô tường, không thể đi qua, tower đặt ở đây.


```

PATH (int): 1 – ô đường đi, malware di chuyển trên đây.
TOWER (int): 2 – ô tường có tower, không walkable.
SERVER (int): 3 – ô server, malware đi đến đây gây damage.
SPAWN (int): 4 – ô điểm spawn, walkable (malware xuất hiện ở đây).

```

Usage:

```

from core.graph import GridGraph, Celltype
graph.set_cell(row, col, Celltype.TOWER)
if graph.get_cell(row, col) == Celltype.PATH:
    ...

```

Lớp GridGraph (dòng 27)

Lưới ô vuông biểu diễn map game – lõi dữ liệu dùng cho pathfinding và rendering.

Mỗi ô có kiểu Celltype (WALL/PATH/TOWER/SERVER/SPAWN). GridGraph cung cấp các method để kiểm tra, thay đổi ô và tính toán spawn weights.

Attributes:

```

row (int): Số hàng của lưới.
col (int): Số cột của lưới.
grid (list[list[int]]): Ma trận 2D lưu Celltype của từng ô.
spawn_pos (list[tuple]): Danh sách vị trí (row, col) của các ô SPAWN.
server_pos (tuple | None): Vị trí (row, col) của ô SERVER.
PLATEAU_THRESHOLD (float): Ngưỡng khoảng cách để tính spawn weight (từ JSON).
server_radius (int): Bán kính Chebyshev quanh server không cho đặt tower (từ JSON).

```

Usage:

```

game.py tạo GridGraph và load từ JSON::

graph = GridGraph(rows, cols)
graph.load_from_list(config["grid"], config)
path = astar(graph, spawn_pos, graph.server_pos)

```

Phương thức __init__(self, row, col) (dòng 50)

Khởi tạo GridGraph với toàn bộ ô là WALL.

Args:

```

row (int): Số hàng của lưới.
col (int): Số cột của lưới.

```

Phương thức in_bounds(self, x, y) (dòng 65)

Kiểm tra tọa độ (x, y) có nằm trong phạm vi lưới hay không.

Args:

```

x (int): Hàng cần kiểm tra.
y (int): Cột cần kiểm tra.

```

Returns:
bool: True nếu $0 \leq x < \text{row}$ và $0 \leq y < \text{col}$.

Phương thức `is_walkable(self, x, y)` (dòng 77)

Kiểm tra malware có thể đi qua ô (x, y) không.

Các ô walkable: PATH, SERVER, SPAWN. WALL và TOWER không walkable.

Args:
x (int): Hàng cần kiểm tra.
y (int): Cột cần kiểm tra.

Returns:
bool: True nếu ô trong phạm vi và là PATH/SERVER/SPAWN.

Phương thức `set_cell(self, x, y, celltype)` (dòng 95)

Đặt kiểu ô tại (x, y) và cập nhật spawn_pos/server_pos nếu cần.

Args:
x (int): Hàng của ô cần thay đổi.
y (int): Cột của ô cần thay đổi.
celltype (int): Kiểu ô mới (hằng số Celltype).

Side effects:
- Cập nhật `self.grid[x][y]`.
- Append (x, y) vào `self.spawn_pos` nếu `celltype == SPAWN`.
- Đặt `self.server_pos = (x, y)` nếu `celltype == SERVER`.

Phương thức `get_cell(self, x, y)` (dòng 115)

Lấy kiểu ô tại tọa độ (x, y).

Args:
x (int): Hàng của ô cần lấy.
y (int): Cột của ô cần lấy.

Returns:
int: Giá trị Celltype của ô (0-4).
None: Nếu (x, y) ngoài phạm vi lưới.

Phương thức `get_neighbors(self, x, y)` (dòng 130)

Lấy danh sách các ô lân cận walkable của ô tại (x, y).

Kiểm tra 4 hướng: phải, xuống, trái, lên. Chỉ trả về ô walkable (PATH/SERVER/SPAWN) – WALL và TOWER bị loại.

Args:
x (int): Hàng của ô hiện tại.
y (int): Cột của ô hiện tại.

Returns:

list[tuple]: Danh sách (nx, ny) của các ô lân cận có thể đi qua.

Phương thức `is_tower_placeable(self, x, y)` (dòng 151)

Kiểm tra có thể đặt tower tại (x, y) không.

Điều kiện: ô phải là WALL VÀ không nằm trong vùng bảo vệ server (Chebyshev distance $\leq$ server_radius).

Args:

x (int): Hàng của ô cần kiểm tra.

y (int): Cột của ô cần kiểm tra.

Returns:

bool: True nếu có thể đặt tower, False nếu không hợp lệ.

Phương thức `load_from_list(self, grid_list, level_config)` (dòng 174)

Load bản đồ từ danh sách 2D (đọc từ JSON) và cấu hình level.

Args:

grid_list (list[list[int]]): Ma trận 2D giá trị Celltype từ JSON.

Giá trị: 0=WALL, 1=PATH, 3=SERVER, 4=SPAWN.

level_config (dict): Dict cấu hình level – đọc plateau_threshold và server_radius từ đây.

Side effects:

- Cập nhật PLATEAU_THRESHOLD và server_radius từ level_config.
- Gọi set_cell() cho từng ô để cập nhật grid, spawn_pos, server_pos.

Phương thức `get_all_path_cells(self)` (dòng 194)

Lấy danh sách tất cả ô walkable kết nối đến server bằng BFS.

Dùng BFS từ server_pos để tìm tất cả ô PATH/SERVER/SPAWN có thể đến được từ server. Dùng để tính spawn weights.

Returns:

list[tuple]: Danh sách (row, col) của tất cả ô walkable kết nối.

Phương thức `get_spawn_weight(self)` (dòng 215)

Tính trọng số spawn cho từng ô PATH dựa trên khoảng cách đến server.

Ô càng xa server → trọng số càng cao → dễ được chọn làm spawn hơn.

Ô vượt ngưỡng PLATEAU_THRESHOLD $\times$ max_dist → trọng số = 1.0 (tối đa).

Ô gần server hơn → trọng số = dist / max_dist (giảm dần).

Returns:

list[tuple]: Danh sách (weight, (row, col)) cho từng ô PATH.

```
game.py dùng với random.choices(cells, weights=weights) để  
chọn vị trí spawn ngẫu nhiên có trọng số.
```

Note:

Công thức spawn weight từ CLAUDE.md:

$\text{dist} \geq \text{plateau_threshold} \times \text{max_dist} \rightarrow \text{weight} = 1.0$

$\text{dist} < \text{plateau_threshold} \times \text{max_dist} \rightarrow \text{weight} = \text{dist} / \text{max_dist}$

Phương thức `manhattan_distance(a, b)` (dòng 236)

Tính khoảng cách Manhattan giữa hai ô lưới.

Args:

`a` (tuple[int,int]): Tọa độ ô thứ nhất (row, col).

`b` (tuple[int,int]): Tọa độ ô thứ hai (row, col).

Returns:

int: Tổng chênh lệch hàng và cột.

File: `core/pathfinding.py`

Phương thức `heuristic(r1, c1, r2, c2)` (dòng 8)

Tính Manhattan distance – ước tính chi phí còn lại trong A*.

Args:

`r1` (int): Hàng vị trí hiện tại.

`c1` (int): Cột vị trí hiện tại.

`r2` (int): Hàng vị trí đích.

`c2` (int): Cột vị trí đích.

Returns:

int: Khoảng cách Manhattan $|r1-r2| + |c1-c2|$.

Note:

Dùng Manhattan distance vì malware chỉ đi 4 hướng (không chéo).

Heuristic này admissible (không ước tính quá) $\rightarrow$ A* luôn tìm đường ngắn nhất.

Phương thức `reconstruct_path(came_from, start, goal)` (dòng 27)

Truy vết ngược từ goal về start và trả về CustomLinkedList theo thứ tự xuôi.

Args:

`came_from` (dict): $\{(r,c): (r_prev, c_prev)\}$ – bản đồ truy vết từ A*/BFS.

`start` (tuple): Vị trí bắt đầu (row, col).

`goal` (tuple): Vị trí đích (row, col).

Returns:

CustomLinkedList: Path từ START đến GOAL theo thứ tự đi, bao gồm cả start.

Note:

Dùng `append_head()` để xây path theo thứ tự xuôi khi truy vết ngược.
Cả A* lẫn BFS đều gọi hàm này sau khi tìm xong đường.

Phương thức `astar(graph, start, goal)` (dòng 53)

Tìm đường ngắn nhất từ start đến goal dùng thuật toán A*.

Dùng CustomMinHeap làm priority queue với $f = g + h$.
Heuristic: Manhattan distance đến goal.

Args:

- `graph (GridGraph)`: Bản đồ lưới đang dùng trong game.
- `start (tuple)`: Vị trí (row, col) spawn malware.
- `goal (tuple)`: Vị trí (row, col) đích (thường là `server_pos`).

Returns:

- CustomLinkedList: Path từ start đến goal theo thứ tự đi.
- CustomLinkedList rỗng nếu không tìm được đường.

Usage:

Trojan/Worm gọi khi spawn và khi bản đồ thay đổi::

```
malware.path = astar(graph, spawn_pos, graph.server_pos)
```

Phương thức `bfs(graph, start, goal)` (dòng 97)

Tìm đường ngắn nhất từ start đến goal dùng BFS (không có heuristic).

Dùng CustomQueue thay CustomMinHeap – mỗi bước đi có chi phí bằng nhau nên BFS đảm bảo tìm đường ngắn nhất mà không cần heuristic.

Args:

- `graph (GridGraph)`: Bản đồ lưới đang dùng trong game.
- `start (tuple)`: Vị trí (row, col) bắt đầu.
- `goal (tuple)`: Vị trí (row, col) đích (thường là ô PATH kề tower).

Returns:

- CustomLinkedList: Path từ start đến goal theo thứ tự đi.
- CustomLinkedList rỗng nếu không tìm được đường.

Usage:

Spyware/Ransomware gọi với `goal = ô PATH` kề tower gần nhất::

```
malware.path = bfs(graph, malware.pos, attack_pos)
```

Phương thức `find_nearest_tower(graph, start)` (dòng 136)

Dùng BFS tìm ô PATH kề tower gần nhất từ vị trí start.

BFS lan rộng từ start trên các ô walkable. Khi tìm thấy ô có ít nhất một ô láng giềng là TOWER → trả về ô PATH đó (không phải ô TOWER).

Spyware/Ransomware dùng ô này làm goal của BFS path.

Args:

graph (GridGraph): Bản đồ lưới đang dùng trong game.
start (tuple): Vị trí (row, col) hiện tại của Spyware/Ransomware.

Returns:

tuple: Vị trí (row, col) ô PATH kề tower gần nhất.
None nếu không có tower nào trên map.

Note:

Trả về ô PATH kề tower (không phải ô TOWER) vì malware không thể đứng trên TOWER – chỉ đứng kề để tấn công.

File: entities/malware.py

Lớp Malware (dòng 14)

Lớp cơ sở cho tất cả loại Malware trong game.

Malware di chuyển theo đường đi tính sẵn (path), bị tháp bắn hạ, hoặc đến được SERVER để trù HP server. Mỗi subclass định nghĩa chiến lược di chuyển riêng (A* hay BFS).

Attributes chính:

pos (tuple): Vị trí hiện tại (row, col) trên lưới.
graph (GridGraph): Tham chiếu bản đồ lưới – dùng để pathfinding.
hp (int): Máu hiện tại. Khi về 0 → is_dead() trả True.
max_hp (int): Máu tối đa – dùng tính tỉ lệ thanh máu khi vẽ.
speed (float): Số ô di chuyển mỗi giây (vd. 2.0 = 2 ô/giây).
move_timer (float): Bộ đếm tích lũy dt để biết khi nào bước tiếp.
reward (int): Tiền người chơi nhận khi tiêu diệt malware này.
reached_server (bool): True khi malware đã chạm ô SERVER.
color (tuple): Màu RGB vẽ hình tròn đại diện.
path (CustomLinkedList): Hàng đợi các ô cần đi – kết quả từ A*/BFS.

Usage:

Không khởi tạo trực tiếp – dùng subclass Trojan/Worm/Spyware/Ransomware.
game.py tạo malware, gọi update(dt) mỗi frame, draw(screen, cell_size) để vẽ.
Kiểm tra is_dead() và has_reached_server() sau mỗi update() để xử lý kết quả.

Phương thức __init__(self, pos, graph) (dòng 39)

Khởi tạo Malware tại vị trí pos với giá trị mặc định.

Subclass ghi đè hp, speed, reward, color, rồi gọi _calculate_path().
Không gọi pathfinding ở đây – để subclass quyết định thuật toán.

Args:

```
pos (tuple): (row, col) vị trí ban đầu – nên là ô PATH hoặc SPAWN.  
graph (GridGraph): Bản đồ lưới dùng cho pathfinding.
```

Phương thức `_should_attack_tower(self)` (dòng 83)

Kiểm tra malware có nên chuyển sang trạng thái tấn công tower không.

Mặc định trả False – chỉ Spyware/Ransomware ghi đè để kiểm tra xem malware đã đứng kề tower chưa.

Returns:

bool: False (lớp cơ sở). Subclass ghi đè trả True khi đứng kề tower.

Phương thức `get_pending_hit(self)` (dòng 94)

Trả về thông tin đòn tấn công đang chờ xử lý trong frame này.

Returns:

dict: {"type": "server"/"tower", "dmg": int} nếu có đòn chờ, hoặc dict rỗng {} nếu không có.

Note:

game.py đọc sau mỗi malware.update(dt) để xử lý damage, rồi gọi clear_pending_hit() để tránh tính đòn hai lần.

Phương thức `clear_pending_hit(self)` (dòng 107)

Xóa thông tin đòn tấn công sau khi game.py đã xử lý.

Side effects:

- Đặt self._pending_hit về dict rỗng {}.

Phương thức `_calculate_path(self)` (dòng 114)

Tính đường đi đến đích và lưu vào self.path.

Mặc định dùng A* đến server_pos. Subclass ghi đè để chọn thuật toán khác.

Gọi lại mỗi khi bản đồ thay đổi (đặt/xóa tower, kích hoạt Partition).

Sau tính xong → set self.goal = mục tiêu.

Returns:

None – kết quả lưu vào self.path (CustomLinkedList).

Phương thức `update(self, dt)` (dòng 128)

Cập nhật vị trí malware mỗi frame theo kỹ thuật delta-time.

Tích lũy dt vào move_timer. Khi đủ lớn ($\geq 1/\text{speed}$), pop ô tiếp theo từ path và di chuyển đến đó. Dùng while thay vì if để xử lý đúng khi FPS thấp (cần bước nhiều ô trong cùng 1 frame).

Args:

dt (float): Thời gian frame tính bằng giây, do game.clock.tick() cung cấp.

Side effects:

- Cập nhật self.pos khi di chuyển.
- Đặt self.reached_server = True nếu đến ô server_pos.
- Pop ô khỏi self.path mỗi bước.
- Khôi phục self.speed khi hết hiệu ứng slow.

Note:

game.py phải lưu old_pos TRƯỚC khi gọi update() để cập nhật SpatialHash.

Ranged malware (attack_type=="ranged"): while loop kiểm tra khoảng cách đến goal mỗi bước. Nếu remaining_dist <= attack_range → break khỏi while, đặt state thành "attacking_server" hoặc "attacking_tower", không di chuyển tiếp.

Phương thức take_damage(self, amount) (dòng 207)

Trừ máu malware khi bị đạn bắn trúng.

Args:

amount (int): Lượng sát thương cần trừ (≥ 0).

Side effects:

Giảm self.hp. Không clamp về 0 – is_dead() xử lý điều kiện đó.

Usage:

Gọi bởi Projectile.update() khi đạn chạm target:
target.take_damage(self.damage)

Phương thức on_hit_by_projectile(self, projectile) (dòng 223)

Hook gọi khi malware bị đạn trúng. Cho phép subclass xử lý riposte hoặc effect đặc biệt.

Args:

projectile (Projectile): Đạn vừa trúng.

Side effects:

Mặc định: gọi self.take_damage(projectile.damage).
Subclass có thể override để xử lý riposte, reflect, v.v.

Phương thức apply_slow(self, factor, duration) (dòng 235)

Áp dụng hiệu ứng làm chậm từ IceWall tower.

Giảm tốc độ còn (factor × tốc độ gốc) trong duration giây, sau đó tự khôi phục về _original_speed trong update().

Args:

factor (float): Hệ số nhân tốc độ. 0.5 = còn 50%, 0.3 = còn 30%.
duration (float): Thời gian hiệu ứng tính bằng giây.

Side effects:

- Cập nhật `self.speed = _original_speed * factor`.
- Đặt `self._slow_time = duration` (đếm ngược trong `update()`).

Usage:

Gọi bởi `IceWall.shoot()` ngay trước khi tạo `Projectile`:

```
target.apply_slow(self.slow_factor, self.slow_duration)
```

Phương thức `is_dead(self)` (dòng 257)

Kiểm tra malware đã hết máu chưa.

Returns:

bool: True nếu `self.hp ≤ 0`, False nếu còn sống.

Usage:

game.py kiểm tra sau mỗi frame để xóa malware và cộng tiền:

```
if malware.is_dead():
    self.money += malware.reward
    dead.append(malware)
```

Phương thức `is_death_animation_done(self)` (dòng 273)

Kiểm tra animation chết đã phát xong chưa để game.py xóa malware.

Lớp cơ sở trả True ngay lập tức (xóa malware ngay khi `hp ≤ 0`).

Subclass ghi đè để trì hoãn xóa cho đến khi animation chết kết thúc.

Returns:

bool: True nếu có thể xóa malware khỏi danh sách, False nếu còn đang phát animation.

Phương thức `has_reached_server(self)` (dòng 284)

Kiểm tra malware đã đến ô SERVER chưa.

Returns:

bool: True nếu `self.reached_server == True`.

Usage:

game.py kiểm tra sau mỗi frame để trừ HP server và xóa malware:

```
if malware.has_reached_server():
    self.server_hp -= 1
    dead.append(malware)
```

Phương thức `get_render_data(self, cell_size)` (dòng 297)

Tạo Surface tổng hợp (Malware + HP bar + Slow tint) trả về cho game.py để xếp lớp Y-Sort.

Surface tổng hợp cao hơn sprite 10px (chứa HP bar phía trên).

Vị trí vẽ bottom-center aligned: đáy sprite = đáy ô lưới.

Args:

`cell_size (int)`: Kích thước pixel mỗi ô lưới (`settings.CELL_SIZE`).

Returns:

```
dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "malware"}  
dùng để xếp vào render_list và sort theo sort_y trước khi vẽ.  
Trả về dict với fallback hình tròn nếu không tìm được sprite.
```

Phương thức `draw(self, screen, cell_size)` (dòng 366)

Vẽ malware lên màn hình: sprite animation + thanh máu phía trên.

Vẽ 2 thành phần:

1. Sprite animation từ cache (frame hiện tại). Fallback: hình tròn self.color nếu sprite không tìm.
2. Thanh máu ngang phía trên – nền đỏ đậm, phần còn lại màu xanh lá.

Args:

screen (pygame.Surface): Bề mặt màn hình game, do game.py truyền vào.
cell_size (int): Kích thước pixel mỗi ô lưới (settings.CELL_SIZE).

Side effects:

- Blit sprite frame (nếu tìm được) hoặc vẽ fallback circle.
- Vẽ thanh máu trên sprite.
- Vẽ slow effect (Ice VFX) nếu đang bị slow.

Note:

Quy đổi tọa độ: pixel_x = col * cell_size, pixel_y = row * cell_size.
col → x (ngang), row → y (dọc) – ngược với ký hiệu (row, col) của lưới.

Lớp Trojan (dòng 425)

Malware loại Trojan – tấn công thẳng đến Server bằng A*.

HP cao, tốc độ trung bình. Luôn tìm đường ngắn nhất đến Server, không quan tâm đến Tower. Animation states: Run, Idle, Attack, Hit, Die, Stun, AttackWithStun.

Attributes:

Kế thừa toàn bộ từ Malware. Stats lấy từ settings:
MALWARE_TROJAN_HP, MALWARE_TROJAN_SPEED, MALWARE_TROJAN_REWARD.

Usage:

Tạo bởi game.py khi JSON wave config có chuỗi "trojan":
malware = Trojan(pos=(3, 5), graph=grid_graph)
Gọi lại `_calculate_path()` khi Partition skill được kích hoạt.

Phương thức `__init__(self, pos, graph)` (dòng 441)

Khởi tạo Trojan: gọi `super().__init__`, ghi đè stats, tính đường A*.

Args:

```
pos (tuple): (row, col) vị trí spawn trên lưới.  
graph (GridGraph): Bản đồ lưới.
```

Phương thức `_calculate_path(self)` (dòng 469)

Tính đường ngắn nhất đến Server bằng A*.

Ghi đề `Malware._calculate_path()`. Gọi khi khởi tạo và khi bản đồ thay đổi (Partition skill bật/tắt).

Returns:

None – kết quả lưu vào `self.path`.

Phương thức `update(self, dt)` (dòng 481)

Override `update` để handle animation state machine.

Phương thức `_update_animation(self, dt)` (dòng 501)

Cập nhật sprite key và frame số theo state hiện tại của Trojan.

State machine: Run (bình thường) → Hit (hp thấp) → Stun (slow) → Attack (tấn công server/tower) → Die (chết).

Args:

`dt (float)`: Thời gian frame tính bằng giây.

Side effects:

- Cập nhật `self._anim_timer`, `self._anim_frame`.
- Cập nhật `self._trojan_action`, `self._sprite_key`, `self._anim_frames` theo state.
- Reset `self._anim_frame` về 0 khi action thay đổi.

Phương thức `is_death_animation_done(self)` (dòng 542)

Kiểm tra animation chết của Trojan đã phát xong chưa.

Animation Die có 13 frame ở tốc độ `ANIM_FPS` (from `Skeleton Warrior`).

Thời gian = 13 / `ANIM_FPS` giây.

Returns:

bool: True nếu còn sống (`hp > 0`) hoặc death timer $\geq$ thời gian animation.
False nếu đang phát animation chết chưa xong.

Lớp `Worm` (dòng 559)

Malware loại `Worm` – nhanh nhất nhưng ít máu. Dùng A* đến Server.

Animation states: Walk (bình thường) → Run (nhanh) → Attack (tấn công) →

Hurt (bị damage) → Idle (slow) → Death (chết).

Hỗ trợ 4-directional sprites: up, down, left, right dựa vào hướng di chuyển.

Attributes:

`_worm_action (str)`: Action animation hiện tại (Run/Walk/Attack/Hurt/Idle/Death).

```
_direction (str): Hướng di chuyển hiện tại (up/down/left/right) – chọn sprite.  
_last_pos (tuple): Vị trí frame trước để phát hiện hướng di chuyển.  
Kế thừa từ Malware. Stats lấy từ settings: MALWARE_WORM_*.
```

Usage:

```
Tạo bởi game.py khi JSON wave config có chuỗi "worm"::
```

```
malware = Worm(pos=(3, 5), graph=grid_graph)
```

Phương thức `__init__(self, pos, graph)` (dòng 578)

Khởi tạo Worm: gọi `super().__init__`, ghi đè stats, tính đường A*.

Args:

```
pos (tuple): (row, col) vị trí spawn trên lưới.  
graph (GridGraph): Bản đồ lưới.
```

Phương thức `update(self, dt)` (dòng 601)

Override update để theo dõi hướng di chuyển và xử lý animation chết.

Args:

```
dt (float): Thời gian frame tính bằng giây.
```

Side effects:

- Khi `hp <= 0`: chỉ cập nhật death animation timer, không di chuyển.
- Khi còn sống: gọi `super().update(dt)`, rồi tính hướng từ thay đổi pos.
- Cập nhật `self._direction` dựa trên dr/dc giữa pos mới và pos cũ.

Phương thức `_update_action(self)` (dòng 638)

Cập nhật action animation và sprite key dựa trên trạng thái hiện tại.

Xác định action từ `hp/state/slow`, build sprite key dạng
"worm_{action}_{direction}", fallback về "worm_Run_right" nếu thiếu sprite.

Side effects:

- Cập nhật `self._worm_action`, `self._sprite_key`, `self._anim_frames`.
- Reset `self._anim_frame` về 0 khi action thay đổi.

Phương thức `get_render_data(self, cell_size)` (dòng 681)

Override để cập nhật animation state trước khi render.

Gọi `_update_action()` để chọn đúng sprite key theo hướng và state,
rồi delegate cho `Malware.get_render_data()` để tạo Surface tổng hợp.

Args:

```
cell_size (int): Kích thước pixel mỗi ô lưới (settings.CELL_SIZE).
```

Returns:

```
dict: Render data dict giống Malware.get_render_data() -  
{"surf", "pos", "sort_y", "type"}.
```

Phương thức `is_death_animation_done(self)` (dòng 697)

Kiểm tra animation chết của Worm đã phát xong chưa.

Animation Death có 10 frame ở tốc độ ANIM_FPS.

Thời gian = 10 / ANIM_FPS giây.

Returns:

bool: True nếu còn sống (hp > 0) hoặc death timer >= thời gian animation.

False nếu đang phát animation chết chưa xong.

Phương thức `_calculate_path(self)` (dòng 712)

Tính đường ngắn nhất đến Server bằng A*.

Ghi đè `Malware._calculate_path()`. Gọi khi khởi tạo và khi

bản đồ thay đổi (Partition skill bật/tắt).

Returns:

None - kết quả lưu vào `self.path` và `self.goal`.

Lớp `WormPoison` (dòng 725)

Malware loại `WormPoison` - Worm có khả năng tiêm độc Server.

Kế thừa mọi tính năng từ Worm (nhanh, ít máu, A* đến Server).

Thêm khả năng: mỗi lần tấn công Server, tiêm độc vào Server.

Attributes:

`poison_duration` (float): Thời gian độc mỗi lần tiêm (giây). Default: 3.0s.

Usage:

Tạo bởi `game.py` khi JSON wave config có chuỗi `"worm_poison":`:

```
malware = WormPoison(pos=(3, 5), graph=grid_graph)
```

Phương thức `__init__(self, pos, graph, poison_duration)` (dòng 740)

Khởi tạo `WormPoison`: kế thừa Worm + thêm poison config.

Args:

`pos` (tuple): (row, col) vị trí spawn trên lưới.

`graph` (GridGraph): Bản đồ lưới.

`poison_duration` (float): Thời gian độc mỗi lần tiêm (giây). Default 3.0.

Phương thức `_update_action(self)` (dòng 754)

Override để sử dụng sprite `worm_poison_*` thay vì `worm_*`.

Kế thừa logic từ Worm nhưng thay đổi sprite key prefix từ `"worm_"` → `"worm_poison_"`.

Side effects:

- Cập nhật `self._sprite_key`, `self._anim_frames` dựa trên action và direction.
- Reset animation frame khi action thay đổi.

Phương thức `get_pending_hit(self)` (dòng 794)

Override để thêm `poison_duration` vào `pending_hit` khi tấn công Server.

Returns:

dict: {"type": "server"/"tower", "dmg": int, "poison": float (nếu Server)}
hoặc dict rỗng {} nếu không có đòn chờ.

Note:

game.py sẽ kiểm tra "poison" key và gọi `server.get_poison(hit["poison"])`
nếu có khi xử lý damage cho Server.

Lớp Spyware (dòng 811)

Malware loại Spyware – ưu tiên phá Tower thay vì đến Server.

Chiến lược: BFS tìm ô PATH kẻ Tower gần nhất, di chuyển đến đó.

Nếu không có tower nào trên map → fallback A* đến Server.

Buộc người chơi phải bảo vệ tower, không chỉ chặn đường.

Attributes:

`attack_pos` (tuple | None): Ô PATH kẻ Tower đang nhắm tới.
None nếu không có tower trên map.
Kế thừa từ Malware. Stats lấy từ settings: `MALWARE_SPYWARE_*`.

Usage:

Tạo bởi game.py khi JSON wave có "spyware":
`malware = Spyware(pos=(3, 5), graph=grid_graph)`
Khi game.py đặt/xóa tower → gọi `malware._calculate_path()`
để Spyware cập nhật mục tiêu mới.

Phương thức `__init__(self, pos, graph)` (dòng 830)

Khởi tạo Spyware: gọi `super().__init__`, ghi đề stats, tính đường BFS.

Args:

`pos` (tuple): (row, col) vị trí spawn.
`graph` (GridGraph): Bản đồ lưới.

Phương thức `_calculate_path(self)` (dòng 852)

Tính đường BFS đến Tower gần nhất, hoặc A* đến Server nếu không có tower.

Bước 1: `find_nearest_tower()` trả về ô PATH kẻ Tower gần nhất (hoặc None).

Bước 2: Nếu có tower → BFS đến ô đó. Nếu không → A* đến Server.

Returns:

None – kết quả lưu vào self.path và self.attack_pos.

Note:

Gọi lại mỗi khi tower được đặt/xóa hoặc Partition được kích hoạt.

Phương thức `_should_attack_tower(self)` (dòng 880)

Kiểm tra điều kiện để bắt đầu tấn công tower.

Returns:

bool: True nếu đã có mục tiêu tower và đang đứng tại ô tấn công.

Lớp `LightSpy` (dòng 889)

Malware loại `LightSpy` – kế thừa từ `Spyware`, tấn công lan sang tháp gần.

Kế thừa toàn bộ chiến lược tower-seeking từ `Spyware`.

Khác biệt: Khi tấn công 1 tháp, dùng `shock_spread` để các tháp gần cũng nhận damage.

Cơ chế: 2 tháp cách nhau ≤ 1 cell thì sẽ bị lan truyền.

Attributes:

Kế thừa từ `Spyware/Malware`. Stats lấy từ settings: `MALWARE_LIGHTSPY_*`.

Usage:

Tạo bởi `game.py` khi JSON wave có "lightspy":

```
malware = LightSpy(pos=(3, 5), graph=grid_graph)
```

Phương thức `__init__(self, pos, graph)` (dòng 904)

Khởi tạo `LightSpy`: kế thừa `Spyware` + ghi đè stats.

Args:

pos (tuple): (row, col) vị trí spawn.

graph (GridGraph): Bản đồ lưới.

Phương thức `get_pending_hit(self)` (dòng 923)

Override để trả về `pending_hit` với `affected_cells` từ `shock_spread`.

Returns:

dict: {"type": "tower", "dmg": int, "cell": tuple, "affected_cells": list}
hoặc dict rỗng {} nếu không có đòn chờ.

Note:

`affected_cells` chứa tất cả ô bị ảnh hưởng bởi shock spread.

`game.py` sẽ áp dụng damage cho tất cả tower trong `affected_cells`.

Lớp `LightSpy_Ranged` (dòng 943)

Malware loại `LightSpy_Ranged` – tấn công từ xa với shock spread effect.

Kế thừa từ `LightSpy` nhưng bắn projectile thay vì melee attack.

Mỗi projectile trúng tháp vẫn kích hoạt shock spread như `LightSpy` melee.

Dùng cùng sprite của LightSpy, projectile màu vàng sáng (gần trắng).

Attributes:

attack_type: "ranged" (đánh xa)
attack_range: 3 (ô) — dùng khi cách mục tiêu ≤ 3 ô

Phương thức `__init__(self, pos, graph)` (dòng 955)

Khởi tạo LightSpy_Ranged: kế thừa từ LightSpy, thiết lập ranged attack.

Args:

pos (tuple): (row, col) vị trí spawn trên lưới.
graph (GridGraph): Bản đồ lưới.

Side effects:

- Kế thừa toàn bộ từ LightSpy (shock spread, stats).
- Đặt attack_type = "ranged", attack_range = 3.
- Khởi tạo _projectile_queue để buffer projectiles.

Phương thức `get_projectiles(self)` (dòng 976)

Trả về danh sách MalwareProjectile mới tạo trong frame này.

game.py gọi sau mỗi update(dt) để thu thập projectile từ ranged malware.

Returns:

list: Danh sách MalwareProjectile tạo trong frame này.

Phương thức `update(self, dt)` (dòng 992)

Override update để chuyển đổi _pending_hit thành MalwareProjectile với shock spread.

Dùng attack_timer từ parent để bắn liên tục theo attack_speed.
Mỗi lần attack_timer $\geq 1/\text{attack_speed}$, parent set _pending_hit
→ thay vì melee damage, tạo projectile có shock spread effect.

Args:

dt (float): Thời gian frame tính bằng giây.

Phương thức `get_pending_hit(self)` (dòng 1045)

Override để giữ shock spread effect cho projectile.

Lấy hit từ parent, thêm affected_cells (shock spread) nếu tấn công tháp.
Khi projectile trúng, game.py sẽ xử lý affected_cells damage + 25% stun.

Returns:

dict: Hit data với keys: "type", "dmg", "cell", "affected_cells"

Lớp Ransomware (dòng 1063)

Malware loại Ransomware – phiên bản tinh nhuệ hơn của Spyware.

Kế thừa toàn bộ chiến lược tower-seeking từ Spyware.

Chỉ khác ở stats: HP cao nhất, tốc độ chậm nhất.

Attributes:

Kế thừa toàn bộ từ Spyware/Malware. Stats lấy từ settings:

MALWARE_RANSOMWARE_HP, MALWARE_RANSOMWARE_SPEED, MALWARE_RANSOMWARE_REWARD.

Usage:

Tạo bởi game.py khi JSON wave có "ransomware":

malware = Ransomware(pos=(3, 5), graph=grid_graph)

_calculate_path(), update(), draw() kế thừa từ Spyware/Malware.

Phương thức `__init__(self, pos, graph)` (dòng 1080)

Khởi tạo Ransomware: gọi Spyware.__init__, ghi đè stats + animation riêng.

Args:

pos (tuple): (row, col) vị trí spawn.

graph (GridGraph): Bản đồ lưới.

Phương thức `update(self, dt)` (dòng 1100)

Override update để cập nhật animation theo state.

Args:

dt (float): Thời gian frame tính bằng giây.

Phương thức `_update_animation(self, dt)` (dòng 1122)

Cập nhật sprite key và frame số theo state hiện tại của Ransomware.

State machine: moving → Run (15 frames), attacking → Attack (15 frames).

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật self._anim_timer, self._anim_frame.
- Cập nhật self._sprite_key, self._anim_frames theo state.
- Reset self._anim_frame = 0 khi vượt khỏi tổng số frame.

Lớp SlowSpy (dòng 1156)

Malware loại SlowSpy – kế thừa từ Spyware, có khả năng làm chậm Tower.

Kế thừa toàn bộ chiến lược tower-seeking từ Spyware.

Thêm khả năng: khi tấn công Tower, vừa gây sát thương vừa làm chậm fire_rate của Tower.

Attributes:

slow_factor (float): Hệ số nhân tốc độ bắn của Tower (0.5 = 50%).

```
slow_duration (float): Thời gian hiệu ứng slow (giây).  
Kế thừa từ Spyware/Malware. Stats lấy từ settings: MALWARE_SPYWARE_*.  
  
Usage:  
Tạo bởi game.py khi JSON wave có "slowspy":  
malware = SlowSpy(pos=(3, 5), graph=grid_graph)
```

Phương thức `__init__(self, pos, graph)` (dòng 1172)

```
Khởi tạo SlowSpy: kế thừa Spyware + thêm slow config.  
  
Args:  
pos (tuple): (row, col) vị trí spawn.  
graph (GridGraph): Bản đồ lưới.
```

Phương thức `get_pending_hit(self)` (dòng 1185)

```
Override để thêm slow info vào pending_hit khi tấn công Tower.  
  
Vừa gây sát thương vừa làm chậm Tower.  
  
Returns:  
dict: {"type": "server"/"tower", "dmg": int, "slow": tuple (nếu Tower)}  
hoặc dict rỗng {} nếu không có đòn chờ.  
  
Note:  
game.py sẽ kiểm tra "slow" key và gọi tower.apply_slow(factor, duration)  
nếu có khi xử lý damage cho Tower.
```

Lớp `VaultWare` (dòng 1206)

```
Malware loại VaultWare – phiên bản ưu tiên của Ransomware.  
  
Kế thừa toàn bộ chiến lược tower-seeking từ Ransomware.  
Có HP cao hơn, sát thương cao hơn, tốc độ tấn công chậm hơn.  
Tower sẽ tự động ưu tiên nhắm vào nó (Tower nhận biết VaultWare).  
  
Attributes:  
Kế thừa từ Ransomware/Spyware/Malware.  
Stats: HP + 20%, Damage + 30%, Attack Speed 30% chậm hơn.  
Là mục tiêu ưu tiên của Tower (xem tower._find_target()).  
  
Usage:  
Tạo bởi game.py khi JSON wave có "vaultware":  
malware = VaultWare(pos=(3, 5), graph=grid_graph)
```

Phương thức `__init__(self, pos, graph)` (dòng 1224)

Khởi tạo VaultWare: kế thừa Ransomware + tăng stats + dùng animation riêng.

Args:

pos (tuple): (row, col) vị trí spawn.
graph (GridGraph): Bản đồ lưới.

Phương thức `update(self, dt)` (dòng 1243)

Override update để cập nhật animation theo state.

Args:

dt (float): Thời gian frame tính bằng giây.

Phương thức `_update_animation(self, dt)` (dòng 1265)

Cập nhật sprite key và frame số theo state hiện tại của VaultWare.

State machine: moving → Run (8 frames), attacking → Attack (19 frames).
Animation chạy chậm hơn dựa trên `_anim_speed_multiplier`.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật `self._anim_timer`, `self._anim_frame`.
- Cập nhật `self._sprite_key`, `self._anim_frames` theo state.
- Reset `self._anim_frame = 0` khi vượt khỏi tổng số frame.

Lớp `TrojanRanged` (dòng 1300)

Malware loại Trojan Ranged – tấn công từ xa bằng projectiles.

Kế thừa từ Trojan nhưng thay vì melee attack, nó bắn projectile đến Server. Có full animation support (Fly, Attack, Hit, Death).

Attributes:

`attack_type`: "ranged" (đánh xa)
`attack_range`: 3 (ô) – dùng khi cách mục tiêu ≤ 3 ô

Phương thức `__init__(self, pos, graph)` (dòng 1311)

Khởi tạo TrojanRanged: kế thừa từ Trojan, thiết lập ranged attack và animation.

Args:

pos (tuple): (row, col) vị trí spawn trên lưới.
graph (GridGraph): Bản đồ lưới.

Side effects:

- Kế thừa toàn bộ từ Trojan.`__init__` (hp, speed, reward, path A*).
- Đặt `attack_type = "ranged"`, `attack_range = 3`.
- Khởi tạo `_projectile_queue` (CustomQueue) để buffer projectile mỗi frame.

Phương thức `get_projectiles(self)` (dòng 1339)

Trả về danh sách MalwareProjectile mới tạo trong frame này và xóa queue.

game.py gọi sau mỗi update(dt) để thu thập projectile từ ranged malware và thêm vào self.projectiles.

Returns:

list: Danh sách MalwareProjectile tạo trong frame này.
Rỗng nếu không có projectile mới.

Side effects:

- Drain toàn bộ _projectile_queue (dequeue đến khi rỗng).

Phương thức update(self, dt) (dòng 1360)

Override update để chuyển đổi _pending_hit thành MalwareProjectile.

Khi parent (Trojan/Malware) set _pending_hit (đòn tấn công), TrojanRanged intercept và tạo MalwareProjectile thay vì gây damage trực tiếp. Projectile được enqueue vào _projectile_queue để game.py thu thập.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Khi hp <= 0: chỉ cập nhật death animation timer, không di chuyển.
- Intercept self._pending_hit và enqueue MalwareProjectile vào _projectile_queue.
- Gọi _update_animation(dt) để cập nhật sprite key theo state.

Phương thức _update_animation(self, dt) (dòng 1435)

Cập nhật sprite key và frame số theo state hiện tại của TrojanRanged.

State machine: moving → Run, attacking → Attack (loop từ frame 0 → 21).

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật self._anim_timer, self._anim_frame.
- Cập nhật self._sprite_key, self._anim_frames theo state (Run vs Attack).
- Reset self._anim_frame = 0 khi vượt khỏi tổng số frame.

Phương thức draw(self, screen, cell_size) (dòng 1468)

Vẽ TrojanRanged lên màn hình với animation và center alignment.

Dùng sprite key hiện tại (_sprite_key) và frame hiện tại (_anim_frame).

Fallback về hình tròn màu self.color nếu thiếu sprite hoặc frame vượt giới hạn.

Args:

```
screen (pygame.Surface): Bề mặt màn hình game.  
cell_size (int): Kích thước pixel mỗi ô lưới (settings.CELL_SIZE).
```

Lớp Spyware_Ranged (dòng 1495)

Malware loại Spyware Ranged – tấn công Tower từ xa bằng projectiles.

Kế thừa từ Spyware nhưng thay vì melee attack, nó bắn projectile đến Tower hoặc Server. Dùng cùng sprite melee của Spyware.

Attributes:

```
attack_type: "ranged" (đánh xa)  
attack_range: 3 (ô) – dùng khi cách mục tiêu <= 3 ô
```

Phương thức __init__(self, pos, graph) (dòng 1506)

Khởi tạo Spyware_Ranged: kế thừa từ Spyware, thiết lập ranged attack.

Args:

```
pos (tuple): (row, col) vị trí spawn trên lưới.  
graph (GridGraph): Bản đồ lưới.
```

Side effects:

- Kế thừa toàn bộ từ Spyware.__init__ (tower-seeking, stats).
- Đặt attack_type = "ranged", attack_range = 3.
- Khởi tạo _projectile_queue để buffer projectiles.

Phương thức get_projectiles(self) (dòng 1527)

Trả về danh sách MalwareProjectile mới tạo trong frame này.

game.py gọi sau mỗi update(dt) để thu thập projectile từ ranged malware.

Returns:

```
list: Danh sách MalwareProjectile tạo trong frame này.
```

Phương thức update(self, dt) (dòng 1543)

Override update để chuyển đổi _pending_hit thành MalwareProjectile.

Dùng attack_timer từ parent để bắn liên tục theo attack_speed.
Mỗi lần attack_timer >= 1/attack_speed, parent set _pending_hit
→ thay vì melee damage, tạo projectile.

Args:

```
dt (float): Thời gian frame tính bằng giây.
```

Lớp SlowSpy_Ranged (dòng 1592)

Malware loại SlowSpy Ranged – tấn công Tower từ xa + làm chậm.

Kế thừa từ SlowSpy nhưng bắn projectile thay vì melee attack.
Projectile vẫn áp dụng slow effect khi trúng Tower.

Dùng cùng sprite melee của SlowSpy.

Attributes:

attack_type: "ranged" (đánh xa)
attack_range: 3 (ô)

Phương thức `__init__(self, pos, graph)` (dòng 1604)

Khởi tạo SlowSpy_Ranged: kế thừa từ SlowSpy, thiết lập ranged attack.

Args:

pos (tuple): (row, col) vị trí spawn trên lưới.
graph (GridGraph): Bản đồ lưới.

Phương thức `get_projectiles(self)` (dòng 1620)

Drain toàn bộ MalwareProjectile mới tạo trong frame này.

Returns:

list[MalwareProjectile]: Danh sách projectile chờ trong queue.
Trả về [] nếu không có projectile mới.

Side effects:

- Làm rỗng `_projectile_queue` sau khi drain.

Phương thức `update(self, dt)` (dòng 1638)

Override update: chuyển `_pending_hit` thành MalwareProjectile với slow effect.

Args:

dt (float): Thời gian frame (giây).

Side effects:

- Gọi `super().update(dt)` để tính `attack_timer` và set `_pending_hit`.
- Khi `_pending_hit` tồn tại: tạo MalwareProjectile (server hoặc tower) và enqueue vào `_projectile_queue` thay vì gây melee damage.

Lớp RiposteWare (dòng 1688)

Malware loại RiposteWare – phiên bản phản đạn của Ransomware.

Kế thừa toàn bộ chiến lược tower-seeking từ Ransomware.

Có khả năng 50% phản đạn projectile của Tower trở lại chính Tower đó.

Sát thương phản đạn = damage gốc của RiposteWare + damage của projectile Tower.

Attributes:

Kế thừa từ Ransomware/Spyware/Malware.
Thêm: `_riposte_chance` (50% cơ hội phản đạn).

Usage:

```
Tạo bởi game.py khi JSON wave có "riposteware":
malware = RiposteWare(pos=(3, 5), graph=grid_graph)
```

Phương thức `__init__(self, pos, graph)` (dòng 1704)

Khởi tạo RiposteWare: kế thừa Ransomware + thêm riposte config + animation.

Args:

pos (tuple): (row, col) vị trí spawn.
graph (GridGraph): Bản đồ lưới.

Phương thức `update(self, dt)` (dòng 1718)

Override update để cập nhật animation theo state.

Args:

dt (float): Thời gian frame tính bằng giây.

Phương thức `_update_animation(self, dt)` (dòng 1740)

Cập nhật sprite key và frame số theo state hiện tại của RiposteWare.

State machine: moving → Run (8 frames), attacking → Attack (8 frames).

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật self._anim_timer, self._anim_frame.
- Cập nhật self._sprite_key, self._anim_frames theo state.
- Reset self._anim_frame = 0 khi vượt khỏi tổng số frame.

Phương thức `on_hit_by_projectile(self, projectile)` (dòng 1773)

Override để xử lý riposte: 50% phản đạn projectile trở lại Tower.

Args:

projectile (Projectile): Đạn vừa trúng.

Side effects:

- 50% cơ hội: tạo MalwareProjectile phản đạn về phía Tower
- Luôn gọi take_damage() để trừ máu bình thường

Note:

Sát thương phản đạn = malware.attack_damage + projectile.damage.
Projectile được tạo hướng về vị trí của source_tower nếu có.

Phương thức `get_projectiles(self)` (dòng 1811)

Lấy tất cả projectile phản đạn từ queue.

Returns:

list[MalwareProjectile]: Danh sách projectile phản đạn được tạo trong frame này.

Side effects:

- Drain toàn bộ `_projectile_queue` (dequeue đến khi rỗng).
- Gọi bởi `game.py` mỗi frame để thu thập projectile từ malware.

Usage:

```
Gọi từ game.py:
if hasattr(malware, 'get_projectiles'):
    for proj in malware.get_projectiles():
        self.projectiles.append(proj)
```

File: entities/boss.py

Lớp Boss (dòng 10)

Lớp Boss – enemy mạnh với cơ chế tấn công đặc biệt.

Không kế thừa từ Malware nhưng có cơ chế tương tự (di chuyển, tấn công).

Boss có máu cao, sát thương cao, và kỹ năng AoE/lan tỏa.

Attributes:

```
hp (int): Máu hiện tại
max_hp (int): Máu tối đa
pos (tuple): Vị trí hiện tại (row, col)
graph (GridGraph): Bản đồ lưới
path (list): Đường đi đến server
speed (float): Tốc độ di chuyển (ô/giây)
original_speed (float): Tốc độ gốc (trước khi slow)
slow_timer (float): Thời gian bị slow
slow_factor (float): Hệ số giảm tốc (0.0-1.0)
```

Phương thức `__init__(self, pos, graph)` (dòng 28)

Khởi tạo Boss tại vị trí spawn.

Args:

```
pos (tuple): (row, col) vị trí spawn
graph (GridGraph): Bản đồ lưới
```

Phương thức `is_dead(self)` (dòng 46)

Kiểm tra Boss đã chết hay chưa.

Phương thức `take_damage(self, damage)` (dòng 50)

Nhận sát thương.

Phương thức `apply_slow(self, factor, duration)` (dòng 54)

Áp dụng hiệu ứng slow.

Phương thức `update(self, dt)` (dòng 59)

Cập nhật Boss mỗi frame (override trong subclass).

Phương thức `draw(self, screen, cell_size)` (dòng 67)

Vẽ Boss lên màn hình (override trong subclass).

Lớp `FireWorm` (dòng 72)

Boss Level 2 - Quái lửa tấn công AoE với hiệu ứng đốt.

Tấn công toàn bộ tháp trong phạm vi cứ mỗi `duration_attack_tower` giây.
Gây hiệu ứng thiêu đốt và lan cháy sang tường gần đó.

Attributes:

- `attack_damage_tower` (int): Sát thương khi tấn công tháp
- `attack_damage_server` (int): Sát thương khi tấn công server
- `attack_speed` (float): Tốc độ tấn công server (phát/giây)
- `attack_range` (int): Phạm vi quét tháp (ô)
- `duration_attack_tower` (float): Khoảng thời gian quét tháp (giây)
- `fire_spread_range` (int): Phạm vi lan cháy (ô)
- `move_progress` (float): Tiến độ di chuyển (0.0-1.0) đến ô tiếp theo
- `attack_timer` (float): Đếm ngược tấn công
- `tower_attack_timer` (float): Đếm ngược tấn công tháp
- `attacking_server` (bool): Đang tấn công server

Phương thức `__init__(self, pos, graph)` (dòng 91)

Khởi tạo `FireWorm`.

Args:

- `pos` (tuple): (row, col) vị trí spawn
- `graph` (GridGraph): Bản đồ lưới

Phương thức `calculate_path(self)` (dòng 128)

Tính đường đi đến server bằng A*.

Phương thức `update(self, dt)` (dòng 133)

Cập nhật `FireWorm`: di chuyển, tấn công tháp, tấn công server.

Args:

- `dt` (float): Thời gian frame (giây)

Phương thức `_update_animation(self, dt)` (dòng 179)

Cập nhật sprite và frame theo state hiện tại của `FireWorm`.

Args:

- `dt` (float): Thời gian frame (giây)

Phương thức `get_tower_attacks(self)` (dòng 212)

Trả về danh sách tháp cần tấn công trong phạm vi.

Được gọi từ `game.py` để lấy tháp cần hit.

Returns:

```
list: Danh sách {pos: (row,col), damage: int, affected_cells: list}  
hoặc [] nếu chưa đến lúc tấn công
```

Phương thức `get_server_attack(self)` (dòng 234)

Trả về info tấn công server (nếu đang tấn công).

Returns:

```
dict: {"damage": ...} hoặc None
```

Phương thức `is_alive(self)` (dòng 245)

Kiểm tra Boss còn sống.

Phương thức `has_reached_server(self)` (dòng 249)

Kiểm tra đã chạm server.

Phương thức `get_render_data(self, cell_size)` (dòng 253)

Tạo Surface tổng hợp (Boss + HP bar) cho Y-Sort rendering.

Args:

```
cell_size (int): Kích thước pixel mỗi ô lưới.
```

Returns:

```
dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "boss"}
```

Lớp `FlyingDemon` (dòng 311)

Boss Level 3 - Quái bay thả Bomb ngẫu nhiên + tấn công server.

Cơ chế chính:

- Thả bomb ngẫu nhiên mỗi `bomb_duration` giây:
 - 75% Bomb bình thường (-300 HP, nổ 4-7s random)
 - 25% Atomic bomb (-10000 HP, nổ 4-7s random, thua game nếu nổ)
- Tấn công server khi chạm (giống FireWorm)

Attributes:

```
attack_damage_server (int): Sát thương khi tấn công server  
attack_speed (float): Tốc độ tấn công server (phát/giây)  
bomb_duration (float): Khoảng thời gian drop bomb (giây)  
move_progress (float): Tiến độ di chuyển (0.0-1.0)  
attack_timer (float): Đếm ngược tấn công server  
bomb_timer (float): Đếm ngược drop bomb  
attacking_server (bool): Đang tấn công server  
dropped_bombs (list): Danh sách bomb được drop (BossBomb)
```

Phương thức `__init__(self, pos, graph)` (dòng 331)

Khởi tạo `FlyingDemon`.

Args:

- pos (tuple): (row, col) vị trí spawn
- graph (GridGraph): Bản đồ lưới

Phương thức `calculate_path(self)` (dòng 374)

Tính đường đi đến server bằng A*.

Phương thức `_drop_bomb(self)` (dòng 379)

Drop bomb ngẫu nhiên (75% normal, 25% atomic) tại vị trí hiện tại.

Trigger drop effect animation trên boss.

Returns:

- BossBomb: Bomb vừa drop

Phương thức `update(self, dt)` (dòng 419)

Cập nhật FlyingDemon: di chuyển, thả bomb, tấn công server.

Args:

- dt (float): Thời gian frame (giây)

Phương thức `_update_animation(self, dt)` (dòng 478)

Cập nhật sprite và frame theo state của FlyingDemon.

Args:

- dt (float): Thời gian frame (giây)

Phương thức `get_server_attack(self)` (dòng 508)

Trả về info tấn công server (nếu đang tấn công).

Returns:

- dict: {"damage": ...} hoặc None

Phương thức `is_alive(self)` (dòng 522)

Kiểm tra Boss còn sống.

Phương thức `has_reached_server(self)` (dòng 526)

Kiểm tra đã chạm server.

Phương thức `get_bombs(self)` (dòng 530)

Trả về danh sách bomb hiện tại.

Returns:

- list: Danh sách BossBomb

Phương thức `remove_bomb(self, bomb)` (dòng 538)

Xóa bomb khỏi danh sách khi đã remove khỏi game.

```
Args:
    bomb (BossBomb): Bomb cần xóa
```

Phương thức `get_render_data(self, cell_size)` (dòng 547)

Tạo Surface tổng hợp (Boss + HP bar + attack effect) cho Y-Sort rendering.

```
Args:
    cell_size (int): Kích thước pixel mỗi ô lưới.

Returns:
    dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "boss", "attack_effect": {...}}
```

Lớp `Shadow` (dòng 624)

Boss Level 4 - Kẻ tàng hình với cơ chế lẩn bắt tù.

Khi xuất hiện: tất cả malware trên map trở nên tàng hình vĩnh viễn.

Cơ chế Roll: mỗi 5 giây, vào trạng thái lẩn 3 giây:

- Tốc độ di chuyển tăng (x3), bắt tù với đạn tháp
- Hiệu ứng đốt/chạm vẫn dính
- Damage và attack speed tăng khi ở server

```
Attributes:
    roll_cooldown_timer (float): Đếm ngược đến lần Roll tiếp theo (giây).
    roll_timer (float): Đếm ngược thời gian Roll còn lại (giây).
    is_rolling (bool): True khi đang ở trạng thái Roll.
    roll_phase (str): Giai đoạn Roll hiện tại: "pre", "mid", hoặc "end".
    move_progress (float): Tiến độ di chuyển (0.0-1.0) đến ô tiếp theo.
    attacking_server (bool): True khi đã chạm ô server_pos.
    server_attack_timer (float): Đếm ngược giữa các đòn tấn công server.
    _sprite_key (str): Key sprite hiện tại trong cache.
    _anim_frame (int): Frame animation hiện tại.
    _anim_timer (float): Đếm thời gian đến frame kế tiếp.
    _anim_frames (int): Tổng số frame animation hiện tại.
    state (str): Trạng thái hiện tại: "moving", "rolling", "attacking_server", "dead".
```

Animation: Walk(14f) → Roll[Pre(3f)→Mid(4f loop)→End(7f)] → Attack(10f) / Die(33f)

Phương thức `__init__(self, pos, graph)` (dòng 656)

Khởi tạo Shadow Boss tại vị trí spawn.

```
Args:
    pos (tuple): (row, col) vị trí spawn trên lưới.
    graph (GridGraph): Bản đồ lưới.
```

```
Side effects:
    - Gọi calculate_path() để tính A* đến server ngay khi spawn.
    - Đặt tất cả timer và state về giá trị ban đầu.
```

Phương thức `calculate_path(self)` (dòng 693)

Tính đường đi A* từ vị trí hiện tại đến server.

Side effects:

- Ghi đè `self.path` bằng kết quả A*.

Phương thức `take_damage(self, damage)` (dòng 702)

Nhận sát thương – bắt từ khi đang Roll.

Args:

`damage (float)`: Sát thương nhận vào.

Side effects:

- Nếu `is_rolling = True`: bỏ qua hoàn toàn, không trừ HP.
- Ngược lại: trừ HP tối thiểu 0.

Phương thức `update(self, dt)` (dòng 716)

Cập nhật Shadow Boss mỗi frame: slow, roll, di chuyển, tấn công server.

Args:

`dt (float)`: Thời gian frame (giây).

Side effects:

- Gọi `_update_roll()` để quản lý chu kỳ Roll.
- Gọi `_move()` để di chuyển trên lưới.
- Tích lũy `server_attack_timer` khi đang tấn công server.
- Cập nhật animation qua `_update_animation()`.

Phương thức `_move(self, dt)` (dòng 741)

Di chuyển Shadow theo path; tốc độ nhân `ROLL_SPEED_MULT` khi đang Roll.

Args:

`dt (float)`: Thời gian frame (giây).

Side effects:

- Cập nhật `move_progress` và `self.pos` khi `progress >= 1.0`.
- Đặt `attacking_server = True` khi chạm `server_pos`.
- Gọi `calculate_path()` nếu path cạn.

Phương thức `_update_roll(self, dt)` (dòng 767)

Quản lý chu kỳ Roll: đếm cooldown → kích hoạt Roll → đếm hết Roll.

Args:

`dt (float)`: Thời gian frame (giây).

Side effects:

- Khi cooldown hết: đặt `is_rolling=True`, `roll_phase="pre"`, reset timers.

- Khi roll hết thời gian: chuyển roll_phase sang "end".
- _update_animation() xử lý việc kết thúc phase "end" (is_rolling=False).

Phương thức `_update_animation(self, dt)` (dòng 792)

Cập nhật sprite key và frame animation theo state hiện tại.

Args:

dt (float): Thời gian frame (giây).

Side effects:

- Tăng _anim_frame theo ANIM_FPS.
- Cập nhật _sprite_key và loop frame tương ứng state.
- Khi roll_phase "end" kết thúc (≥ 7 frames): đặt is_rolling = False.

Phương thức `_anim_simple(self, key, total, dt)` (dòng 833)

Chạy animation đơn giản một chiều (không loop) theo ANIM_FPS.

Args:

key (str): Sprite key trong cache.

total (int): Tổng số frame; dừng ở frame total-1.

dt (float): Thời gian frame (giây).

Side effects:

- Đặt _sprite_key = key.
- Tăng _anim_frame tối đa đến total-1 (không wrap).

Phương thức `get_server_attack(self)` (dòng 851)

Trả về info tấn công server nếu đến lúc (tốc độ và damage tăng khi Roll).

Khi đang Roll: attack_speed $\times$ ROLL_RATE_MULT, damage $\times$ ROLL_DMG_MULT.

Returns:

dict: {"damage": float} khi đến lúc tấn công.

None: chưa đến lúc hoặc không ở server.

Side effects:

- Reset server_attack_timer về 0 khi tấn công.

Phương thức `has_reached_server(self)` (dòng 872)

Kiểm tra Shadow đã chạm ô server_pos.

Returns:

bool: True nếu self.pos == graph.server_pos.

Phương thức `get_render_data(self, cell_size)` (dòng 880)

Tạo Surface tổng hợp (Boss + HP bar) cho Y-Sort rendering.

Args:

`cell_size (int):` Kích thước pixel mỗi ô lưới.

Returns:

`dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "boss"}.`

Lớp RiposteBoss (dòng 917)

Bosslvl: phiên bản của RiposteWare – cùng cơ chế phản đạn, to hơn, mạnh hơn.

Kế thừa cơ chế riposte: 50% phản đạn projectile trở lại Tower.

Sát thương phản đạn = `attack_damage + projectile.damage`.

Khác RiposteWare (malware): đi A* thẳng đến Server, không hunt Tower.

Attributes:

`_riposte_chance (float):` Xác suất phản đạn (mặc định 0.5).

`_projectile_queue (CustomQueue):` Queue lưu projectile phản đạn chờ game.py thu.

`server_attack_timer (float):` Đếm thời gian tấn công server.

`server_attack_speed (float):` Số đòn/giây khi ở server.

Phương thức `__init__(self, pos, graph)` (dòng 931)

Khởi tạo RiposteBoss với cơ chế phản đạn và A* đến server.

Args:

`pos (tuple):` (row, col) vị trí spawn trên lưới.

`graph (GridGraph):` Bản đồ lưới.

Side effects:

- Đặt tất cả chỉ số chiến đấu và timers về giá trị ban đầu.
- Gọi `calculate_path()` để tính A* đến server ngay khi spawn.

Phương thức `calculate_path(self)` (dòng 966)

A* thẳng đến server.

Phương thức `update(self, dt)` (dòng 971)

Cập nhật RiposteBoss mỗi frame: slow, di chuyển A*, tấn công server, animation.

Args:

`dt (float):` Thời gian frame (giây).

Side effects:

- Di chuyển theo path đến server, tính lại nếu path cạn.
- Đặt `attacking_server = True` khi chạm `server_pos`.
- Tích lũy `server_attack_timer`; cập nhật animation frame.

Phương thức `on_hit_by_projectile(self, projectile)` (dòng 1021)

Xử lý khi bị bắn – 50% phản đạn về Tower nguồn, nhận damage nếu không phản.

Args:

projectile: Đạn bắn vào (phải có thuộc tính damage và source_tower).

Side effects:

- Nếu phân thành công: tạo MalwareProjectile vào _projectile_queue, không trừ HP.
- Nếu không phân: gọi take_damage(projectile.damage).

Phương thức get_projectiles(self) (dòng 1048)

Drain toàn bộ MalwareProjectile phân đạn ra list để game.py xử lý.

Returns:

list[MalwareProjectile]: Danh sách projectile đang chờ trong queue.
Trả về [] nếu không có projectile mới.

Side effects:

- Làm rỗng _projectile_queue sau khi drain.

Phương thức get_server_attack(self) (dòng 1063)

Trả về info tấn công server nếu đến lúc theo server_attack_speed.

Returns:

dict: {"damage": int} khi đến lúc tấn công.
None: chưa đến lúc hoặc không ở server.

Side effects:

- Reset server_attack_timer về 0 khi tấn công.

Phương thức has_reached_server(self) (dòng 1078)

Kiểm tra RiposteBoss đã chạm ô server_pos.

Returns:

bool: True nếu self.pos == graph.server_pos.

Phương thức get_render_data(self, cell_size) (dòng 1086)

Tạo Surface tổng hợp (Boss scale 1.5× + HP bar) cho Y-Sort rendering.

Args:

cell_size (int): Kích thước pixel mỗi ô lưới.

Returns:

dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "boss"}.

Lớp Final (dòng 1132)

Boss Level 5 – Final Boss. Đi thẳng col++ từ (12,0), phá 3×3 ô cản trở, spawn RiposteWare tại mỗi ô bị phá, phân đạn 25% thẳng vào server, thả bomb giống FlyingDemon.

Movement: col++ mỗi bước, không dùng A*/BFS.

Destroy: nếu next cell không phải PATH/SERVER → play Destroy_1(10f)→Destroy_2(10f)
→ queue 3×3 cells cho game.py xử lý (clear + spawn RiposteWare).
Reflection: 25% khi bị bắn → MalwareProjectile bay thẳng vào server.
Bombs: 75% normal / 25% atomic, purple glow 0.6s sau khi thả.

Phương thức `__init__(self, pos, graph)` (dòng 1146)

Khởi tạo Final Boss tại hàng cố định FIXED_ROW, cột 0.

Args:

pos (tuple): (row, col) bị bỏ qua – Final luôn spawn tại (FIXED_ROW, 0).
graph (GridGraph): Bản đồ lưới.

Side effects:

- Ghi đè self.pos thành (FIXED_ROW, 0) bất kể pos truyền vào.
- Khởi tạo spawn_queue, bomb mechanics, projectile queue, animation state.

Phương thức `update(self, dt)` (dòng 1193)

Cập nhật Final Boss mỗi frame: di chuyển col++, phá tường, tấn công server.

Args:

dt (float): Thời gian frame (giây).

Side effects:

- Tick bomb_timer; gọi _drop_bomb() khi đến hạn.
- Nếu state == "destroying": gọi _update_destroy(), block movement.
- Di chuyển sang cột tiếp theo; nếu gặp ô không phải PATH → "destroying".
- Đặt attacking_server = True khi chạm SERVER hoặc vượt qua cột cuối.

Phương thức `_update_destroy(self, dt)` (dòng 1271)

Chạy animation phá hủy 2 phase; thực thi phá hủy cuối phase 2.

Args:

dt (float): Thời gian frame (giây).

Side effects:

- Phase 1 (Destroy_1, 10f): chơi xong → chuyển sang phase 2.
- Phase 2 (Destroy_2, 10f): chơi xong → gọi _execute_destroy(),
đặt state = "moving", reset destroy counters.

Phương thức `_execute_destroy(self)` (dòng 1304)

Queue ô hàng ±1, cột +1..+3 (non-PATH/SERVER/SPAWN) để game.py phá và spawn.

Side effects:

- Append các (row, col) hợp lệ vào spawn_queue.
- game.py drain spawn_queue mỗi frame: clear cell + spawn RiposteWare.

Phương thức `drain_spawn_queue(self)` (dòng 1321)

Trả về và xóa danh sách ô cần phá; game.py gọi mỗi frame.

Returns:

list[tuple]: Danh sách (row, col) ô cần clear + spawn RiposteWare.
Trả về [] nếu không có ô mới.

Side effects:

- Làm rỗng self.spawn_queue sau khi drain.

Phương thức `_drop_bomb(self)` (dòng 1335)

Thả bomb tại vị trí hiện tại (75% normal, 25% atomic) giống FlyingDemon.

Returns:

BossBomb: Bomb vừa tạo và thêm vào dropped_bombs.

Side effects:

- Đặt effect_timer = effect_duration (hiệu ứng glow 1.5s).
- Append bomb mới vào self.dropped_bombs.

Phương thức `get_bombs(self)` (dòng 1366)

Trả về danh sách bomb đang hoạt động.

Returns:

list[BossBomb]: Danh sách BossBomb chưa nổ hoặc đang nổ.

Phương thức `remove_bomb(self, bomb)` (dòng 1374)

Xóa bomb khỏi danh sách khi game.py đã xử lý xong.

Args:

bomb (BossBomb): Bomb cần xóa.

Side effects:

- Xóa bomb khỏi self.dropped_bombs nếu tồn tại.

Phương thức `on_hit_by_projectile(self, projectile)` (dòng 1386)

Xử lý khi bị bắn – 25% phản đạn thẳng vào server, nhận damage nếu không phản.

Args:

projectile: Đạn bắn vào (phải có thuộc tính damage).

Side effects:

- Nếu phản: tạo MalwareProjectile bay thẳng vào server, enqueue vào _projectile_queue.
- Nếu không phản: gọi take_damage(projectile.damage).

Phương thức `get_projectiles(self)` (dòng 1411)

Drain toàn bộ MalwareProjectile phản đạn ra list để game.py xử lý.

Returns:

```
list[MalwareProjectile]: Danh sách projectile đang chờ trong queue.  
Trả về [] nếu không có projectile mới.
```

Side effects:

- Làm rỗng `_projectile_queue` sau khi drain.

Phương thức `get_server_attack(self)` (dòng 1426)

Trả về info tấn công server nếu đến lúc theo `attack_speed`.

Returns:

dict: {"damage": int} khi đến lúc tấn công.
None: chưa đến lúc hoặc không ở server.

Side effects:

- Reset `server_attack_timer` về 0 khi tấn công.

Phương thức `has_reached_server(self)` (dòng 1441)

Kiểm tra Final Boss đang tấn công server.

Returns:

bool: True khi `attacking_server = True` (chạm SERVER hoặc vượt cột cuối).

Phương thức `_advance_anim(self, dt, total, loop)` (dòng 1449)

Tăng frame animation theo `ANIM_FPS`, hỗ trợ loop hoặc dừng ở frame cuối.

Args:

`dt` (float): Thời gian frame (giây).
`total` (int): Tổng số frame của animation hiện tại.
`loop` (bool): True = vòng lại từ đầu; False = dừng ở frame `total-1`.

Side effects:

- Tích lũy `_anim_timer`; tăng `_anim_frame` khi đủ `1/ANIM_FPS`.

Phương thức `get_render_data(self, cell_size)` (dòng 1468)

Tạo Surface tổng hợp (Boss + HP bar + purple glow khi thả bomb) cho Y-Sort.

Args:

`cell_size` (int): Kích thước pixel mỗi ô lưới.

Returns:

dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "boss"}.
Thêm key "attack_effect" nếu `effect_timer > 0`.

File: `entities/tower.py`

Lớp `Tower` (dòng 13)

Lớp cơ sở cho tất cả Tower trong game.

Tower ngồi trên ô WALL, bắn Malware đang đi trên PATH kẻ xung quanh.
Subclass ghi đè stats và chiến lược bắn để tạo Tower có đặc tính riêng.

Attributes:

pos (tuple): Vị trí ô lưới (row, col) – luôn là ô WALL.
graph (GridGraph): Tham chiếu GridGraph để kiểm tra vị trí.
hp (int): Máu hiện tại của tower. Về 0 → tower bị phá hủy.
max_hp (int): Máu tối đa – dùng tính tỉ lệ thanh máu khi vẽ.
range (int): Bán kính tấn công (đơn vị: ô lưới).
damage (int): Sát thương mỗi phát bắn.
fire_rate (float): Số phát bắn mỗi giây.
fire_timer (float): Bộ đếm thời gian tích lũy dt – khi $\geq 1/\text{fire_rate}$ thì bắn.
cost (int): Tiền cần để xây tower này.
color (tuple): Màu RGB hiển thị trên lưới.
targeting (str): Chiến lược chọn mục tiêu – "min_dist" hoặc "max_hp".
tower_type (str): Loại tower ("basic", "ice", "radar") – dùng cho sprite.
path (dict): Tập hợp ô PATH có thể bị bắn (BFS từ pos).
slow_timer (float): Bộ đếm thời gian tính bằng giây.
stunned_timer (float): Bộ đếm thời gian tính bằng giây.
prev_slow_timer (float): Lưu giá trị slow_timer của frame trước để theo dõi transitions.
slow_effect_stage (str): "start", "active", "ending" – giai đoạn của hiệu ứng slow.
slow_effect_timer (float): Bộ đếm thời gian cho hiệu ứng slow animation.
prev_slow_effect_timer (float): Lưu giá trị slow_effect_timer của frame trước để theo dõi transitions.

Usage:

Không khởi tạo trực tiếp – dùng subclass BasicNode/IceWall/RadarNode.
game.py tạo Tower khi người chơi click ô WALL::

```
tower = BasicNode((row, col), graph)
projectile = tower.update(dt, candidates)
```

Phương thức `__init__(self, pos, graph, targeting)` (dòng 48)

Khởi tạo Tower tại vị trí pos với giá trị mặc định.

Subclass ghi đè range, damage, fire_rate, cost, color, tower_type sau khi gọi `super().__init__()` để thiết lập stats riêng.

Args:

pos (tuple): (row, col) vị trí ô lưới – phải là ô WALL.
graph (GridGraph): Bản đồ lưới, dùng để BFS tìm ô PATH có thể bắn.
targeting (str): Chiến lược chọn mục tiêu mặc định: "min_dist" (nhắm kẻ gần server nhất). Subclass có thể truyền "max_hp".

Side effects:

- Gọi `self._path_can_shoot()` để tính sẵn tập hợp ô PATH trong tầm bắn.

Phương thức `update(self, dt, candidates)` (dòng 86)

Cập nhật cooldown và bắn nếu đến lượt.

Args:

dt (float): Thời gian frame tính bằng giây.
candidates (list): Danh sách Malware trong tầm bắn (từ SpatialHash).
Nếu danh sách rỗng → không bắn.

Returns:

Projectile: Đạn vừa bắn ra nếu đến lượt và có mục tiêu.
None: Nếu chưa đến cooldown hoặc không có mục tiêu hợp lệ.

Side effects:

- Tích lũy self.fire_timer mỗi frame.
- Reset self.fire_timer về 0 mỗi lần bắn để fire_rate luôn chính xác.

Phương thức `apply_burn(self, damage_per_sec, duration)` (dòng 164)

Áp dụng hiệu ứng đốt cháy (DOT) lên tower.

Args:

damage_per_sec (float): Sát thương phải chịu mỗi giây.
duration (float): Thời gian kéo dài của hiệu ứng.

Phương thức `apply_slow(self, factor, duration)` (dòng 175)

Áp dụng hiệu ứng slow lên tower

Args:

factor (float): Hệ số nhân tốc độ khi bị slow (0.5 = còn 50%).
duration (float): Thời gian hiệu ứng slow tính bằng giây.

Side effects:

- Cộng dồn hiệu ứng slow nếu bị tấn công nhiều lần.
- Khi self.slow_timer > 0 → tower bị slow, fire_rate giảm theo factor.
- Khi self.slow_timer <= 0 → tower hết slow, fire_rate trở lại bình thường.

Phương thức `apply_stun(self, duration)` (dòng 189)

Áp dụng hiệu ứng stun lên tower

Args:

duration (float): Thời gian hiệu ứng stun tính bằng giây.

Side effects:

- Cộng dồn hiệu ứng stun nếu bị tấn công nhiều lần.
- Khi self.stunned_timer > 0 → tower bị stun
- Khi self.stunned_timer <= 0 → tower hết stun

Phương thức `_path_can_shoot(self)` (dòng 203)

Tính tập hợp tất cả ô PATH có thể bị tower này bắn (BFS từ pos).

Dùng BFS lan ra từ các ô láng giềng của tower để tìm mọi ô PATH kết nối. Tower chỉ bắn malware đang đứng trên một trong các ô này.

Returns:

dict: {(row, col): True} – tập hợp ô PATH trong phạm vi bắn.
Dùng dict thay set để tra cứu O(1) trong `_find_target()`.

Side effects:

- Kết quả lưu vào `self.path` khi `__init__` gọi phương thức này.

Phương thức `take_damage(self, damage)` (dòng 229)

Trừ máu tower khi bị malware tấn công.

Args:

damage (int): Lượng sát thương cần trừ (≥ 0).

Side effects:

- Giảm `self.hp`. Gọi `is_destroyed()` để kiểm tra sau khi `take_damage()`.

Phương thức `is_destroyed(self)` (dòng 240)

Kiểm tra tower đã bị phá hủy chưa.

Returns:

bool: True nếu `self.hp` ≤ 0 , False nếu còn sống.

Phương thức `_find_target(self, candidates)` (dòng 247)

Chọn mục tiêu tối ưu từ danh sách candidates bằng Heap (DSA).

Chiến lược ưu tiên (theo thứ tự):

1. VaultWare (cao nhất – trừ khi không dính attraction 50/50)
2. Bomb (sau khi check xong VaultWare)
3. Malware khác theo targeting strategy (min_dist hoặc max_hp)

Args:

candidates (list): Danh sách Malware + Bomb từ spatial hash.

Returns:

VaultWare | Bomb | Malware: Mục tiêu được chọn, hoặc None nếu không có.

Note:

Dùng Heap thay `sort()` để thể hiện ứng dụng DSA heap trong game thực tế.

Phương thức `shoot(self, target)` (dòng 308)

Bắn vào mục tiêu – tạo và trả về Projectile.

Args:

target (Malware): Đối tượng Malware đang bị nhắm bắn.

Returns:

Projectile: Đạn mới tạo ra, bay từ pos của tower đến target.

Note:

Import Projectile bên trong hàm để tránh circular import (projectile.py import settings, không import tower.py).
Projectile chứa tham chiếu đến target – Projectile.update() kiểm tra target.is_dead() trước khi gây damage.

Phương thức `draw(self, screen, cell_size)` (dòng 333)

Vẽ Tower lên màn hình – sprite hoặc fallback hình tròn.

Dùng sprite từ cache (key: "tower_{tower_type}") nếu có.

Nếu không tìm được sprite → vẽ hình tròn màu `self.color` làm fallback.

Args:

`screen` (pygame.Surface): Bề mặt màn hình game, do game.py truyền vào.
`cell_size` (int): Kích thước pixel mỗi ô lưới (settings.CELL_SIZE).

Note:

Quy đổi tọa độ: `pixel_x = col * cell_size`, `pixel_y = row * cell_size`.
Phương thức này ít dùng trực tiếp – `game._draw_grid()` dùng `get_render_data()` thay thế để hỗ trợ Y-sorting.

Phương thức `get_render_data(self, cell_size)` (dòng 359)

Trả về dict render-data để Y-sort trong `_draw_grid()`, bottom-center aligned + HP bar.

Tạo Surface tổng hợp (sprite + HP bar nếu bị damage), tính vị trí vẽ bottom-center aligned theo hệ Y-sorting của `game._draw_grid()`.

Args:

`cell_size` (int): Kích thước pixel mỗi ô lưới (settings.CELL_SIZE).

Returns:

dict: {"surf": Surface, "pos": (x,y), "sort_y": int, "type": "tower"}
dùng để xếp vào `render_list` và sort theo `sort_y` trước khi vẽ.
None nếu sprite không tìm được trong cache.

Lớp `BasicNode` (dòng 415)

Tower cơ bản – single target, damage trung bình, giá rẻ nhất.

Chiến lược: nhắm kẻ gần Server nhất (targeting = "min_dist").

Thích hợp khi người chơi mới bắt đầu, chưa đủ tiền mua tower xịn hơn.

Attributes:

Kế thừa từ Tower. Stats lấy từ settings:
TOWER_BASIC_RANGE, TOWER_BASIC_DAMAGE, TOWER_BASIC_FIRE_RATE,
TOWER_BASIC_COST, TOWER_BASIC_HP.

Usage:

Tạo khi người chơi nhấn phím [1] rồi click ô WALL::

```
tower = BasicNode((row, col), graph)
```

Phương thức `__init__(self, pos, graph, targeting)` (dòng 432)

Khởi tạo BasicNode: gọi `super().__init__`, ghi đè stats BasicNode.

Args:

`pos` (tuple): (row, col) vị trí ô WALL.

`graph` (GridGraph): Bản đồ lưới.

`targeting` (str): Chiến lược chọn mục tiêu. Mặc định "min_dist".

Lớp IceWall (dòng 453)

Tower làm chậm – damage thấp nhưng áp dụng slow effect lên mục tiêu.

Sau khi bắn: gọi `target.apply_slow(factor, duration)` để giảm tốc độ Malware trong `slow_duration` giây. Kết hợp với BasicNode để tiêu diệt kẻ cứng nhất.

Attributes:

`slow_factor` (float): Hệ số nhân tốc độ khi bị đóng băng (0.5 = còn 50%).

`slow_duration` (float): Thời gian hiệu ứng slow tính bằng giây.

Kế thừa từ Tower. Stats lấy từ settings: TOWER_ICE*.

Usage:

Tạo khi người chơi nhấn phím [2] rồi click ô WALL::

```
tower = IceWall((row, col), graph)
```

Phương thức `__init__(self, pos, graph, targeting)` (dòng 470)

Khởi tạo IceWall: gọi `super().__init__`, ghi đè stats IceWall.

Args:

`pos` (tuple): (row, col) vị trí ô WALL.

`graph` (GridGraph): Bản đồ lưới.

`targeting` (str): Chiến lược chọn mục tiêu. Mặc định "min_dist".

Phương thức `shoot(self, target)` (dòng 492)

Bắn và áp dụng slow effect lên mục tiêu trước khi tạo đạn.

Args:

`target` (Malware): Đối tượng Malware đang bị nhắm bắn.

Returns:

Projectile: Đạn mới tạo ra (kế thừa từ Tower.shoot()).

Side effects:

- Gọi `target.apply_slow(slow_factor, slow_duration)` ngay lập tức.
Hiệu ứng làm chậm áp dụng trước khi đạn bay tới.

Note:

`apply_slow()` TRƯỚC `super().shoot()` - slow áp dụng ngay khi bắn,
không chờ đạn chạm target (thiết kế game Week 2).

Lớp `RadarNode` (dòng 523)

Tower chỉ dò map - range lớn nhất, KHÔNG bắn đạn.

Vai trò trong game: Đặt ở vị trí chiến lược để `SpatialHash.query_range()`
có thể "nhìn thấy" malware từ xa hơn, hỗ trợ các tower khác nhắm mục tiêu.
`shoot()` bị override để luôn trả về `None` → `game.py` không tạo `Projectile`.

Attributes:

Kế thừa từ `Tower`. Stats lấy từ settings: `TOWER_RADAR_RANGE`,
`TOWER_RADAR_COST`, `TOWER_RADAR_HP`. Không có `damage/fire_rate` riêng.

Usage:

Tạo khi người chơi nhấn phím [3] rồi click ô WALL::

```
tower = RadarNode((row, col), graph)
```

Phương thức `__init__(self, pos, graph, targetting)` (dòng 540)

Khởi tạo `RadarNode`: gọi `super().__init__`, ghi đè stats `RadarNode`.

Args:

`pos` (tuple): (row, col) vị trí ô WALL.
`graph` (GridGraph): Bản đồ lưới.
`targetting` (str): Chiến lược chọn mục tiêu. Mặc định "min_dist".

Phương thức `shoot(self, target)` (dòng 558)

Override để vô hiệu hóa bắn - luôn trả về `None`.

Args:

`target` (Malware): Mục tiêu (không dùng - `RadarNode` không bắn).

Returns:

`None`: Luôn trả về `None` để `game.py` bỏ qua, không tạo `Projectile`.

Note:

`game.py` kiểm tra "if projectile is not None" trước khi append →
`RadarNode` chỉ dò map, không bắn đạn.

Phương thức `update(self, dt, candidates)` (dòng 573)

Cập nhật `RadarNode`: xử lý slow/stun/burn rồi tự trừ 5 HP mỗi giây.

Args:

```

    dt (float): Thời gian frame tính bằng giây.
    candidates (list): Danh sách Malware trong tầm (không dùng – RadarNode không bắn).

Returns:
    None: RadarNode không bắn projectile.

Side effects:
    - Gọi super().update() để xử lý slow, stun, burn.
    - Tích lũy tick_timer; trừ 5 HP khi tick_timer >= 1.0 giây.

```

Lớp SpeedNode (dòng 594)

```

Tower tăng tốc – bắn liên thanh với tốc độ cao nhất.

Kế thừa từ BasicNode nhưng với fire_rate nhanh gấp 5 lần. Dùng để tiêu diệt
nhanh những mục tiêu yếu hoặc đám quái.

Attributes:
    Kế thừa từ BasicNode. Nâng cấp fire_rate (5.0 phút/giây), damage thấp hơn BasicNode.

Usage:
    Tạo khi người chơi nâng cấp tháp BasicNode (từ nhánh Tăng Tốc Đánh I) lên SpeedNode
    trong UpgradeTree:

    tower = SpeedNode((row, col), graph)

```

Phương thức __init__(self, pos, graph, targeting) (dòng 609)

```

Khởi tạo SpeedNode: gọi super().__init__, ghi đè stats SpeedNode.

Args:
    pos (tuple): (row, col) vị trí ô WALL.
    graph (GridGraph): Bản đồ lưới.
    targeting (str): Chiến lược chọn mục tiêu. Mặc định "min_dist".

```

Lớp FireNode (dòng 626)

```

Tower lửa – bắn tạo ô lửa lại vị trí quái đang đứng tồn tại trong vài giây, gây sát thương
lâu dài, hiệu ứng đốt các quái đi qua.

Attributes:
    Kế thừa từ BasicNode. Thêm skill bắn ra "vết lửa" tồn tại vài giây tại vị trí quái trúng
    đạn, gây sát thương theo thời gian lên quái đứng trên đó và quái đi qua. Stats vết lửa lấy
    từ settings: FIRE_DAMAGE, TOWER_FIRE_DURATION.

Usage:
    Tạo khi người chơi nâng cấp tháp BasicNode (từ nhánh nâng cấp damage) lên FireNode
    trong UpgradeTree:

    tower = FireNode((row, col), graph)

```

Phương thức __init__(self, pos, graph, targeting) (dòng 638)

Khởi tạo FireNode: gọi `super().__init__`, ghi đè stats FireNode.

Args:

`pos (tuple)`: (row, col) vị trí ô WALL.
`graph (GridGraph)`: Bản đồ lưới.
`targeting (str)`: Chiến lược chọn mục tiêu. Mặc định "min_dist".

Phương thức `shoot(self, target)` (dòng 658)

Override `BasicNode.shoot()` để bắn ra lửa.

Args:

`target (Malware)`: Mục tiêu được bắn.

Returns:

`Projectile`: Đạn lửa bay từ tower đến target.

Lớp `SniperNode` (dòng 678)

Tower xạ thù – bắn sát thương cao nhưng đơn mục tiêu, tốc độ chậm.

Kế thừa từ `BasicNode` nhưng với damage cao gấp đôi (40 vs 25) và `fire_rate` chậm (0.5 vs 1.0). Tầm bắn lớn hơn để dễ bảo vệ.

Attributes:

Kế thừa từ `BasicNode`. Nâng cấp damage (40), giảm `fire_rate` (0.5), tăng tầm bắn (4).

Usage:

Tạo khi người chơi nâng cấp tháp `BasicNode` (từ nhánh Tăng Damage I) lên `SniperNode` trong `UpgradeTree`:

```
tower = SniperNode((row, col), graph)
```

Phương thức `__init__(self, pos, graph, targeting)` (dòng 693)

Khởi tạo `SniperNode`: gọi `super().__init__`, ghi đè stats `SniperNode`.

Args:

`pos (tuple)`: (row, col) vị trí ô WALL.
`graph (GridGraph)`: Bản đồ lưới.
`targeting (str)`: Chiến lược chọn mục tiêu. Mặc định "min_dist".

Lớp `FreezeNode` (dòng 710)

Tower đóng băng mạnh – quái đứng yên tại chỗ trong vài giây.

Attributes:

Kế thừa từ `IceWall`. Nhưng thay vì làm chậm thì đóng băng

Usage:

Tạo khi người chơi nâng cấp tháp IceWall (từ nhánh nâng cấp slow) lên FreezeNode trong UpgradeTree:

```
tower = FreezeNode((row, col), graph)
```

Phương thức `__init__(self, pos, graph, targeting)` (dòng 722)

Khởi tạo FreezeNode: gọi `super().__init__`, ghi đè stats FreezeNode.

Args:

`pos (tuple)`: (row, col) vị trí ô WALL.

`graph (GridGraph)`: Bản đồ lưới.

`targeting (str)`: Chiến lược chọn mục tiêu. Mặc định "min_dist".

Lớp SpreadNode (dòng 738)

Tower làm chậm lây lan – khi bắn trúng, không chỉ slow mục tiêu mà còn lan sang các quái trong 1 phạm vi nhất định.

Attributes:

Kế thừa từ IceWall. Nhưng thêm hiệu ứng lan sang các quái kề nhau trong 1 phạm vi.

Usage:

Tạo khi người chơi nâng cấp tháp IceWall (từ nhánh nâng cấp range) lên SpreadNode trong UpgradeTree:

```
tower = SpreadNode((row, col), graph)
```

Phương thức `__init__(self, pos, graph, targeting)` (dòng 749)

Khởi tạo SpreadNode: gọi `super().__init__`, ghi đè stats SpreadNode.

Args:

`pos (tuple)`: (row, col) vị trí ô WALL.

`graph (GridGraph)`: Bản đồ lưới.

`targeting (str)`: Chiến lược chọn mục tiêu. Mặc định "min_dist".

Phương thức `shoot(self, target)` (dòng 768)

Bắn + slow mục tiêu, projectile đánh dấu `tower_type="spread"` để game.py xử lý lan slow.

Cơ chế:

1. Slow mục tiêu bị bắn trúng ngay lập tức

2. Tạo projectile với `tower_type="spread"`

3. Game.py khi projectile hit: tìm tất cả malware trong `spread_range` xung quanh vị trí hit → slow tất cả

Args:

`target (Malware)`: Mục tiêu bị bắn trúng.

Returns:

Projectile: Đạn với tower_type="spread" để game.py xử lý lan slow.

Lớp **PoisonNode** (dòng 802)

Tower độc – không bắn đạn, gây damage liên tục cho tất cả quái trong phạm vi quét.

Attributes:

Kế thừa từ RadarNode. Override update() để gây damage thay vì bắn projectile.

Usage:

Tạo khi người chơi nâng cấp tháp RadarNode lên PoisonNode trong UpgradeTree:

```
tower = PoisonNode((row, col), graph)
```

Phương thức **__init__(self, pos, graph, targeting)** (dòng 813)

Khởi tạo PoisonNode: gọi super().__init__, ghi đè stats PoisonNode.

Args:

pos (tuple): (row, col) vị trí ô WALL.

graph (GridGraph): Bản đồ lưới.

targeting (str): Chiến lược chọn mục tiêu. Mặc định "min_dist".

Phương thức **update(self, dt, candidates)** (dòng 830)

Cập nhật cooldown và gây damage cho tất cả malware trong phạm vi.

Args:

dt (float): Thời gian frame tính bằng giây.

candidates (list): Danh sách Malware trong tầm quét (từ SpatialHash).

Returns:

None: PoisonNode không bắn projectile.

Side effects:

- Gây damage cho tất cả malware trong candidates mỗi lần cooldown reset.

- Xử lý stun timer và slow timer (sau slow hết, khôi phục fire_rate).

Note:

Khác với Tower.update() – PoisonNode không tìm mục tiêu đơn lẻ, không gọi shoot(),

và gây damage cho TẤT CẢ malware trong candidates thay vì chỉ 1 mục tiêu.

game.py xử lý PoisonNode khác các tower khác vì return value là None.

File: **entities/projectile.py**

Lớp **BaseProjectile** (dòng 10)

Lớp cơ sở cho tất cả projectiles.

Chứa logic chung: vị trí pixel, tốc độ, rendering, hit detection.

Subclass ghi đè update(dt) để định nghĩa cách di chuyển.

Attributes:

x (float): Tọa độ pixel X hiện tại của đạn.
y (float): Tọa độ pixel Y hiện tại của đạn.
damage (int): Sát thương gây ra.
speed (float): Tốc độ đạn tính bằng pixel/giây.
_hit (bool): True khi đạn đã kết thúc vòng đời.
color (tuple): Màu RGB đạn.
radius (int): Bán kính vẽ đạn.
tower_type (str): Loại tower/malware bắn ra (dùng cho sprite).

Phương thức `__init__(self, source_pos, damage, speed, tower_type, spread_range, slow_factor, slow_duration)` (dòng 27)

Khởi tạo BaseProjectile.

Args:

source_pos (tuple): Vị trí (row, col) ô lưới.
damage (int): Sát thương.
speed (float): Tốc độ tính bằng ô/giây (nhân CELL_SIZE → pixel/giây).
tower_type (str): Loại (basic, ice, radar, worm, etc.).
spread_range (float): Bán kính lan slow (cho SpreadNode projectile).
slow_factor (float): Hệ số slow khi lan (cho SpreadNode projectile).
slow_duration (float): Thời gian slow khi lan (cho SpreadNode projectile).

Phương thức `update(self, dt)` (dòng 55)

Cập nhật vị trí đạn. Override trong subclass.

Phương thức `has_hit(self)` (dòng 59)

Kiểm tra đạn kết thúc vòng đời.

Phương thức `apply_hit(self, game)` (dòng 63)

Hook called by game.py when projectile hits. Override in subclass for custom behavior.

Phương thức `draw(self, screen, camera_x, camera_y)` (dòng 67)

Vẽ đạn lên màn hình với camera offset.

Args:

screen: pygame.Surface để vẽ
camera_x: offset camera theo trục X
camera_y: offset camera theo trục Y

Phương thức `_draw_trojan_ranged_projectile(self, screen, screen_x, screen_y)` (dòng 145)

Vẽ projectile trojan_ranged: hình tròn gradient với trail (màu tím).

Phương thức `_draw_spyware_ranged_projectile(self, screen, screen_x, screen_y)` (dòng 176)

Vẽ projectile spyware_ranged: hình tròn gradient với trail (màu đỏ).

Phương thức `_draw_slowspy_ranged_projectile(self, screen, screen_x, screen_y)` (dòng 203)

Vẽ projectile slowspy_ranged: hình tròn gradient với trail (màu xanh dương).

Phương thức `_draw_lightspy_ranged_projectile(self, screen, screen_x, screen_y)` (dòng 230)

Vẽ projectile lightspy_ranged: hình tròn sáng vàng (gần trắng) với trail sáng.

Phương thức `_draw_fire_projectile(self, screen, screen_x, screen_y)` (dòng 257)

Vẽ projectile fire: quả cầu lửa với trail lửa.

Phương thức `_draw_speed_projectile(self, screen, screen_x, screen_y)` (dòng 285)

Vẽ projectile speed: tia sét vàng với trail điện.

Phương thức `_draw_sniper_projectile(self, screen, screen_x, screen_y)` (dòng 308)

Vẽ projectile sniper: mũi tên đỏ sắc với trail máu.

Phương thức `_draw_freeze_projectile(self, screen, screen_x, screen_y)` (dòng 331)

Vẽ projectile freeze: mảnh sông lạnh với trail băng.

Phương thức `_draw_spread_projectile(self, screen, screen_x, screen_y)` (dòng 354)

Vẽ projectile spread: khí độc tím với trail giãn nở.

Phương thức `_draw_poison_projectile(self, screen, screen_x, screen_y)` (dòng 378)

Vẽ projectile poison: quả cầu nọc độc với trail độc tố.

Phương thức `_draw_riposteware_projectile(self, screen, screen_x, screen_y)` (dòng 401)

Vẽ projectile riposte: quả cầu xanh lá (phản đạn) với glow sáng.

Lớp `Projectile` (dòng 425)

Dạn bắn từ Tower, bay theo đường thẳng đến Malware và gây damage khi chạm.

Hoạt động trong không gian pixel (float) thay vì ô lưới để animation mượt mà.

Mỗi frame, game.py gọi `update(dt)` để di chuyển đạn; khi `has_hit()` trả True thì game.py lọc đạn ra khỏi danh sách `self.projectiles`.

Attributes:

`target` (Malware): Tham chiếu malware đang bị nhắm. Có thể chết trước khi đạn đến.

Usage:

Không tạo trực tiếp – `Tower.shoot()` tạo và trả về, game.py quản lý vòng đời::

```
# Tower.shoot() trả về:
Projectile(source_pos=self.pos, target=target,
           damage=self.damage, speed=settings.PROJECTILE_SPEED)

# game.py mỗi frame:
for proj in self.projectiles:
    proj.update(dt)
self.projectiles = [p for p in self.projectiles if not p.has_hit()]
```

Phương thức `__init__(self, source_pos, target, damage, speed, tower_type, spread_range, slow_factor, slow_duration, source_tower)` (dòng 448)

Khởi tạo Projectile tại vị trí pixel tâm Tower, nhắm về phía target.

Args:

`source_pos` (tuple): Vị trí (row, col) ô lưới của Tower bắn ra.
`target` (Malware): Đối tượng Malware đang bị nhắm bắn.
`damage` (int): Sát thương gây ra khi chạm.
`speed` (float): Tốc độ đạn tính bằng ô/giây.
`tower_type` (str): Loại tower (basic, ice, radar).
`spread_range` (float): Bán kính lan slow (cho SpreadNode projectile).
`slow_factor` (float): Hệ số slow khi lan (cho SpreadNode projectile).
`slow_duration` (float): Thời gian slow khi lan (cho SpreadNode projectile).
`source_tower`: Tham chiếu Tower bắn ra đạn này (để phục vụ riposte, etc).

Note:

Đạn bay theo kiểu "homing" – tính lại hướng đến target mỗi frame, vì malware di chuyển trong khi đạn bay.

Phương thức `update(self, dt)` (dòng 471)

Di chuyển đạn về phía target mỗi frame theo vector đơn vị (homing).

Args:

`dt` (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật `self.x`, `self.y` theo hướng đến vị trí pixel hiện tại của target.
- Đặt `self._hit` = True khi đạn chạm target.

Lớp `MalwareProjectile` (dòng 505)

Đạn bắn từ Ranged Malware, bay thẳng đến goal position.

Khác với Tower Projectile (homing), MalwareProjectile bay thẳng đến một vị trí cố định (`goal_pos`) mà không theo dõi mục tiêu di chuyển.

Attributes:

`goal_pos` (tuple): Vị trí (row, col) ô lưới đích (server hoặc tower).
`goal_type` (str): Loại mục tiêu: "server" hoặc "tower".
`malware_type` (str): Loại malware bắn ra (worm, spyware, etc.).

Usage:

Tạo khi ranged malware ready to attack:

```
proj = MalwareProjectile(source_pos=malware.pos, goal_pos=malware.goal,
                        goal_type="server", damage=..., speed=...,
                        malware_type="worm")
self.projectiles.append(proj)
```



```

game.py cần xử lý hit result từ get_hit_info():
    if proj.has_hit():
        hit_info = proj.get_hit_info()
        # Apply damage đến server hoặc tower tại goal_pos

```

Phương thức `__init__(self, source_pos, goal_pos, goal_type, damage, speed, malware_type, affected_cells)` (dòng 529)

Khởi tạo MalwareProjectile.

Args:

```

source_pos (tuple): Vị trí (row, col) malware bắn ra.
goal_pos (tuple): Vị trí (row, col) mục tiêu (server hoặc tower).
goal_type (str): "server" hoặc "tower".
damage (int): Sát thương.
speed (float): Tốc độ tính bằng ô/giây.
malware_type (str): Loại malware (worm, spyware, etc.).
affected_cells (list): Danh sách ô bị ảnh hưởng (shock spread cho LightSpy_Ranged).

```

Phương thức `update(self, dt)` (dòng 548)

Di chuyển đạn thẳng về phía goal_pos.

Args:

```

dt (float): Thời gian frame tính bằng giây.

```

Side effects:

- Cập nhật self.x, self.y hướng đến goal_pos (cố định).
- Đặt self._hit = True khi đạn đến goal_pos.

Phương thức `apply_hit(self, game)` (dòng 575)

Áp dụng damage từ projectile đến server hoặc tower.

Args:

```

game (Game): Game object để truy cập server, towers, etc.

```

Side effects:

- Nếu goal_type == "server": Gọi game.server.take_damage(), đặt game.game_over = True nếu server bị phá.
- Nếu goal_type == "tower": Gọi tower.take_damage(), gọi game._on_tower_destroyed() nếu tower bị phá.

Note:

```

Ranged malware tự gọi apply_hit() từ game.py sau khi projectile kết thúc (_hit=True).

```

Phương thức `get_hit_info(self)` (dòng 599)

Trả về thông tin hit để game.py xử lý damage.

Returns:

```
dict: {"goal_pos": (row,col), "goal_type": "server"/"tower", "dmg": damage,
"affected_cells": [...]}
```

File: entities/fire_mark.py

Lớp FireMark (dòng 7)

Vết lửa trên 1 ô lưới - quái đi qua ô này sẽ nhận damage/giây.

Attributes:

pos (tuple): Vị trí ô lưới (row, col)
damage_per_sec (int): Sát thương mỗi giây
duration (float): Thời gian tồn tại (giây)
elapsed_time (float): Thời gian đã trôi qua
is_active (bool): Vết lửa còn hoạt động hay không
animation_frame (int): Frame hiện tại (0-17 từ spritesheet)

Phương thức __init__(self, pos, damage_per_sec, duration) (dòng 19)

Khởi tạo FireMark tại vị trí quái trúng đạn.

Args:

pos (tuple): (row, col) vị trí ô lưới
damage_per_sec (int): Sát thương/giây từ lửa
duration (float): Thời gian vết lửa tồn tại

Phương thức update(self, dt) (dòng 34)

Cập nhật timer và animation frame của vết lửa.

Args:

dt (float): Thời gian frame (giây)

Phương thức is_alive(self) (dòng 59)

Kiểm tra vết lửa còn hoạt động hay không.

Returns:

bool: True nếu vết lửa chưa hết thời gian tồn tại.

Phương thức affects_enemy(self, malware) (dòng 67)

Kiểm tra quái có đứng trên ô lửa không.

Args:

malware (Malware): Đối tượng Malware cần kiểm tra

Returns:

bool: True nếu quái đứng tại vị trí vết lửa

File: entities/bomb.py

Lớp Bomb (dòng 4)

Bom rơi ngẫu nhiên trên ô PATH trong wave – phát nổ sau timer_explode giây.

Tower có thể bắn hạ bomb trước khi nổ. Khi nổ: stun toàn bộ tower và trừ HP server. Nếu bị tiêu diệt bởi tower: vẫn trigger animation nổ nhưng KHÔNG gây stun/damage.

Attributes:

```
pos (tuple[int,int]): Vị trí (row, col) trên lưới PATH.  
hp (int): Máu bomb – về 0 khi bị tower bắn.  
timer_explode (float): Đếm ngược (giây) đến khi tự nổ.  
bomb_damage_to_server (int): Sát thương gây ra cho Server khi nổ.  
bomb_stun_duration (float): Thời gian stun tất cả tower (giây).  
exploded (bool): True khi bomb đã phát nổ (tự hết giờ hoặc bị tiêu diệt).  
bomb_dead (bool): True khi bomb đã bị xóa khỏi game.  
anim_timer (float): Thời gian đã trôi qua trong animation hiện tại.  
warning_timer (float): Timer hiệu ứng cảnh báo (~1 giây).
```

Usage::

```
bomb = Bomb(pos=(5, 10))  
bomb.update(dt)  
if bomb.is_exploded() and bomb.anim_timer >= 1.4:  
    data = bomb.get_explosion_data()
```

Phương thức `__init__(self, pos)` (dòng 29)

Khởi tạo Bomb tại vị trí pos với các thông số từ settings.

Args:

```
pos (tuple[int,int]): Vị trí (row, col) ô PATH trên lưới.
```

Phương thức `update(self, dt)` (dòng 45)

Cập nhật trạng thái của Bomb mỗi frame.

Args:

```
dt (float): Thời gian frame tính bằng giây.
```

Side effects:

- Giảm self.timer_explode theo dt.
- Khi self.timer_explode $\leq$ 0, đặt self.exploded = True để đánh dấu đã phát nổ.
- Cập nhật animation timer.
- Giảm warning_timer.

Phương thức `is_dead(self)` (dòng 73)

Kiểm tra bomb đã bị xóa khỏi game (bị tower tiêu diệt hoặc animation nổ xong).

Returns:

```
bool: True nếu self.bomb_dead == True.
```

Note:

Sau khi bom nổ (`is_exploded() == True`), `game.py` chỉ xóa nó khỏi danh sách sau khi animation chạy xong (`anim_timer >= 1.4`), lúc đó gọi hàm này để kiểm tra trước khi `remove`.

Phương thức `is_exploded(self)` (dòng 86)

Kiểm tra bomb đã phát nổ chưa (hết timer hoặc bị tower tiêu diệt).

Returns:

bool: True nếu `self.exploded == True`.

Note:

Không đồng nghĩa với `is_dead()` – bomb vẫn cần phát xong animation nổ trước khi `is_dead()` trả True. Giữa `is_exploded() == True` và `is_dead() == True` khoảng 1.4 giây animation.

Phương thức `take_damage(self, amount)` (dòng 98)

Trừ HP của Bomb khi bị tấn công.

Args:

amount (int): Lượng sát thương cần trừ (≥ 0).

Side effects:

- Giảm `self.hp` (không đặt lower bound, có thể âm).
- Nếu `HP <= 0`, đặt `bomb_dead = True` và trigger explosion.

Phương thức `get_explosion_data(self)` (dòng 117)

Trả về dữ liệu khi bomb nổ: sát thương server + stun towers.

Returns:

dict với keys: "server_damage", "stun_duration"

Phương thức `get_render_data(self, cell_size)` (dòng 128)

Trả về dữ liệu vẽ bomb cho Y-sorting trong `game.py`.

Args:

cell_size (int): Kích thước ô lưới (pixel).

Returns:

dict với keys: 'surf', 'pos', 'sort_y', 'type', 'warning' cho Y-sorting layer.

Lớp `BossBomb` (dòng 195)

Bom của Boss – có damage tùy chỉnh và loại bomb (normal hoặc atomic).

Dùng cho `FlyingDemon`: bomb bình thường (-300 HP) hoặc atomic bomb (-10000 HP).

Thời gian nổ được random từ `min_explode` đến `max_explode`.

Attributes:

```
bomb_type (str): "normal" hoặc "atomic" – ảnh hưởng damage  
is_atomic (bool): True nếu là atomic bomb (gây -10000 HP)
```

Phương thức `__init__(self, pos, bomb_type, damage, stun_duration, explode_time)` (dòng 206)

Khởi tạo BossBomb.

Args:

```
pos (tuple[int,int]): Vị trí (row, col)  
bomb_type (str): "normal" hoặc "atomic"  
damage (int): Sát thương gây cho server  
stun_duration (float): Thời gian stun tháp  
explode_time (float): Thời gian nổ (giây)
```

File: `entities/server.py`

Lớp `Server` (dòng 8)

Mục tiêu chính mà Malware cố gắng đến và tấn công.

Server nằm tại vị trí cố định trong lưới (`server_pos`), có máu riêng.
Mỗi Malware tới server trừ 1 HP. Khi HP về 0 → GAME OVER.

Attributes:

```
pos (tuple[int,int]): Tọa độ ô lưới (row, col) trên map.  
hp (int): Máu server hiện tại.  
max_hp (int): Máu tối đa (từ settings.SERVER_MAX_HP).  
_anim_frame (int): Frame animation hiện tại (0-3).  
_anim_timer (float): Đếm thời gian (giây) đến frame kế tiếp.
```

Usage::

```
server = Server(pos=(12, 12))  
server.take_damage(1)  
if server.is_destroyed():  
    print("Game Over!")  
render_data = server.get_render_data(cell_size=48)
```

Phương thức `__init__(self, pos)` (dòng 30)

Khởi tạo Server tại vị trí `pos`.

Args:

```
pos (tuple[int,int]): Tọa độ ô lưới (row, col).
```

Side effects:

- Đặt `self.hp = settings.SERVER_MAX_HP`.
- Đặt `self._anim_frame = 0`, `self._anim_timer = 0.0`.
- Đặt `self.poison_timer = 0.0`, `self.poison_end_time = 0.0`.

Phương thức `take_damage(self, amount)` (dòng 49)

Trừ máu server khi Malware tấn công.

Args:

amount (int): Lượng sát thương cần trừ (≥ 0).

Side effects:

- Giảm self.hp (không đặt lower bound, có thể âm).

Phương thức `is_destroyed(self)` (dòng 60)

Kiểm tra server đã bị phá hủy ($HP \leq 0$) chưa.

Returns:

bool: True nếu self.hp ≤ 0 , False nếu còn sống.

Phương thức `get_poison(self, poison_duration)` (dòng 68)

Áp dụng độc lên server (cộng dồn).

Args:

poison_duration (float): Thời gian độc tác động thêm (giây).

Side effects:

- Cộng thêm poison_duration vào self.poison_end_time.
- Poison sẽ gây sát thương mỗi 1.5 giây cho đến khi hết thời gian.

Phương thức `update(self, dt)` (dòng 80)

Cập nhật animation server và xử lý độc mỗi frame.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Tích lũy dt vào self._anim_timer, chuyển frame kế tiếp khi đủ khoảng cách.
- Xử lý độc: tích lũy thời gian, gây sát thương mỗi 1.5 giây.

Phương thức `get_render_data(self, cell_size)` (dòng 109)

Tạo render data cho Server (sprite animation) cho Y-Sort.

Args:

cell_size (int): Kích thước ô lưới (pixel) – thường 48.

Returns:

```
dict: {
    'surf': pygame.Surface, # sprite hiện tại.
    'pos': (pixel_x, pixel_y), # vị trí để blit (bottom-center aligned).
    'sort_y': float, # tọa độ Y để Y-Sort.
}
```

Note:

```
Server lấy sprite từ cache key "server" (4 frame animation).  
Nếu không có sprite, fallback vẽ hình tròn xanh.  
HP hiển thị riêng trong HUD, không vẽ trên sprite.
```

Phương thức `draw(self, screen, cell_size)` (dòng 155)

Vẽ server lên màn hình (fallback nếu không dùng Y-Sort).

Args:

`screen` (pygame.Surface): Surface của sổ game.
`cell_size` (int): Kích thước ô lưới (pixel).

Side effects:

- Vẽ sprite server và HP bar lên screen.

Note:

Phương thức này là fallback – bình thường game.py dùng `get_render_data()` để đảm bảo Y-Sort đúng.

File: `entities/wall.py`

Lớp `Wall` (dòng 4)

Tường được đặt bởi người chơi, tồn tại 30s sau đó biến mất.

Khi đặt: cell thay đổi WALL → lưu lại cell cũ.

Khi mất: cell trở lại giá trị cũ (thường là PATH), enemy tính lại đường đi.

Phương thức `__init__(self, pos, original_cell)` (dòng 10)

Khởi tạo Wall tại vị trí pos.

Args:

`pos`: (row, col) vị trí đặt tường trên lưới.
`original_cell`: Giá trị cell ban đầu (thường `Celltype.PATH`).

Phương thức `update(self, dt)` (dòng 22)

Cập nhật timer tồn tại tường mỗi frame.

Args:

`dt`: Thời gian frame (giây).

Phương thức `is_dead(self)` (dòng 34)

Trả về True nếu tường đã hết thời gian tồn tại.

Returns:

bool: True nếu `wall_dead = True` (`duration <= 0`).

File: `entities/shock_effect.py`

Lớp `ShockEffect` (dòng 7)

Hiệu ứng shock từ LightSpy - tháp bị lan truyền nhận hiệu ứng tương tác.

Attributes:

`pos` (tuple): Vị trí ô lưới của tháp bị tấn công
`duration` (float): Thời gian hiệu ứng tồn tại (giây)
`elapsed_time` (float): Thời gian đã trôi qua
`is_active` (bool): Hiệu ứng còn hoạt động hay không
`animation_frame` (int): Frame hiện tại (8-14, ping-pong animation)

Phương thức `__init__(self, pos, duration)` (dòng 18)

Khởi tạo ShockEffect tại vị trí tháp bị LightSpy tấn công.

Args:

`pos` (tuple): (row, col) vị trí ô lưới
`duration` (float): Thời gian hiệu ứng tồn tại (mặc định 0.5 giây)

Phương thức `update(self, dt)` (dòng 32)

Cập nhật animation frame của hiệu ứng shock.

Args:

`dt` (float): Thời gian frame (giây)

Side effects:

- Cập nhật `animation_frame` theo `frame_sequence`
- Đặt `is_active` = False khi hết `duration`

Phương thức `is_alive(self)` (dòng 55)

Kiểm tra hiệu ứng còn hoạt động hay không.

Returns:

`bool`: True nếu hiệu ứng chưa hết thời gian tồn tại.

File: `systems/spatial_hash.py`

Lớp `SpatialHash` (dòng 20)

Spatial hash map dùng Python dict làm lõi lưu trữ.

Chia lưới thành các "bucket" (vùng ô vuông `bucket_cell × bucket_cell`).

Mỗi bucket là một key (brow, bcol) trong dict. Khi Tower hỏi "ai trong tầm?", chỉ cần kiểm tra vài bucket quanh Tower thay vì toàn bộ malware.

Attributes:

`_bucket_cell` (int): Số ô lưới mỗi chiều của 1 bucket. Mặc định 2 → vùng 2×2 ô.
`_bucket_rows` (int): Số bucket theo chiều dọc = `grid_rows // bucket_cell`.
`_bucket_cols` (int): Số bucket theo chiều ngang = `grid_cols // bucket_cell`.
`_buckets` (dict): {(brow, bcol): list[entity]} – chỉ chứa bucket có ít nhất 1 entity.

Usage:


```

game.py khởi tạo một lần khi load level, sau đó cập nhật mỗi frame::

    spatial_hash = SpatialHash(grid_rows=15, grid_cols=15, bucket_cell_size=2)

    # Khi malware spawn:
    spatial_hash.insert(malware)

    # Mỗi frame, sau malware.update(dt):
    spatial_hash.update_position(malware, old_pos)

    # Tower hỏi danh sách trong tầm:
    candidates = spatial_hash.query_range(tower.pos, tower.range)

    # Khi malware chết hoặc đến server:
    spatial_hash.remove(malware)

```

Note:

Dict dùng tuple (brow, bcol) làm key – không có collision giả tạo như hash dựa trên modulo. Hai bucket ở góc đối nhau luôn có key khác nhau.

Phương thức `__init__(self, grid_rows, grid_cols, bucket_cell_size)` (dòng 55)

Khởi tạo SpatialHash với dict rỗng.

Args:

`grid_rows` (int): Số hàng thực tế của map – đọc từ config["rows"] trong JSON.
 Không dùng settings.GRID_ROWS vì các level có kích thước khác nhau.
`grid_cols` (int): Số cột thực tế của map – đọc từ config["cols"] trong JSON.
`bucket_cell_size` (int): Số ô lưới mỗi chiều của 1 bucket. Mặc định 2.
 Bucket nhỏ → nhiều bucket hơn, query chính xác hơn nhưng tốn bộ nhớ hơn.
 Bucket lớn → ít bucket, query nhanh hơn nhưng lọc nhiều false-positive hơn.

Note:

Truyền `grid_rows/cols` từ JSON thay vì settings để hỗ trợ map nhiều kích thước.
 Nếu dùng settings cứng → `_bucket_rows/_bucket_cols` sai → `query_range()` clamp sai biên → bỏ sót malware ở rìa map.

Example::

```

# map 15×15, bucket_cell=2 → _bucket_rows=7, _bucket_cols=7
SpatialHash(15, 15, 2)

# map 20×30, bucket_cell=2 → _bucket_rows=10, _bucket_cols=15
SpatialHash(20, 30, 2)

```

Phương thức `_hash(self, row, col)` (dòng 89)

Ảnh xạ tọa độ ô lưới (row, col) → key bucket (brow, bcol).

Đây là "hàm hash" của class – quyết định entity ở ô nào thuộc bucket nào.

Args:

row (int): Hàng ô lưới.
col (int): Cột ô lưới.

Returns:

tuple: (brow, bcol) – key dict cho bucket chứa ô (row, col).

Example::

```
# bucket_cell_size = 2
_hash(0, 0) → (0, 0)   # ô (0,0) (0,1) (1,0) (1,1) cùng 1 bucket
_hash(0, 1) → (0, 0)   # 1//2 = 0
_hash(2, 0) → (1, 0)   # 2//2 = 1, sang bucket mới
_hash(3, 7) → (1, 3)   # 3//2=1, 7//2=3
```

Note:

Dùng tuple thay vì integer tránh collision giả tạo – hai bucket ở gốc đối nhau sẽ không bao giờ chung key.

Phương thức insert(self, entity) (dòng 121)

Thêm entity vào bucket tương ứng với vị trí hiện tại của nó.

Args:

entity: Object có thuộc tính .pos = (row, col). Trong game là Malware.

Side effects:

- Tạo bucket mới trong _buckets nếu chưa tồn tại.
- Append entity vào list của bucket tương ứng.

Usage:

game.py gọi khi malware mới được spawn::

```
spatial_hash.insert(malware)
```

Example::

```
# malware.pos = (3, 5), bucket_cell=2
# key = (3//2, 5//2) = (1, 2)
# _buckets[(1,2)] = [malware] ← bucket mới được tạo
```

Phương thức remove(self, entity) (dòng 149)

Xóa entity khỏi bucket của nó.

Args:

entity: Object có thuộc tính .pos = (row, col). Vị trí phải là vị trí HIỆN TẠI của entity (không phải vị trí cũ).

Side effects:

- Xóa entity khỏi list của bucket tương ứng với entity.pos.
- Xóa luôn key khỏi dict nếu bucket trở nên rỗng (giữ dict gọn).

Usage:

game.py gọi khi malware chết hoặc đến server::

```
spatial_hash.remove(malware)
```

Note:

list.remove() xóa theo tham chiếu object, không theo giá trị.
Nếu entity không tồn tại trong bucket, không gây lỗi (kiểm tra trước).

Phương thức `update_position(self, entity, old_pos)` (dòng 179)

Cập nhật vị trí entity trong dict khi entity di chuyển sang ô mới.

Method đặc thù của game – entity DI CHUYỂN nên bucket key có thể thay đổi mỗi frame. Nếu entity vẫn trong cùng bucket → không làm gì (tối ưu hóa).

Args:

entity: Object có .pos = (row, col) ĐÃ CẬP NHẬT (vị trí mới sau update).
old_pos (tuple): Vị trí (row, col) CŨ – trước khi malware.update(dt) chạy.
game.py phải lưu old_pos = malware.pos TRƯỚC khi gọi malware.update().

Side effects:

- Xóa entity khỏi bucket cũ (tính từ old_pos).
- Thêm entity vào bucket mới (tính từ entity.pos hiện tại).
- Xóa bucket cũ khỏi dict nếu trở nên rỗng.

Usage:

game.py trong `_update_malwares()`:

```
old_pos = malware.pos
malware.update(dt)
if malware.pos != old_pos:
    spatial_hash.update_position(malware, old_pos)
```

Example::

```
# Malware (2,4)→(2,5), bucket_cell=2
# old_key = (1,2), new_key = (1,2) → CÙNG BUCKET → không làm gì

# Malware (2,5)→(2,6), bucket_cell=2
# old_key = (1,2), new_key = (1,3) → KHÁC → chuyển bucket
```

Phương thức `query_range(self, center_pos, radius)` (dòng 229)

Trả về tất cả entity trong bán kính radius ô quanh center_pos.

Đây là method cốt lõi của spatial hash – lý do class này tồn tại.
Thay vì duyệt toàn bộ entity, chỉ kiểm tra các bucket trong vùng bao quanh.

Args:

`center_pos (tuple)`: Vị trí (row, col) ô lưới của Tower đang hỏi.
`radius (int)`: Bán kính tấn công của Tower (đơn vị ô lưới, Manhattan distance).

Returns:

`list`: Tất cả entity trong tầm (Manhattan distance $\leq$ radius).
Có thể rỗng nếu không có entity nào gần Tower.

Note:

- 3 bước xử lý:
1. Xác định hình chữ nhật bucket bao quanh vòng tròn bán kính radius.
 2. Duyệt từng bucket trong hình chữ nhật đó (bỏ qua bucket rỗng).
 3. Lọc bằng Manhattan distance thực tế – bucket hình chữ nhật có thể bao gồm góc nằm ngoài vòng tròn thực.

Tại sao không cần seen set?

Dict dùng (brow, bcol) làm key → mỗi bucket là list RIÊNG BIỆT → không trùng.

Example::

```
# Tower tại (6,6), radius=3, bucket_cell=2
# min_brow=(6-3)//2=1, max_brow=(6+3)//2+1=5
# min_bcol=1, max_bcol=5 → duyệt 16 bucket
candidates = spatial_hash.query_range((6, 6), 3)
```

Phương thức `clear(self)` (dòng 280)

Xóa toàn bộ entity – reset dict về rỗng.

Side effects:

- `self._buckets` trở thành dict rỗng {}.

Usage:

game.py gọi khi chuyển level hoặc restart game::

```
spatial_hash.clear()
```

Phương thức `bucket_count(self)` (dòng 298)

Trả về số bucket đang có ít nhất 1 entity.

Returns:

`int`: Số lượng key trong `_buckets` (= số bucket không rỗng).

Usage:

Dùng khi debug để quan sát mức độ phân tán của spatial hash::

```
print(spatial_hash.bucket_count())
# Nếu bằng tổng malware → mỗi malware 1 bucket riêng → bucket quá nhỏ
```

Phương thức `entities_in_bucket(self, row, col)` (dòng 313)

Trả về list entity trong bucket chứa ô (row, col).

Args:

row (int): Hàng ô lưới (không phải hàng bucket).
col (int): Cột ô lưới (không phải cột bucket).

Returns:

list: Danh sách entity trong bucket tương ứng.
Trả về [] nếu bucket không tồn tại (không raise KeyError).

Usage:

Dùng khi debug để kiểm tra một ô lưới cụ thể đang có entity nào::

```
entities = spatial_hash.entities_in_bucket(3, 5)
```

File: `systems/upgrade_tree.py`

Lớp `SkillNode` (dòng 1)

Một node trong cây nâng cấp tower – đại diện cho một lựa chọn upgrade.

Mỗi node có thể là:

- "base": Node gốc (đã unlock sẵn, cost=0).
- "stat_buff": Tăng chỉ số tower (damage, fire_rate, range, slow_duration).
- "tower_upgrade": Thay thế tower bằng class mới (BasicNode → FireNode).

Attributes:

node_id (str): ID duy nhất của node trong cây.
name (str): Tên hiển thị trong UpgradeMenu.
cost (int): Tiền cần để mở khóa node này.
upgrade_type (str): "base", "stat_buff", hoặc "tower_upgrade".
tower_class (str | None): Tên class tower khi upgrade_type == "tower_upgrade".
stat_buff (dict | None): {stat_name: delta} khi upgrade_type == "stat_buff".
is_unlocked (bool): True nếu đã được mua. Mặc định False.
parent (SkillNode | None): Node cha trong cây. None nếu là root.
children (list[SkillNode]): Danh sách node con có thể mở sau node này.

Usage::

```
root = SkillNode("basic_root", "BasicNode", cost=0, upgrade_type="base",
                 tower_class="BasicNode")
root.is_unlocked = True
child = SkillNode("fire_upgrade", "FireNode", cost=120,
                 upgrade_type="tower_upgrade", tower_class="FireNode")
root.add(child)
```

Phương thức `__init__(self, node_id, name, cost, upgrade_type, tower_class, stat_buff)` (dòng 29)

Khởi tạo SkillNode với metadata upgrade.

Args:

`node_id (str)`: ID duy nhất trong UpgradeTree.
`name (str)`: Tên hiển thị trong UpgradeMenu.
`cost (int)`: Tiền người chơi phải trả để mở khóa.
`upgrade_type (str)`: "base", "stat_buff", hoặc "tower_upgrade".
`tower_class (str | None)`: Tên class tower đích (cho tower_upgrade).
`stat_buff (dict | None)`: Map stat → delta (cho stat_buff).

Phương thức `add(self, child_node)` (dòng 49)

Gắn child_node làm con của node này và đặt parent tương ứng.

Args:

`child_node (SkillNode)`: Node con cần thêm vào cây.

Side effects:

- Append child_node vào self.children.
- Đặt child_node.parent = self.

Phương thức `can_be_unlocked(self, money)` (dòng 62)

Kiểm tra xem node này có thể được mở khóa hay không dựa trên trạng thái của player.

Args:

`money (int)`: Số tiền hiện tại của player.

Returns:

`bool`: True nếu node cha đã unlock AND player đủ tiền, False ngược lại.

Note:

- Nếu là root node (parent=None), không cần kiểm tra parent.
- Cost = 0 (root) luôn có thể unlock.

Phương thức `unlock(self)` (dòng 81)

Mở khóa node này (người chơi đã trả tiền).

Side effects:

- Đặt self.is_unlocked = True.

Note:

Không tự trừ tiền – gọi UpgradeTree.try_unlock() hoặc UpgradeMenu.try_upgrade() để trừ tiền và gọi unlock() an toàn.

Lớp UpgradeTree (dòng 92)

Cây nâng cấp tower – quản lý toàn bộ SkillNode theo cấu trúc cây.

Mỗi tower loại (BasicNode, IceWall, RadarNode) có một UpgradeTree riêng.
game.py tạo các tree khi load level và lưu vào self.upgrade_trees.

Attributes:

root (SkillNode): Node gốc của cây (luôn is_unlocked=True, cost=0).
nodes (dict[str, SkillNode]): Map node_id → SkillNode để tra cứu O(1).

Usage::

```
tree = create_basic_upgrade_tree()
menu = UpgradeMenu(tower, tree)
success, new_money = tree.try_unlock("fire_upgrade", current_money)
```

Phương thức `__init__(self, root)` (dòng 108)

Khởi tạo UpgradeTree từ root node và đánh chỉ số toàn bộ cây.

Args:

root (SkillNode): Node gốc của cây nâng cấp (cost=0, đã unlock).

Side effects:

- Gọi `_index_nodes(root)` để xây dựng self.nodes dict ngay khi khởi tạo.

Phương thức `_index_nodes(self, node)` (dòng 121)

Đánh chỉ số tất cả nodes để dễ tìm kiếm bằng node_id.

Args:

node (SkillNode): Node hiện tại để index.

Side effects:

- Thêm node vào self.nodes[node.node_id].
- Đệ quy gọi trên tất cả children.

Note:

Gọi từ `__init__()` một lần duy nhất. Tạo DFS traversal toàn bộ cây.

Phương thức `get_node(self, node_id)` (dòng 138)

Lấy node theo ID duy nhất.

Args:

node_id (str): ID của node cần lấy.

Returns:

SkillNode | None: Node nếu tồn tại, None ngược lại.

Phương thức `get_available_upgrades(self, current_node_id)` (dòng 149)

Lấy danh sách upgrade có thể làm được từ node hiện tại.

Args:

`current_node_id (str)`: ID của node hiện tại (thường là tower class).

Returns:

`list[SkillNode]`: Danh sách các node con (upgrades khả dụng).
Trả về [] nếu node không tồn tại hoặc node này là leaf.

Note:

UpgradeMenu sử dụng hàm này để hiển thị danh sách nút upgrade.

Phương thức `try_unlock(self, node_id, money)` (dòng 167)

Thử mở khóa node nếu điều kiện được thỏa mãn.

Args:

`node_id (str)`: ID của node cần mở khóa.
`money (int)`: Số tiền hiện tại của player.

Returns:

`tuple[bool, int]`:
- `bool`: True nếu mở khóa thành công, False nếu thất bại.
- `int`: Số tiền còn lại (trừ cost nếu thành công, giữ nguyên nếu thất bại).

Side effects:

- Gọi `node.unlock()` nếu điều kiện thỏa mãn (node tồn tại, parent unlock, đủ tiền).

Note:

Gọi từ `UpgradeMenu.try_upgrade()` sau khi player click nút upgrade.

Phương thức `create_basic_upgrade_tree()` (dòng 194)

Tạo và trả về `UpgradeTree` đầy đủ cho `BasicNode`.

Cấu trúc cây:

```
BasicNode (root)
├── Damage I → SniperNode, FireNode
└── Speed I  → SpeedNode
```

Returns:

`UpgradeTree`: Cây nâng cấp với root đã unlock, sẵn sàng dùng cho `UpgradeMenu`.

Phương thức `create_ice_upgrade_tree()` (dòng 273)

Tạo và trả về `UpgradeTree` đầy đủ cho `IceWall`.

Cấu trúc cây:

```
IceWall (root)
├── Range I      → SpreadNode
└── Slow I       → FreezeNode
```


Returns:

UpgradeTree: Cây nâng cấp với root đã unlock, sẵn sàng dùng cho UpgradeMenu.

Phương thức `create_radar_upgrade_tree()` (dòng 343)

Tạo và trả về UpgradeTree đầy đủ cho RadarNode.

Cấu trúc cây:

```
RadarNode (root)
└─ Range I → PoisonNode
```

Returns:

UpgradeTree: Cây nâng cấp với root đã unlock, sẵn sàng dùng cho UpgradeMenu.

File: `systems/shock_spread.py`

Phương thức `shock_spread(self, graph, pos)` (dòng 5)

Tính toán các ô bị ảnh hưởng bởi hiệu ứng shock khi một Malware bị tiêu diệt.

Args:

`self (LightSpy)`: Đối tượng LightSpy gọi hàm này (dùng `self.graph` nội bộ).
`graph (GridGraph)`: Đồ thị lưới của bản đồ, dùng để xác định ô kề nhau.
`pos (tuple[int, int])`: Tọa độ ô lưới nơi Tower bị điện giật (`row, col`).

Returns:

`list[tuple[int, int]]`: Danh sách tọa độ các ô bị ảnh hưởng bởi shock.

File: `systems/audio.py`

Lớp `AudioManager` (dòng 5)

Singleton quản lý âm thanh và nhạc nền cho game.

Chỉ tồn tại một instance duy nhất – truy cập qua biến module `audio_manager`.

Tự động disable nếu `pygame.mixer` không khởi tạo được.

Attributes:

`enabled (bool)`: True nếu `pygame.mixer` hoạt động và audio được bật.
`sounds (dict)`: Map key → `pygame.mixer.Sound` đã tải.
`music_playing (str | None)`: Key nhạc nền đang phát, None nếu chưa phát.

Usage::

```
from systems.audio import audio_manager
audio_manager.play_sound("tower_shoot")
audio_manager.play_music("music_gameplay")
audio_manager.stop_music()
```

Phương thức `__new__(cls)` (dòng 25)

Tạo hoặc trả về singleton instance của AudioManager.

Returns:

AudioManager: Instance duy nhất, tạo mới nếu chưa tồn tại.

Phương thức `__init__(self)` (dòng 36)

Khởi tạo AudioManager: init mixer và load toàn bộ sounds từ settings.

Side effects:

- Gọi `pygame.mixer.init()` một lần duy nhất.
- Gọi `_load_sounds()` để tải SFX từ `settings.SOUNDS`.
- Nếu mixer lỗi: đặt `enabled = False`, audio bị vô hiệu hóa.

Phương thức `_load_sounds(self)` (dòng 61)

Tải tất cả sound effects vào memory.

Phương thức `play_sound(self, key)` (dòng 85)

Phát một sound effect theo key.

Args:

key (str): Key của sound trong `self.sounds` (khớp với `settings.SOUNDS`).

Side effects:

- Gọi `pygame.mixer.Sound.play()` nếu `enabled` và key tồn tại.
- Không làm gì nếu audio bị disable hoặc key không tìm thấy.

Phương thức `play_music(self, key, loops)` (dòng 99)

Phát nhạc nền (ogg/mp3) theo key; bỏ qua nếu đang phát cùng track.

Args:

key (str): Key nhạc nền trong `settings.SOUNDS` (phải bắt đầu bằng `"music_"`).
loops (int): Số lần lặp (-1 = vô hạn). Mặc định -1.

Side effects:

- Load và phát file nhạc qua `pygame.mixer.music`.
- Cập nhật `self.music_playing = key` khi phát thành công.
- Không làm gì nếu audio disable hoặc đang phát cùng key.

Phương thức `stop_music(self)` (dòng 138)

Dừng nhạc nền đang phát và reset trạng thái.

Side effects:

- Gọi `pygame.mixer.music.stop()` nếu audio enabled.
- Đặt `self.music_playing = None`.

File: `ui/sprites.py`

Phương thức `init()` (dòng 17)

Pre-load toàn bộ sprites vào `_cache`. Phải gọi sau `pygame.init()` và `display.set_mode()`.

Side effects:

- Lưu tất cả `Surface`/`list[Surface]` vào `_cache` với các key chuẩn.
- Gọi `_load_map_tiles()` để load tile `PATH/WALL` từ ảnh thực.
- Tạo procedural fallback cho mọi sprite thiếu file.

Note:

`convert_alpha()` yêu cầu `display` đã được khởi tạo.

Các key cache: `"tower_basic"`, `"trojan_Run"`, `"fireworm_Walk"`, `"shadow_Walk"`, v.v.

Phương thức `get(name)` (dòng 422)

Lấy sprite đã load từ cache theo tên key.

Args:

`name (str)`: Tên sprite key (vd. `"trojan_Run"`, `"tower_basic"`, `"proj_ice"`).

Returns:

`pygame.Surface` | `list[pygame.Surface]` | `None`:

`Surface` đơn (`tower`, `projectile`), `list` frame (animated sprite),

hoặc `None` nếu key không tồn tại trong cache.

Phương thức `get_temp_wall()` (dòng 436)

Lấy sprite tường tạm thời (đỏ, variant cuối cùng của `WALL_FILES`).

Phương thức `action_frame_count(key)` (dòng 444)

Trả về số frame của một sprite key trong cache.

Args:

`key (str)`: Tên sprite key cần kiểm tra.

Returns:

`int`: Số frame nếu key tồn tại và là `list`. 4 nếu không tìm thấy (fallback mặc định).

Phương thức `_load_spaced_frames(path, frame_width, frame_height, frame_spacing, num_frames, target_size, plus)` (dòng 461)

Load frames từ spritesheet với `spacing/padding` giữa các frame.

Args:

`path (str)`: Đường dẫn spritesheet.

`frame_width (int)`: Chiều rộng mỗi frame.

`frame_height (int)`: Chiều cao mỗi frame.

`frame_spacing (int)`: Tổng khoảng cách giữa frame (`frame_width` + padding).

`num_frames (int)`: Số frame cần load.

`target_size (tuple)`: Kích thước đích (`w`, `h`).

Returns:

list: Danh sách pygame.Surface đã scale.

Phương thức `_create_attack_effect(size, num_frames)` (dòng 498)

Tạo animated purple ring + tail effect khi boss drop bomb hoặc attack server.

Effect: Vòng tím phát triển từ giữa ra ngoài, với tail particles rải rác quanh vòng.

Animation smooth từ frame 0 (nhỏ) -> frame end (lớn, fade out).

Args:

size (int): Kích thước surface (size × size).

num_frames (int): Số frame animation (mặc định 12).

Returns:

list[pygame.Surface]: Danh sách frame animation SRCALPHA.

Phương thức `_img(path, w, h)` (dòng 569)

Load và scale một ảnh đơn từ file. Trả về placeholder màu magenta nếu lỗi.

Args:

path (str): Đường dẫn file ảnh (PNG/JPG).

w (int): Chiều rộng đích sau khi scale (pixel).

h (int): Chiều cao đích sau khi scale (pixel).

Returns:

pygame.Surface: Surface đã scale đúng kích thước (w × h).

Surface magenta bán trong suốt (200, 0, 200, 160) nếu file không tìm thấy.

Phương thức `_sheet(path, fw, fh, max_frames, target)` (dòng 591)

Slice spritesheet thành danh sách frame Surface, tùy chọn scale.

Đọc ảnh spritesheet và cắt thành các frame theo kích thước fw × fh.

Frame đi từ trái sang phải, trên xuống dưới.

Args:

path (str): Đường dẫn file spritesheet.

fw (int): Chiều rộng mỗi frame trong sheet (pixel).

fh (int): Chiều cao mỗi frame trong sheet (pixel).

max_frames (int): Số frame tối đa cần lấy. Mặc định 999 (lấy tất cả).

target (tuple | None): (w, h) kích thước đích sau khi scale.

None = giữ nguyên kích thước gốc fw × fh.

Returns:

list[pygame.Surface]: Danh sách frame Surface đã scale (nếu có target).

List chứa 1 placeholder magenta nếu file không tìm thấy.

Phương thức `_sheet_row(path, fw, fh, row_idx, max_frames, target)` (dòng 632)

Lấy một hàng cụ thể từ spritesheet grid.

Args:

path (str): Đường dẫn file spritesheet.
fw (int): Chiều rộng mỗi frame.
fh (int): Chiều cao mỗi frame.
row_idx (int): Chỉ số hàng (0-indexed).
max_frames (int): Số frame tối đa từ hàng này.
target (tuple | None): Kích thước scale đích.

Returns:

list[pygame.Surface]: Frame từ hàng được chỉ định.

Phương thức `_pulse_tint(base, frame)` (dòng 672)

Tạo hiệu ứng glow pulse nhẹ cho server sprite theo frame number.

Args:

base (pygame.Surface): Surface gốc của server sprite.
frame (int): Chỉ số frame (0-3) – xác định cường độ glow theo sin.

Returns:

pygame.Surface: Bản sao của base với lớp tint cyan nhạt đề lên.
Cường độ tint dao động theo $\sin(\text{frame} \times \pi/2)$.

Phương thức `_load_map_tiles(cs)` (dòng 696)

Load tile ảnh cho PATH và WALL từ thư mục data/sprites/Map/.

Xử lý ảnh PATH (2 biến thể, overlay tối nhẹ) và WALL (3 biến thể, tạo hiệu ứng pseudo-3D bằng cách kéo dài thân tường theo TALL_FACTOR).
Kết quả lưu vào `_cache["path"]` và `_cache["wall"]`.

Args:

cs (int): Kích thước ô lưới (pixel) – settings.CELL_SIZE.

Side effects:

- Lưu list Surface vào `_cache["path"]` và `_cache["wall"]`.
- Nếu load thất bại → lưu list Surface trống làm fallback.

Note:

Gọi bởi `init()` sau `pygame.display.set_mode()` để `convert_alpha()` hoạt động.
TALL_FACTOR (settings) xác định chiều cao thân tường so với cs.

Phương thức `_wall_proc(cs)` (dòng 787)

Tạo procedural wall tile màu xanh đậm làm fallback khi thiếu ảnh.

Args:

cs (int): Kích thước ô lưới (pixel).

Returns:

pygame.Surface: Surface $cs \times cs$ với pattern đường kẻ và vòng tròn trung tâm.

Phương thức `_path_proc(cs)` (dòng 812)

Tạo procedural path tile màu tím đậm với grid lines làm fallback.

Args:

`cs (int)`: Kích thước ô lưới (pixel).

Returns:

pygame.Surface: Surface $cs \times cs$ với lưới kẻ mờ trên nền tím đậm.

Phương thức `_proc_proj(radius, inner, outer)` (dòng 835)

Tạo procedural projectile sprite hình tròn glow làm fallback.

Args:

`radius (int)`: Bán kính lõi đạn (pixel).

`inner (tuple)`: Màu RGB lõi trong (vd. (200, 100, 255)).

`outer (tuple)`: Màu RGBA vòng ngoài glow (vd. (240, 180, 255)).

Returns:

pygame.Surface: Surface vuông ($radius*3 \times radius*3$) với SRCALPHA, vẽ vòng ngoài mờ và lõi sáng ở trung tâm.

File: `ui/upgrade_menu.py`

Lớp `UpgradeMenu` (dòng 8)

Menu nâng cấp tower, hiển thị node hiện tại và các nâng cấp sẵn có.

Attributes:

`tower`: Tower object đang được upgrade

`upgrade_tree`: UpgradeTree chứa cây nâng cấp

`current_node_id`: ID của node hiện tại trong cây

`available_upgrades`: Danh sách SkillNode có thể nâng cấp

`is_visible`: Menu đang hiển thị hay không

Phương thức `__init__(self, tower, upgrade_tree)` (dòng 19)

Khởi tạo UpgradeMenu.

Args:

`tower`: Tower object cần nâng cấp

`upgrade_tree`: UpgradeTree chứa cây nâng cấp

Phương thức `_init_fonts(self)` (dòng 53)

Khởi tạo fonts cho menu.

Phương thức `_update_upgrades(self)` (dòng 62)

Cập nhật danh sách nâng cấp có sẵn từ node hiện tại.

Phương thức `show(self)` (dòng 67)

```
Hiển thị menu upgrade.
```

Phương thức `hide(self)` (dòng 71)

```
Ẩn menu upgrade.
```

Phương thức `get_clicked_upgrade(self, pos)` (dòng 75)

```
Kiểm tra xem click có trùng nút nâng cấp nào không.
```

Args:

`pos: (x, y)` vị trí click

Returns:

`SkillNode` của nâng cấp được click, hoặc "DEMOLISH" nếu click nút phá, hoặc None

Phương thức `try_upgrade(self, upgrade_node, money)` (dòng 100)

```
Thử nâng cấp tower.
```

Args:

`upgrade_node (SkillNode): SkillNode` muốn nâng cấp.

`money (int): Tiền` hiện tại của player.

Returns:

`tuple[bool, int, str]:`

- `bool`: True nếu nâng cấp thành công, False nếu thất bại.
- `int`: Số tiền còn lại sau nâng cấp (đã trừ cost nếu thành công).
- `str`: Thông báo kết quả (thành công hoặc lý do thất bại).

Phương thức `get_demolish_refund(self)` (dòng 126)

```
Tính toán tiền hoàn lại khi phá tháp (50% cost).
```

Returns:

`int`: 50% của `tower.cost` (làm tròn xuống)

Phương thức `draw(self, screen, player_money, camera_x, camera_y, current_level)` (dòng 134)

```
Vẽ menu upgrade trên màn hình.
```

Args:

`screen`: pygame surface để vẽ

`player_money`: Số tiền hiện tại của player để kiểm tra nâng cấp khả dụng

`camera_x`: Camera horizontal offset (world → screen conversion)

`camera_y`: Camera vertical offset (world → screen conversion)

File: `ui/right_panel.py`

Lớp `RightPanel` (dòng 169)

```
Panel HUD bên phải – hiển thị sprite, stats, và upgrade tree của tower được chọn.
```

Người chơi nhấn phím 1-4 để chọn loại tower; click vào node trong cây để xem trước stats của tower nâng cấp đó.

Attributes:

```
upgrade_trees (dict): Map tree_key → UpgradeTree (BasicNode/IceWall/RadarNode).
selected_type (str): Tên class tower đang được chọn (vd. "BasicNode").
preview_type (str | None): Tên class tower đang được preview qua node click.
preview_node (SkillNode | None): Node upgrade đang được preview.
node_rects (dict[str, pygame.Rect]): Map node_id → Rect để hit-test click.
hovered_node (str | None): node_id đang được hover, None nếu không hover.
current_level (int): Level game hiện tại – dùng để kiểm tra tower unlock.
panel_x (int): Tọa độ pixel x bắt đầu của panel.
panel_y (int): Tọa độ pixel y bắt đầu của panel.
panel_w (int): Chiều rộng panel (pixel).
panel_h (int): Chiều cao panel (pixel).
```

Usage::

```
panel = RightPanel(upgrade_trees)
panel.set_tower_type("BasicNode")
panel.draw(screen)
panel.handle_event(event)
```

Phương thức `__init__(self, upgrade_trees)` (dòng 211)

Khởi tạo RightPanel với upgrade trees và giá trị mặc định.

Args:

```
upgrade_trees (dict): Map tree_key → UpgradeTree do game.py tạo khi load level.
```

Side effects:

- Gọi `_init_dims()` để tính kích thước panel từ settings.
- Fonts được lazy-init lần đầu khi `draw()` được gọi.

Phương thức `_init_dims(self)` (dòng 235)

Tính tọa độ và kích thước panel dựa trên settings.

Side effects:

- Đặt `panel_x`, `panel_y`, `panel_w`, `panel_h` từ `GRID_COLS`, `CELL_SIZE`, `SCREEN_WIDTH/HEIGHT`.

Phương thức `_init_fonts(self)` (dòng 247)

Khởi tạo fonts lần đầu (lazy init). Không làm gì nếu đã khởi tạo.

Side effects:

- Tạo `f_title`, `f_section`, `f_body`, `f_desc`, `f_small`, `f_node` từ `pygame.font.SysFont`.
- Fallback về `Font(None, 16)` nếu "courier new" không tìm thấy.
- Đặt `_fonts_ready = True` sau khi hoàn tất.

Phương thức `set_tower_type(self, tower_class_name)` (dòng 271)

Đặt loại tower đang được chọn và reset preview nếu type thay đổi.

Args:

`tower_class_name (str)`: Tên class tower (vd. "BasicNode", "IceWall").

Side effects:

- Cập nhật `selected_type`, xóa `preview_type/preview_node`.
- Đặt `_dirty = True` để panel re-render.

Phương thức `set_level(self, level)` (dòng 287)

Cập nhật level game hiện tại để kiểm tra trạng thái unlock tower.

Args:

`level (int)`: Level game hiện tại.

Side effects:

- Cập nhật `current_level`; đặt `_dirty = True` nếu level thay đổi.

Phương thức `invalidate(self)` (dòng 300)

Force re-render next frame (call after upgrades that change unlock state).

Phương thức `handle_event(self, event)` (dòng 304)

Xử lý sự kiện chuột; trả về True nếu event được panel tiêu thụ.

Args:

`event (pygame.event.Event)`: Sự kiện pygame (MOUSEMOTION hoặc MOUSEBUTTONDOWN).

Returns:

- `bool`: True nếu con trỏ nằm trong panel và event đã được xử lý.
- False nếu con trỏ ngoài panel hoặc event không liên quan.

Side effects:

- MOUSEMOTION: cập nhật `hovered_node`; đặt `_dirty` nếu hover thay đổi.
- MOUSEBUTTONDOWN (left): gọi `_on_node_click()` nếu click trúng node.

Phương thức `draw(self, screen)` (dòng 343)

Vẽ panel lên màn hình; re-render cache nếu `_dirty = True`.

Args:

`screen (pygame.Surface)`: Màn hình game do game.py truyền vào.

Side effects:

- Gọi `_render()` và lưu vào `_surf` khi `_dirty`.
- Blit `_surf` vào (`panel_x`, `panel_y`) trên screen.

Phương thức `_render(self)` (dòng 361)

Vẽ toàn bộ nội dung panel vào Surface mới và trả về.

Returns:

pygame.Surface: Surface kích thước panel_w × panel_h đã vẽ đầy đủ.

Side effects:

- Gọi các hàm `_draw_*` theo thứ tự từ trên xuống.

Phương thức `_draw_instruction(self, surf, y)` (dòng 388)

Vẽ dòng hướng dẫn phím tắt ở đầu panel.

Args:

surf (pygame.Surface): Surface đang render.

y (int): Tọa độ y bắt đầu vẽ.

Returns:

int: Tọa độ y sau khi vẽ xong section này.

Phương thức `_draw_header(self, surf, y)` (dòng 423)

Vẽ card header: sprite tower, tên, cost/lock badge, và description.

Args:

surf (pygame.Surface): Surface đang render.

y (int): Tọa độ y bắt đầu vẽ card.

Returns:

int: Tọa độ y phía dưới card (card.bottom).

Phương thức `_blit_sprite(self, surf, key, x, y, sz, accent)` (dòng 493)

Vẽ sprite tower (hoặc fallback hình tròn) với glow backdrop.

Args:

surf (pygame.Surface): Surface đang render.

key (str | None): Sprite key trong cache (vd. "tower_basic").

x (int): Tọa độ x vẽ sprite.

y (int): Tọa độ y vẽ sprite.

sz (int): Kích thước sprite (sz × sz pixel).

accent (tuple): Màu RGB accent của tower – dùng cho glow backdrop.

Phương thức `_draw_stats(self, surf, y)` (dòng 526)

Vẽ bảng stats của tower (hoặc tower đang preview).

Nếu preview_node là stat_buff, hiển thị giá trị gốc + delta (highlight xanh).

Args:

surf (pygame.Surface): Surface đang render.

y (int): Tọa độ y bắt đầu section stats.

Returns:

int: Tọa độ y phía dưới sau khi vẽ xong tất cả dòng stats.

Phương thức `_draw_tree(self, surf, y)` (dòng 594)

Vẽ toàn bộ upgrade tree của tower đang chọn: `header`, `connectors`, `nodes`.

Args:

`surf` (pygame.Surface): Surface đang render.
`y` (int): Tọa độ y bắt đầu section upgrade tree.

Side effects:

- Xóa và tái tạo `node_rects` cho hit-testing click/hover.
- Gọi `_layout()`, `_draw_connections()`, `_draw_node()` theo thứ tự.

Phương thức `_layout(self, root, start_y)` (dòng 635)

BFS level layout: phân bổ đều các node của mỗi level theo chiều ngang panel.

Args:

`root` (SkillNode): Node gốc của UpgradeTree.
`start_y` (int): Tọa độ y bắt đầu vẽ level đầu tiên.

Returns:

`dict[str, tuple[int, int]]`: Map `node_id` $\rightarrow$ (x, y) pixel tọa độ góc trên-trái node.

Phương thức `_draw_connections(self, surf, node, positions)` (dòng 670)

Vẽ đường nối hình chữ L từ node cha đến từng node con (đệ quy).

Args:

`surf` (pygame.Surface): Surface đang render.
`node` (SkillNode): Node hiện tại (xuất phát điểm của đường nối).
`positions` (dict[str, tuple[int, int]]): Map `node_id` $\rightarrow$ (x, y) từ `_layout()`.

Side effects:

- Vẽ 3 đoạn thẳng màu CONN cho mỗi cặp cha-con: xuống $\rightarrow$ ngang $\rightarrow$ lên.

Phương thức `_draw_node(self, surf, node, nx, ny, accent)` (dòng 695)

Vẽ một node trong upgrade tree với icon, tên, cost/badge, và trạng thái màu sắc.

Args:

`surf` (pygame.Surface): Surface đang render.
`node` (SkillNode): Node cần vẽ.
`nx` (int): Tọa độ x góc trên-trái của node box.
`ny` (int): Tọa độ y góc trên-trái của node box.
`accent` (tuple): Màu RGB accent của tower gốc – không dùng trực tiếp ở đây nhưng truyền từ `_draw_tree` để nhất quán với phong cách màu.

Side effects:

- Thêm `node.node_id` $\rightarrow$ Rect vào `self.node_rects` để hit-test click/hover.

Phương thức `_draw_sep(self, surf, y)` (dòng 791)

Vẽ đường kẻ ngang phân cách giữa các section.

Args:

`surf (pygame.Surface):` Surface đang render.
`y (int):` Tọa độ y của đường kẻ.

Returns:

`int:` Cùng giá trị y (caller dùng để tính offset tiếp theo).

Phương thức `_blit_wrapped(self, surf, text, font, color, x, y, max_w)` (dòng 805)

Vẽ văn bản tự động xuống dòng khi vượt quá max_w pixel.

Args:

`surf (pygame.Surface):` Surface đang render.
`text (str):` Chuỗi văn bản cần vẽ.
`font (pygame.font.Font):` Font dùng để render.
`color (tuple):` Màu RGB chữ.
`x (int):` Tọa độ x bắt đầu mỗi dòng.
`y (int):` Tọa độ y dòng đầu tiên.
`max_w (int):` Chiều rộng tối đa (pixel) trước khi xuống dòng.

Phương thức `_on_node_click(self, node_id)` (dòng 833)

Xử lý click vào node: toggle preview hoặc xóa preview nếu click lại node đó.

Args:

`node_id (str):` ID của node vừa được click.

Side effects:

- Nếu node đang preview được click lại: xóa `preview_type` và `preview_node`.
- Nếu click node khác: đặt `preview_node = node`, `preview_type = node.tower_class`.
- Không làm gì nếu node bị khóa theo level (level-locked).
- Đặt `_dirty = True` để panel re-render.

File: game.py

Lớp Game (dòng 65)

Lớp điều phối toàn bộ vòng đời game – khởi tạo, cập nhật, và vẽ.

Là điểm kết nối duy nhất giữa tất cả hệ thống: GridGraph, Malware, Tower, Projectile, SpatialHash. main.py chỉ cần: `Game().run()`.

Attributes:

`screen (pygame.Surface):` Cửa sổ game kích thước `SCREEN_WIDTH × SCREEN_HEIGHT`.
`clock (pygame.time.Clock):` Đồng hồ giới hạn FPS và cung cấp dt.
`running (bool):` False → thoát vòng lặp `run()`.
`game_over (bool):` True → server bị phá, dừng `update()`, hiện "GAME OVER".
`victory (bool):` True → hết tất cả wave, dừng `update()`, hiện "YOU WIN!".
`selected_tower (class):` Loại tower đang chọn (BasicNode/IceWall/RadarNode).

```

config (dict): Dữ liệu JSON của level hiện tại.
graph (GridGraph): Bản đồ lưới – CellType, pathfinding, spawn weights.
spatial_hash (SpatialHash): Tra cứu malware trong tầm bắn.
server (Server): Mục tiêu chính – có HP, animation, vẽ được.
malwares (list[Malware]): Tất cả malware đang sống trên map.
towers (list[Tower]): Tất cả tower đã xây.
projectiles (list[Projectile]): Tất cả đạn đang bay.
undo_stack (CustomStack): Lịch sử xây tower để Ctrl+Z. Mỗi entry:
    {"pos": (row,col), "tower": Tower, "cost": int}.
money (int): Tiền hiện tại của người chơi.
wave_index (int): Chỉ số wave đang chạy (0-based, tính từ config["waves"]).
spawn_queue (CustomQueue): Tên malware cần spawn trong wave hiện tại.
spawn_timer (float): Đếm ngược (giây) đến lần spawn tiếp theo.
spawn_interval (float): Khoảng cách giữa hai lần spawn (giây), từ JSON.
font (pygame.font.Font): Font monospace 18px cho HUD.
font_big (pygame.font.Font): Font monospace 36px cho màn hình kết thúc.

Usage::

game = Game()
game.run() # vòng lặp chính – block cho đến khi thoát

```

Phương thức `__init__(self, level, screen, managed)` (dòng 101)

```

Khởi tạo Game instance với screen được truyền từ GameManager.

Args:
    level (int): Level number to load (default 1)
    screen: pygame.Surface được tạo từ GameManager (required)
    managed (bool): True nếu được quản lý bởi GameManager (không xử lý ESC)

Side effects:
    - Gọi sprites.init() để load sprites nếu chưa initialized
    - Gọi self.load_level(level) để nạp bản đồ

```

Phương thức `load_level(self, level_num)` (dòng 150)

```

Đọc file JSON và khởi tạo toàn bộ trạng thái game cho level đó.

Args:
    level_num (int): Số level cần load (1-5). File tương ứng:
        data/levels/level{level_num}.json.

Side effects:
    - Đọc và parse JSON vào self.config.
    - Tạo GridGraph từ config["grid"], config["rows"], config["cols"].
    - Khởi tạo SpatialHash với kích thước map thực tế từ JSON.
    - Reset tất cả danh sách: malwares, towers, projectiles, undo_stack.
    - Đặt lại money và server_hp theo config và settings.
    - Gọi _start_wave(0) để bắt đầu wave đầu tiên ngay.

```

- Tạo font cho HUD.

Note:

Truyền `grid_rows/cols` từ JSON vào `SpatialHash` thay vì `settings` để hỗ trợ map nhiều kích thước khác nhau giữa các level.

Phương thức `_init_upgrade_trees(self)` (dòng 225)

Khởi tạo upgrade trees cho các loại tower.

Phương thức `_start_wave(self, wave_index)` (dòng 242)

Nạp dữ liệu wave vào `spawn_queue`, thiết lập portal, và đặt lại spawn timer.

Args:

`wave_index (int)`: Chỉ số wave trong `self.config["waves"]` (0-based).

Side effects:

- Đặt `self.victory = True` và return nếu `wave_index >=` số lượng wave.
- Nạp danh sách tên malware vào `self.spawn_queue`.
- Cập nhật `self.spawn_interval` từ `wave_data["interval_seconds"]`.
- Reset `self.spawn_timer` về 0.0 để spawn ngay lập tức khi wave bắt đầu.
- Chọn ngẫu nhiên `num_portals` ô từ SPAWN cells làm portal mới.
- Chuyển portal cũ từ SPAWN về PATH trước khi set portal mới.

Note:

Portal random giúp tránh cho phép xây tháp chặn spawn point cố định. Số lượng portal theo `wave_data["portal_count"]`, mặc định từ config.

Usage:

Gọi tự động trong `load_level()` và khi wave cũ kết thúc (`_check_wave_complete()` phát hiện `spawn_queue` rỗng và malwares rỗng).

Phương thức `_init_bomb_timers(self)` (dòng 288)

Khởi tạo thời gian spawn ngẫu nhiên cho tất cả bomb trong wave.

Mỗi bomb rơi tại một thời gian ngẫu nhiên trong khoảng `[0, max_wave_duration]`.
`max_wave_duration = số lượng enemy × spawn_interval`.

Phương thức `_init_stealth_timers(self)` (dòng 304)

Tính ngẫu nhiên thời điểm bắt đầu mỗi đợt tàng hình.

Cửa sổ hợp lệ: [đầu wave 2, cuối wave 4 - 15s - `stealth_duration`].

Phương thức `_update_stealth(self, dt)` (dòng 331)

Kích hoạt đợt tàng hình và đếm ngược `invisible_duration`.

Phương thức `run(self)` (dòng 354)

Vòng lặp game chính – chạy cho đến khi `self.running = False`.

Side effects:

- Gọi `clock.tick(FPS)` để giới hạn frame rate và lấy `dt`.
- Gọi `handle_events()`, `update(dt)`, `draw()` mỗi frame.
- Khi `self.running = False`, gọi `pygame.quit()` để dọn dẹp.

Note:

`dt = clock.tick(FPS) / 1000.0` chuyển milliseconds → giây.
Mọi chuyển động dùng "`tốc độ × dt`" → frame rate không ảnh hưởng đến tốc độ game thực tế (frame-rate independent movement).
`update()` bị bỏ qua khi `game_over` hoặc `victory` – chỉ vẽ màn hình kết thúc.

Phương thức `handle_events(self, events)` (dòng 383)

Xử lý tất cả sự kiện từ bàn phím và chuột trong frame hiện tại.

Args:

`events`: Danh sách events (nếu None, sẽ gọi `pygame.event.get()` tự động)

Side effects:

- Gọi `self.undo()` khi nhấn `Ctrl+Z`.
- Cập nhật `self.selected_tower` khi nhấn phím 1/2/3.
- Gọi `self.place_tower(row, col)` khi click chuột trái vào vùng lưới.
- Toggle `UpgradeMenu` khi click vào tower đã xây.
- Xử lý click upgrade button khi menu hiển thị (gọi `_apply_upgrade()`).

Note:

Click chuột chỉ được xử lý khi `my < GRID_ROWS * CELL_SIZE`
(trong vùng lưới, không phải thanh HUD phía dưới).
Tọa độ ô: `row = my // CELL_SIZE`, `col = mx // CELL_SIZE`.
Ưu tiên kiểm tra: upgrade menu button → tower click → place tower.

Phương thức `_update_camera(self, dt)` (dòng 494)

Update camera position based on WASD key presses.

W: pan up (camera_y decreases)
S: pan down (camera_y increases)
A: pan left (camera_x decreases)
D: pan right (camera_x increases)

Camera is clamped to map boundaries.

Playable area is `SCREEN_HEIGHT - HUD_HEIGHT` (HUD is fixed at bottom).

Phương thức `_world_to_screen(self, world_x, world_y)` (dòng 529)

Convert world coordinates to screen coordinates using camera offset.

Args:

`world_x`, `world_y`: World pixel coordinates

Returns:

Tuple of (screen_x, screen_y)

Phương thức `update(self, dt)` (dòng 544)

Cập nhật toàn bộ trạng thái game mỗi frame theo thứ tự cố định.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Spawn malware mới theo wave timer.
- Di chuyển tất cả malware, cập nhật SpatialHash, xử lý server damage.
- Tower chọn mục tiêu và bắn Projectile mới.
- Di chuyển đạn, gây damage, xóa đạn đã kết thúc.
- Kiểm tra chuyển wave nếu wave hiện tại đã xong.

Note:

Thứ tự quan trọng: spawner → malwares → towers → projectiles → wave check.
Spawner trước malwares để malware mới được di chuyển ngay trong frame spawn.
Towers sau malwares để query SpatialHash với vị trí đã cập nhật.

Phương thức `_update_burning_cells(self, dt)` (dòng 599)

Cập nhật thời gian cháy của các ô tường (tạm thời không cho đặt tháp).

Phương thức `_update_animations(self, dt)` (dòng 610)

Cập nhật animation timer cho portal và server.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật portal animation frame (`_portal_timer`, `_portal_frame`).
- Gọi `self.server.update(dt)` để cập nhật server animation.

Phương thức `_update_spawner(self, dt)` (dòng 626)

Xử lý logic spawn malware theo wave timer.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Giảm `self.spawn_timer` mỗi frame.
- Khi timer ≤ 0: reset timer, lấy tên malware đầu tiên từ `spawn_queue`, chọn vị trí spawn ngẫu nhiên có trọng số, gọi `spawn_malware()`.

Note:

Dùng `CustomQueue.dequeue()` để spawn theo thứ tự trong JSON (FIFO).


```
get_spawn_weight() trả về danh sách (weight, cell) – random.choices()
chọn ngẫu nhiên có trọng số: malware khó spawn gần server hơn.
```

Phương thức `_update_bombs(self, dt)` (dòng 664)

Xử lý spawn và cập nhật bomb mỗi frame.

Args:

dt: Thời gian frame (giây).

Side effects:

- Spawn bomb tại thời gian ngẫu nhiên (từ `_init_bomb_timers`).
- Cập nhật timer explosion cho từng bomb.
- Khi bomb phát nổ: stun tất cả towers + damage server.
- Xóa bomb đã chết.

Phương thức `_update_malwares(self, dt)` (dòng 728)

Di chuyển tất cả malware và xử lý kết quả (chết hoặc đến server).

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Gọi `malware.update(dt)` để di chuyển từng malware.
- Gọi `spatial_hash.update_position()` nếu malware đã di chuyển.
- Gọi `self.server.take_damage()` khi malware đến server; đặt `game_over` nếu server HP ≤ 0 .
- Cộng `self.money += malware.reward` khi malware chết.
- Xóa malware đã chết hoặc đến server khỏi `self.malwares` và `spatial_hash`.
- Collect projectiles từ ranged malware (`SlowSpy_Ranged`, `Spyware_Ranged`, `TrojanRanged`).

Note:

old_pos phải được lưu TRƯỚC `malware.update(dt)` để `update_position()` biết bucket cũ của malware.
Kiểm tra `has_reached_server()` TRƯỚC `is_dead()` – malware bị giết đúng lúc chạm server vẫn trừ HP server.

Phương thức `_update_bosses(self, dt)` (dòng 811)

Cập nhật tất cả boss: di chuyển, tấn công, kiểm tra chết.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật vị trí boss dựa trên movement.
- Xử lý tấn công AoE lên tháp (nếu có).
- Xử lý tấn công server (nếu có).
- Xóa boss đã chết khỏi `self.bosses`.

Phương thức `_get_tower_at(self, cell)` (dòng 923)

Tìm tower đang chiếm ô lưới chỉ định.

Args:

cell (tuple[int,int]): Tọa độ ô cần tìm (row, col).

Returns:

Tower | None: Đối tượng Tower tại ô đó, hoặc None nếu không có.

Phương thức `_on_tower_destroyed(self, tower)` (dòng 937)

Xử lý khi một tower bị phá hủy.

Chuyển ô tower về PATH, xóa khỏi danh sách, tính lại đường đi cho tất cả malware đang sống.

Args:

tower (Tower): Tower vừa bị phá hủy.

Side effects:

- Cập nhật graph cell từ TOWER → PATH.
- Xóa tower khỏi self.towers.
- Gọi `_calculate_path()` cho tất cả malware.

Phương thức `_update_radar_detection(self)` (dòng 964)

Cập nhật `is_radar_range` cho tất cả malware dựa trên RadarNode.

Duyệt qua tất cả RadarNode (và PoisonNode – kế thừa RadarNode).

Gom malware trong tầm vào set, rồi flag `is_radar_range` cho từng malware.

Phải gọi `SAU _update_malwares()` và `TRƯỚC _update_towers()`.

Phương thức `_update_towers(self, dt)` (dòng 981)

Cho từng tower query SpatialHash, thêm Bomb, chọn mục tiêu, bắn Projectile.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Gọi `spatial_hash.query_range()` để lấy candidates trong tầm bắn.
- Thêm Bomb trong tầm bắn vào candidates.
- Gọi `tower.update(dt, candidates)` – trả về Projectile hoặc None.
- Append Projectile vào `self.projectiles` nếu không None.

Note:

SpatialHash lọc candidates trong $O(1)$ trung bình → chỉ xây Heap từ số ít malware gần tower. Nếu dùng toàn bộ `self.malwares` → Heap tồn kém không cần thiết.

Bomb được thêm vào candidates để tower có thể bắn (ưu tiên cao hơn malware).

Phương thức `_update_walls(self, dt)` (dòng 1028)

Cập nhật tường - xóa tường hết hạn, khôi phục cell, quản lý cooldown.

Args:

dt: Thời gian frame (giây).

Side effects:

- Cập nhật timer duration cho từng wall.
- Khi wall hết hạn: khôi phục cell, bắt đầu cooldown.
- Tính lại đường đi cho tất cả malware.
- Xóa wall đã hết hạn.
- Giảm cooldown timer mỗi frame.

Phương thức `_update_projectiles(self, dt)` (dòng 1062)

Di chuyển đạn và xóa những đạn đã kết thúc vòng đời.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Gọi `proj.update(dt)` để di chuyển từng đạn.
- Gọi `proj.apply_hit(game)` khi `proj.has_hit()` để xử lý damage.
- Lọc `self.projectiles`, giữ lại chỉ những đạn chưa `has_hit()`.

Phương thức `_update_shock_effects(self, dt)` (dòng 1139)

Cập nhật các hiệu ứng shock từ `LightSpy` attacks.

Args:

dt (float): Thời gian frame tính bằng giây.

Side effects:

- Cập nhật animation frame cho mỗi shock effect.
- Xóa shock effects đã hết thời gian (`is_alive() = False`).

Phương thức `_check_wave_complete(self)` (dòng 1154)

Kiểm tra xem wave hiện tại đã xong chưa để chuyển sang wave tiếp.

Side effects:

- Tăng `self.wave_index` và gọi `_start_wave(wave_index)` nếu wave xong.
- `_start_wave()` sẽ đặt `self.victory = True` nếu hết tất cả wave.

Note:

Wave xong khi: `spawn_queue` rỗng (không còn malware cần spawn)
VÀ `self.malwares` rỗng (tất cả malware đã chết hoặc đến server)
VÀ `self.bosses` rỗng (tất cả boss đã chết).
Cả ba điều kiện phải thỏa đồng thời.

Phương thức `place_tower(self, row, col)` (dòng 1176)

Đặt tower tại ô (row, col) nếu hợp lệ và đủ tiền.

Args:

row (int): Hàng ô lưới được click.
col (int): Cột ô lưới được click.

Side effects:

- Trừ self.money theo cost của tower.
- Đổi ô (row, col) từ WALL → TOWER trong GridGraph.
- Thêm tower vào self.towers.
- Lưu action vào self.undo_stack để Ctrl+Z có thể hoàn tác.
- Gọi _calculate_path() cho tất cả Spyware/Ransomware đang sống vì lưới vừa thay đổi (tower mới có thể là target gần hơn).

Note:

Kiểm tra is_tower_placeable(): ô phải là WALL và không quá gần server (trong server_radius Chebyshev). Trojan/Worm không cần tính lại path vì chúng đi trên PATH, không qua WALL.

Phương thức place_wall(self, row, col) (dòng 1228)

Đặt tường tạm thời tại ô (row, col) nếu hợp lệ và cooldown hết.

Args:

row (int): Hàng ô lưới được click.
col (int): Cột ô lưới được click.

Side effects:

- Đổi ô (row, col) từ PATH → WALL (celltype 0).
- Lưu original_cell (PATH).
- Thêm wall vào self.walls.
- Tính lại đường đi cho tất cả malwares (vì PATH đã chuyển thành WALL).
- Bắt đầu cooldown timer để ngăn đặt tường liên tục.

Note:

Tường chỉ có thể đặt trên ô PATH khi cooldown = 0.
Sau WALL_DURATION, tường biến mất và bắt đầu cooldown.
Tường không tốn tiền (là cơ chế bảo vệ tạm thời).

Phương thức _apply_upgrade(self, tower, upgrade_node) (dòng 1273)

Áp dụng upgrade cho tower.

Args:

tower: Tower object cần upgrade
upgrade_node: SkillNode chứa thông tin upgrade

Side effects:

- Nếu stat_buff: cập nhật stats tower (damage, fire_rate, etc.)
- Nếu tower_upgrade: thay thế tower bằng tower class mới, giữ nguyên vị trí

Note:

fire_rate được xử lý đặc biệt: cập nhật cả tower.fire_rate VÀ tower.original_fire_rate để đảm bảo slow/stun effect reset đúng khi hết (dùng original_fire_rate).
Các stat khác chỉ dùng setattr cộng thêm giá trị.

Phương thức `_demolish_tower(self, tower)` (dòng 1347)

Phá tháp và hoàn lại 50% chi phí xây dựng.

Args:

tower: Tower object cần phá

Side effects:

- Tính toán refund = 50% của tower.cost
- Thêm refund vào self.money
- Đổi ô tower từ TOWER → WALL trong GridGraph
- Xóa tower khỏi self.towers
- Xóa tower khỏi self.upgrade_menus
- Tính lại path cho tất cả tower-seeking enemies (trong case chúng đang tấn công tower này)

Note:

Khi tower bị phá, tất cả Spyware/Ransomware đang tấn công nó cần recalculate path vì target tower không còn tồn tại. Họ sẽ tìm tower gần nhất khác hoặc quay về A* đi tới Server.

Phương thức `_handle_boss_destruction(self, cells)` (dòng 1434)

Xử lý ô bị Final Boss phá: xóa tower nếu có, set PATH, spawn RiposteWare.

Args:

cells: List (row, col) từ Final.drain_spawn_queue()

Phương thức `undo(self)` (dòng 1468)

Thực hiện hoàn tác việc đặt tháp cuối cùng.

Phương thức `spawn_malware(self, malware_type, pos)` (dòng 1504)

Tạo Malware theo loại và thêm vào game.

Args:

malware_type (str): Tên loại malware – "trojan", "worm", "spyware", hoặc "ransomware". Phải là key trong MALWARE_FACTORY.
pos (tuple): Vị trí (row, col) ô lưới để spawn. Phải là ô PATH (get_spawn_weight() chỉ trả về ô PATH – luôn hợp lệ).

Side effects:

- Tạo instance Malware tương ứng tại pos với graph hiện tại.
- Thêm malware vào self.malwares.
- Đăng ký malware vào self.spatial_hash để tower có thể query.

Note:

Bỏ qua silently nếu malware_type không hợp lệ (không có trong MALWARE_FACTORY hoặc BOSS_FACTORY).

Malware.__init__() tự tính path từ pos → server_pos (hoặc tower gần nhất) ngay khi khởi tạo.

Boss.__init__() cũng tính path tương tự.

Phương thức `draw(self)` (dòng 1555)

Vẽ toàn bộ màn hình mỗi frame theo thứ tự layer từ dưới lên trên.

Side effects:

- Xóa frame trước bằng `screen.fill(COLOR_BG)`.
- Vẽ các layer theo thứ tự: `grid` → `towers` → `malwares` → `projectiles` → `HUD` → `game_over`.
- Gọi `pygame.display.flip()` để hiển thị frame đã vẽ lên màn hình.

Note:

Thứ tự vẽ quan trọng – layer sau đè lên layer trước:

`grid` (thấp nhất) → `towers` → `malwares` → `projectiles` → `HUD` (cao nhất).

Pygame dùng double buffering – vẽ vào buffer ẩn, `flip()` swap lên màn hình để tránh nhấp nháy (flickering).

Phương thức `_draw_tower_range(self)` (dòng 1584)

Vẽ vòng tròn range của tower khi upgrade menu đang mở.

Hiển thị bán kính tấn công của tower dưới dạng vòng tròn bán trong suốt.

Giúp người chơi hình dung được tầm hoạt động của tower.

Phương thức `_draw_upgrade_menus(self)` (dòng 1612)

Vẽ upgrade menus cho towers.

Phương thức `_draw_grid(self)` (dòng 1616)

Hệ thống kết xuất (Render) Y-Sorting thống nhất cho Tường, Quái, Tháp

Tất cả rendering đều áp dụng camera offset để hỗ trợ panning.

Phương thức `_get_cell_color(self, cell_type)` (dòng 1897)

Trả về màu RGB cho từng loại ô lưới.

Args:

`cell_type (int)`: Giá trị `CellType` (`Celltype.WALL`, `PATH`, `SERVER`, `SPAWN`, `TOWER`).

Returns:

`tuple`: Màu RGB (`r`, `g`, `b`) tương ứng từ settings.

`WALL` → `COLOR_WALL`, `PATH` → `COLOR_PATH`, `SERVER` → `COLOR_SERVER`,

`SPAWN` → `COLOR_SPAWN`, `TOWER` → `COLOR_WALL` (tower vẽ riêng bằng `Tower.draw()`),

Không nhận dạng được → `COLOR_BG`.

Phương thức `_draw_projectiles(self)` (dòng 1925)

Vẽ tất cả đạn đang bay lên màn hình – với camera offset.

Side effects:

- Gọi `proj.draw(screen, camera_offset_x, camera_offset_y)` cho từng projectile.

Note:

Gọi sau `_draw_grid()` để đạn hiển thị trên đầu tất cả objects.

Phương thức `_draw_hud(self)` (dòng 1939)

Vẽ thanh thông tin HUD ở dưới cùng màn hình.

Side effects:

- Vẽ nền HUD màu `COLOR_HUD_BG` từ `y=SCREEN_HEIGHT-HUD_HEIGHT` đến `SCREEN_HEIGHT`.
- Hiển thị: tiền hiện tại, HP server, số wave hiện tại/tổng.
- Hiển thị hướng dẫn phím (1=Basic, 2=Ice, 3=Radar, Ctrl+Z=Undo).
- Hiển thị tên tower đang chọn bằng màu nổi bật.

Note:

HUD nằm trong vùng `y = SCREEN_HEIGHT - HUD_HEIGHT → SCREEN_HEIGHT` (fixed at bottom).
Không bị ảnh hưởng bởi camera panning.

Phương thức `_draw_countdown(self)` (dòng 2013)

Vẽ countdown và thông báo Level ở giữa màn hình khi chờ wave.

Phương thức `_draw_game_over(self)` (dòng 2059)

Vẽ màn hình kết thúc (Game Over/Victory) với phong cách chuyên nghiệp.

Phương thức `_draw_stealth_notif(self)` (dòng 2125)

Vẽ thông báo tàng hình trong 2 giây đầu khi đợt tàng hình kích hoạt.

Phương thức `_draw_bomb_notif(self)` (dòng 2170)

Vẽ thông báo bomb xuất hiện trong 3 giây.

File: `main.py`

Lớp `GameManager` (dòng 13)

Quản lý trạng thái game: menu, level progression, save/load.

`GameManager` nhận input từ người chơi (click, key) và quản lý state transitions.

Game instance xử lý việc render và game logic cho mỗi level.

Attributes:

`state (str)`: Trạng thái ("START", "PLAYING", "LEVEL_COMPLETE", "GAME_OVER", "VICTORY")
`current_level (int)`: Level hiện tại (1-indexed)
`completed_levels (set)`: Tập hợp các level đã hoàn thành
`game (Game | None)`: Instance Game hiện tại (None khi ở menu)
`save_file (str)`: Đường dẫn file lưu tiến độ

Phương thức `main()` (dòng 893)

Khởi động Cypher Defense với GameManagerer.

Phương thức `__init__(self)` (dòng 27)

Khởi tạo GameManagerer và tải tiến độ đã lưu.

Phương thức `_load_menu_background(self)` (dòng 63)

Tải background ảnh cho menu từ data/sprites/Menu.

Phương thức `_load_progress(self)` (dòng 76)

Tải tiến độ từ file lưu (nếu tồn tại).

Phương thức `_save_progress(self)` (dòng 89)

Lưu tiến độ vào file.

Phương thức `_ub_insert(self, entry)` (dòng 100)

Upper-bound insert vào self.leaderboard (tăng dần theo score).

Phương thức `_get_max_levels(self)` (dòng 111)

Đếm số level có sẵn từ data/levels/.

Phương thức `handle_events(self)` (dòng 121)

Xử lý sự kiện: GameManagerer xử lý ESC/QUIT, Game xử lý những events khác.

Phương thức `update(self, dt)` (dòng 140)

Cập nhật trạng thái game.

Phương thức `draw(self)` (dòng 179)

Vẽ màn hình dựa trên state.

Phương thức `_handle_start_menu(self)` (dòng 209)

Xử lý input menu chính.

Phương thức `_handle_level_complete(self)` (dòng 263)

Xử lý input level hoàn thành khớp với UI mới.

Phương thức `_handle_game_over(self)` (dòng 292)

Xử lý input game over khớp với UI mới.

Phương thức `_handle_victory(self)` (dòng 316)

Xử lý input victory khớp với UI mới.

Phương thức `_handle_confirm_new_game(self)` (dòng 333)

Xử lý input hộp xác nhận New Game.

Phương thức `_draw_confirm_new_game(self)` (dòng 360)

Vẽ hộp xác nhận trước khi reset New Game.

Phương thức `_start_level(self)` (dòng 399)

Bắt đầu level hiện tại.

Phương thức `_draw_start_menu(self)` (dòng 409)

Vẽ menu chính chuyên nghiệp (kết hợp với ảnh nền).

Phương thức `_handle_tower_info(self)` (dòng 489)

Xử lý click trong màn hình Tower Codex.

Phương thức `_draw_tower_info(self)` (dòng 514)

Vẽ màn hình Tower Codex.

Phương thức `_codex_blit_wrapped(self, surf, text, color, x, y, max_w)` (dòng 635)

Word-wrap text onto surf using font_small.

Phương thức `_handle_leaderboard(self)` (dòng 651)

Xử lý click trong màn hình bảng xếp hạng.

Phương thức `_draw_leaderboard(self)` (dòng 678)

Vẽ màn hình bảng xếp hạng (hiển thị giảm dần).

Phương thức `_draw_level_complete(self)` (dòng 753)

Vẽ màn hình hoàn thành level chuyên nghiệp.

Phương thức `_draw_game_over(self)` (dòng 805)

Vẽ màn hình game over chuyên nghiệp.

Phương thức `_draw_victory(self)` (dòng 844)

Vẽ màn hình thắng toàn bộ game chuyên nghiệp.

Phương thức `run(self)` (dòng 882)

Chạy game loop chính.

File: `settings.py`

Phương thức `calculate_resolution_and_cell_size()` (dòng 241)

Tính toán SCREEN_WIDTH, SCREEN_HEIGHT, CELL_SIZE dựa trên resolution thực tế.

Returns:

```
tuple: (screen_width, screen_height, cell_size)
```

Logic:

```
- CELL_SIZE = min(
    screen_width / GRID_COLS,
    (screen_height - HUD_HEIGHT) / GRID_ROWS
)
- Đảm bảo map vừa khít màn hình mà không bị cắt
```

Phương thức `is_tower_unlocked(key, current_level)` (dòng 324)

Trả về True nếu tower/node đã được mở khóa ở level hiện tại.